

Faculty of Engineering and Architecture
Department of Computer Engineering
CEN 302 — Software Engineering

Reservio Hotel Management SaaS Platform

Requirements Specification

May 21st, 2026

Prepared by:

Klea Hila
Ina Ndoni
Eleana Zharkalli
Jorida Vrusho
Orest Paja
Licern Beqiri
Joni Begaj

Received by:

M.Sc Shkumbin Fida
M.Sc Megi Nako

Table of Contents

- 1. Introduction**
Project Overview · Purpose · Scope · Definitions · References
 - 2. Overall Description**
Product Perspective · Features · User Classes · Constraints
 - 3. User Stories & Acceptance Criteria**
All 26 User Stories with Acceptance Tests
 - 4. UML Diagrams**
Use Case · Activity · Sequence · State Diagrams
 - 5. System Architecture**
ERD · Component Architecture · Deployment · Non-Functional Requirements
 - 6. Class Diagrams & Data Dictionary**
All Prisma Entity Models with Field Descriptions
 - 7. Data Flow Diagrams**
Guest · Staff · Housekeeper · Hotel Admin Portals
 - 8. Detailed User Story Specifications**
Preconditions · Main Flow · Alternative & Exception Flows
 - 9. API Endpoint Catalogue**
All Express Routes Organised by Module
 - 10. Risk Register**
14 Identified Risks with Likelihood, Impact & Mitigation
 - 11. Glossary**
Technical Terms and Domain-Specific Definitions
 - 12. Future Work**
Planned Deployment · AI Chatbot · Fake Booking Detection
-

1. Introduction

1.1 Project Overview

Reservio is a multi-tenant hotel management Software-as-a-Service (SaaS) platform being developed as a 3rd year Bachelor's software engineering project. The platform is designed to allow any hotel, boutique, chain, hostel, or resort to subscribe and immediately access a fully operational hotel management workspace without any installation or technical setup. Hotels today typically rely on a fragmented set of tools: one system for reservations, another for housekeeping coordination, another for guest billing. Reservio consolidates all of these into a single cloud-hosted, subscription-based platform. Each subscribing hotel receives its own isolated workspace with its own staff accounts, room inventory, guest data, and configuration all hosted on shared infrastructure managed by the Reservio platform. The system supports five distinct user roles: Guest, Receptionist/Staff, Housekeeper, Hotel Administrator, and Super Administrator. Each role has its own dedicated interface and permissions, all served from the same codebase and differentiated through role-based access control enforced at the API layer.

1.2 Purpose of This Document

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for Reservio. It serves as the primary reference for the development team, the project supervisor, and any other stakeholder who needs to understand what the system does, how it is structured, and what constraints it operates under. This document covers all ten chapters including the full UML diagram suite: Use Case diagrams (with stick-figure UML actors), Activity diagrams, Sequence diagrams, State diagrams (with entry/do/exit action compartments), Entity-Relationship Diagram, Component Architecture diagram, and Deployment diagram.

1.3 Scope

Product Name: Reservio Product Type: Multi-tenant SaaS web application Delivery Format: Browser-based, mobile-first, hosted on Railway

Reservio will provide the following high-level capabilities:

- Guest self-service room browsing, availability search, reservation booking, parking and sunbed reservation, in-app service ordering, and real-time notifications
- Staff operations reservation management, guest check-in and check-out, room status control, and housekeeping priority overrides

- Housekeeping management auto-prioritised task queue with real-time updates and room completion marking
- Hotel administration room and staff account management, dynamic pricing configuration, analytics dashboard, and parking lot management
- Platform administration tenant onboarding, subscription plan management, and platform-wide analytics for the Reservio Super Admin

The following are explicitly out of scope for the current version:

- Payment processing billing and subscription payments are referenced conceptually but no payment gateway will be integrated in the academic version
- Native mobile applications the system is mobile-responsive but delivered as a web application only
- Third-party channel integrations no connection to booking platforms such as Booking.com or Expedia
- AI features a virtual concierge chatbot and anomaly detection service are proposed but not committed; inclusion is subject to teacher approval and Sprint 5 capacity

1.4 Definitions, Acronyms & Abbreviations

Term	Definition
SaaS	Software-as-a-Service software delivered over the internet on a subscription basis, hosted by the provider
Multi-tenancy	Architecture where a single application instance serves multiple customers with isolated data per tenant
Tenant	A subscribing hotel organisation with its own isolated workspace within the Reservio platform
tenantId	Unique identifier assigned to each tenant, used to scope all database queries and ensure data isolation
JWT	JSON Web Token carries userId, tenantId, and role for stateless authentication; 7-day expiry
SRS	Software Requirements Specification this document
SPA	Single Page Application React/Vite frontend serving all five role-based portals from one codebase
API	Application Programming Interface Express.js HTTP endpoints (REST architecture)
ORM	Object-Relational Mapper Reservio uses Prisma with a tenant-scoped client extension

Term	Definition
RBAC	Role-Based Access Control enforced via roleGuard middleware on every protected route
RAG	Retrieval-Augmented Generation proposed AI feature (pending teacher approval)
WebSocket	Full-duplex protocol via Socket.io for real-time notifications; tenant-scoped rooms
CI/CD	Continuous Integration/Deployment GitHub Actions pipeline that deploys to Railway on every push to main
Feature Gate	Middleware restricting API endpoints to Basic, Premium, or Custom plan tenants only
Lock (5-min)	Temporary reservation hold on parking/sunbed spot to prevent concurrent double-booking
NFR	Non-Functional Requirement a requirement that describes system quality rather than specific behaviour

1.5 References

Reference	Description / Location
IEEE Std 830-1998	IEEE Recommended Practice for Software Requirements Specifications
Reservio Project Proposal	Full SaaS proposal with architecture, user stories, sprint plan
v2.0	GitHub /docs/proposal
Updated_User_Stories.pdf	Original user stories US-01 to US-22 GitHub /docs
Prisma Documentation	ORM documentation prisma.io/docs
React 18 Documentation	Frontend framework react.dev
Socket.io Documentation	WebSocket library socket.io/docs
Railway Documentation	Cloud hosting platform docs.railway.app

1.6 Document Overview

Ch.	Title	Contents
1	Introduction	Project overview, purpose, scope, definitions, references
2	Overall Description	Product perspective, features, user classes, operating environment, constraints, assumptions
3	User Stories	All 26 user stories with acceptance criteria grouped by role

Ch.	Title	Contents
4	UML Diagrams	Full UML suite: 5 UC + 26 Activity + 26 Sequence + 5 State + Architecture
5	Architecture	ERD, Component diagram, Deployment diagram, NFR table
6	Detailed Specifications	Structured US specs with main flow and exception flows
7	Data Dictionary	All Prisma models with fields, types, and constraints
8	API Catalogue	All Express.js endpoints with auth levels and descriptions
9	Risk Register	14 project risks with likelihood, impact, and mitigation
10	Glossary	28 technical terms defined

2. Overall Description

2.1 Product Perspective

Reservio is a new, self-contained SaaS product. It does not replace or extend any existing system, it is built from scratch as a cloud-native alternative to traditional property management software (PMS) such as Opera, Mews, or Cloudbeds, which are either too expensive, too complex, or not designed for multi-property SaaS delivery. From a SaaS infrastructure perspective, Reservio operates as a shared-infrastructure, isolated-data multi-tenant system. All hotels share the same application codebase and PostgreSQL database instance. Tenant isolation is enforced at the data layer through a `tenantId` column on every database table, and at the API layer through the `tenantResolver` middleware and Prisma client extension.

Actor	Relationship to Reservio	Access Point
Hotel Guest	End user of a hotel's Reservio-powered portal	hotel.reservio.com Guest Portal
Hotel Staff / Receptionist	Operator of the reservation and check-in/out system	hotel.reservio.com Staff Dashboard
Housekeeper	Operator of the automated task	hotel.reservio.com Housekeeper View
Hotel Administrator	Owner and configurator of the hotel's Reservio workspace	hotel.reservio.com Admin Panel

Actor	Relationship to Reservio	Access Point
Super Admin	Platform operator managing all admin.reser	vio.com Super
(Reservio team)	tenant accounts and subscriptions Admin Porta	l
Reservio Platform	Shared infrastructure serving all	Railway; wildcard subdomain *.reservio.com

2.2 Product Features & Subscription Tiers

Reservio uses a three-tier subscription model. Feature access is controlled by the featureGate middleware which reads the tenant's plan field on every API request.

Feature	Basic	Premium Custom User Stories
Room & reservation management	✓	✓ ✓ US-01 to US-05
Guest self-service portal	✓	✓ ✓ US-01, US-02, US-03, US-04
Staff dashboard & check-in/out	✓	✓ ✓ US-09 to US-13
Housekeeping task board	✓	✓ ✓ US-20, US-21
Room inventory	✓	✓ ✓ US-15

Feature	Basic	Premium Custom User Stories
management		
Staff account management	✓	✓ ✓ US-16
Parking lot management	✓	✓ ✓ US-19
Parking spot reservation map	✗	✓ ✓ US-06
Sunbed reservation map	✗	✓ ✓ US-22
Guest service marketplace	✗	✓ ✓ US-07
Dynamic pricing engine	✗	✓ ✓ US-17
Analytics dashboard &	✗	✓ ✓ US-18
exports		
Real-time WebSocket	✗	✓ ✓ US-08, US-14

Feature	Basic	Premium Custom User Stories
notifications		
Custom branding /	X	X ✓ US-24
white-label		
Max staff accounts	5	Unlimited Unlimited US-16
Max rooms	20	Unlimited Unlimited US-15

2.3 User Classes & Characteristics

Reservio serves five distinct user classes. Each class has different goals, technical expertise levels, and interaction patterns with the system. Access to features is determined by the user's assigned role, which is encoded in their JWT at login and enforced by the API's roleGuard middleware.

Guest

Attribute	Detail
Technical level	Non-technical consumer web user
Primary goals	Find and book a room, reserve amenities, order services, receive notifications
Session pattern	Short, task-focused sessions browse, book, confirm
Device	Primarily mobile; desktop supported (mobile-first at 375px minimum viewport)
Tenant scope	Scoped to the hotel they registered with; cannot access other hotels' data

Receptionist / Staff

Attribute	Detail
Technical level	Moderate comfortable with web-based tools
Primary goals	Confirm reservations, process check-ins and check-outs, update room status, flag urgent housekeeping
Session pattern	Extended sessions during shifts; frequent dashboard checks
Device	Desktop or tablet at front desk
Tenant scope	Scoped to their hotel tenant; cannot access other tenants' reservations

Housekeeper

Attribute	Detail
Technical level	Basic designed for minimal training
Primary goals	View task list, see which rooms are highest priority, mark rooms as cleaned
Session pattern	Short, frequent check-ins throughout their shift
Device	Mobile phone
Tenant scope	Scoped to their hotel tenant; only sees their assigned rooms

Hotel Administrator

Attribute	Detail
Technical level	Moderate business-oriented, not developer-level
Primary goals	Configure the hotel workspace, manage staff and inventory, monitor revenue and occupancy, adjust pricing
Session pattern	Regular but not constant configuration sessions and daily performance checks
Device	Desktop
Tenant scope	Full access to their own tenant's data and configuration; cannot access other tenants
Feature access	Determined by subscription tier (Basic, Premium, or Custom)

Super Administrator

Attribute	Detail
Technical level	High internal Reservio team member

Attribute	Detail
Primary goals	Approve and manage tenant accounts, control subscription plans, monitor platform health
Session pattern	Occasional on-demand when hotel onboarding or plan changes are needed
Device	Desktop
Tenant scope	Cross-tenant can read and modify any tenant's account metadata (not guest data)
Access point	admin.reservio.com separate from hotel-facing subdomains

2.4 Operating Environment

Component	Environment	Detail
Hosting platform	Railway Cloud PaaS	Supports *.reservio.com wildcard subdomains; auto-deploys from GitHub main branch
Frontend runtime	Modern web browser	Chrome, Firefox, Safari, Edge latest two major versions. No plugin required.
Frontend framework	React 18 + Vite	Compiled SPA; all role-based views from one codebase
Backend runtime	Node.js 20 LTS	Express.js API server; single Railway service
Database	PostgreSQL 16	Managed instance on Railway; shared across all tenants with row-level isolation
Real-time layer	Socket.io v4	WebSocket server co-located with Express; tenant-scoped rooms
Background jobs	node-cron v3	Runs within Node.js process; nightly pricing engine evaluation
CI/CD pipeline	GitHub Actions	Triggers on push to main; runs ESLint, Jest, then deploys via Railway hook

Component	Environment	Detail
Network	HTTPS / TLS	All traffic encrypted; Railway provides SSL termination with auto-renewing certificates
Device support	Desktop & mobile	Responsive using Tailwind CSS; guest screens mobile-first at 375px minimum width

2.5 Design & Implementation Constraints

Multi-Tenancy as a Core Constraint The system is designed multi-tenant from the ground up. Every database table must have a `tenantId` column. Every API query must be scoped to a tenant. The Prisma client extension (`tenantClient.js`) that auto-injects `tenantId` must be used for all database access; direct Prisma client calls that bypass tenant scoping are explicitly forbidden in the codebase.

Single Programming Language The entire stack frontend and backend must be implemented in JavaScript. This constraint reduces onboarding burden for a 7-person team. The only exception is the proposed AI anomaly detection feature, which would use a Python microservice and is explicitly flagged as such.

Free-Tier Infrastructure The system must operate within Railway's free or low-cost tiers. This constrains memory and the number of concurrent active tenants the system can support during the academic delivery period.

Stateless Authentication Authentication must be stateless, implemented using JSON Web Tokens. No server-side session store will be used. JWT expiry and refresh logic is handled client-side. Token payload includes `userId`, `tenantId`, and `role`.

2.6 Assumptions & Dependencies

ID Assumption	Impact if Invalid
A-01 Hotels have stable internet for real-time	Real-time features unavailable for
WebSocket communication	affected hotels
A-02 All users access via modern browser with	SPA would not function; no
JavaScript enabled	server-rendered fallback planned
A-03 Railway's wildcard subdomain feature remains	Fall back to path-based routing
available throughout the project	(<code>reservio.com/hotel-name</code>)

ID Assumption	Impact if Invalid
A-04 Team has Railway account with sufficient free-tier capacity for development and demo	Migrate to Render or Fly.io
A-05 Each hotel has at most one Hotel Administrator Onboarding at initial setup multi-admin	flow would need support from start
A-06 Guests register with and belong to a single hotel tenant	Unified cross-tenant guest identity requires significant extra architecture
A-07 Dynamic pricing evaluated nightly; sub-minute evaluation not required	Would require message queue or event-driven architecture beyond current stack

3. User Stories & Acceptance Criteria

All 26 user stories are documented below with acceptance criteria, grouped by user role. Each story follows the standard Agile format: “As a [role], I want [goal], so that [reason].”

Guest User Stories

US-01 Guest Account Registration

As a guest, I want to register a personal account on a hotel’s Reservio portal, so that I can manage my reservations and booking history for that property.

Acceptance Criteria:

- Registration page is accessible at the hotel’s subdomain without login
- Full name, email, and password are required
- Email must be unique within the tenant
- On success, account is created scoped to that hotel’s tenant and user can log in

US-02 Guest Login

As a guest, I want to log in with email and password, so that I can access a personalized dashboard showing my upcoming stays.

Acceptance Criteria:

- Login page accepts email and password
- JWT issued contains tenantId, userId, and role
- Correct credentials redirect to the guest dashboard with upcoming stays
- Incorrect credentials show a clear error message

US-03 Browse Available Rooms

As a guest, I want to browse available rooms with photos, descriptions, room type, and price per night, so that I can find a room matching my preferences.

Acceptance Criteria:

- Room listings are scoped to the current hotel tenant
- Each listing shows: room type, price per night, description, and photos
- Guest can browse without logging in

US-04 Check Room Availability

As a guest, I want to enter check-in and check-out dates to filter rooms, so that I only see rooms available during my stay.

Acceptance Criteria:

- User can enter check-in and check-out dates
- System filters rooms available for that period within the tenant
- Rooms already booked for those dates are excluded
- Room blocks set by admin are also excluded

US-05 Make a Reservation

As a guest, I want to book a selected room for chosen dates, so that I can reserve accommodation for my trip.

Acceptance Criteria:

- User selects a room and confirms dates
- System validates availability in real time
- Reservation is saved scoped to the tenant
- User receives a confirmation message with reference code

US-06 Reserve a Parking Spot

As a guest, I want to reserve a parking spot from a visual lot map, so that my selection is locked for 5 minutes to prevent double-booking.

Acceptance Criteria:

- Visual parking map shows the hotel's spots (scoped to tenant)
- Green = available, Red = taken, Yellow = locked
- Selection locks for 5 minutes exclusively
- Lock released after 5 minutes if not confirmed

US-07 Order In-App Services

As a guest, I want to order services (spa, room service, airport transfer) through the marketplace, so that they are automatically billed to my room.

Acceptance Criteria:

- Marketplace lists tenant-configured services with pricing
- Guest can place orders for one or more services
- Cost automatically added to the guest's room bill
- Guest receives order confirmation

US-08 Receive Early Check-In Notification

As a guest, I want to receive a real-time notification when my room is cleaned, so that I can be ready for early check-in.

Acceptance Criteria:

- Notification sent via WebSocket when housekeeper marks room cleaned
- Notification scoped to tenant WebSocket room
- Includes room number and status
- Guest can see this on their dashboard

US-22 Reserve a Sunbed

As a guest, I want to reserve a sunbed from a visual pool map, so that my spot is secured for a chosen time slot without double-booking.

Acceptance Criteria:

- Visual sunbed map scoped to hotel tenant
- Colour coding: green=available, red=taken, yellow=locked
- Guest selects time slot: Morning, Afternoon, or Full Day
- 5-minute lock consistent with US-06 parking mechanic

Receptionist/Staff User Stories

US-09 View Reservation Dashboard

As hotel staff, I want to view all bookings for our property, so that I can monitor occupancy and manage guests.

Acceptance Criteria:

- Dashboard scoped to staff's tenant
- Shows: guest name, room number, check-in/out dates, status
- Filterable by date and status
- Search by guest name

US-10 Confirm Pending Reservations

As hotel staff, I want to confirm a pending reservation so that its status updates and becomes visible to the guest.

Acceptance Criteria:

- Staff selects a pending reservation
- Marks it as confirmed
- Status updates immediately via WebSocket
- Badge changes to Confirmed (green)

US-11 Register Guest Check-In

As hotel staff, I want to register a guest check-in so that the room is marked as occupied.

Acceptance Criteria:

- Staff selects a confirmed reservation for today
- Marks as checked in
- Room status updates to OCCUPIED across the system
- WebSocket broadcasts room status change

US-12 Process Guest Check-Out

As hotel staff, I want to process a check-out so that housekeeping is triggered and guest access is cleared.

Acceptance Criteria:

- Staff selects an active reservation
- On check-out: housekeeping task auto-created, room set to AVAILABLE
- WebSocket broadcasts update

- Reservation status changes to CHECKED_OUT

US-13 Update Room Status

As hotel staff, I want to update a room's status so that changes are instantly visible across the system.

Acceptance Criteria:

- Staff can set: Available, Occupied, or Maintenance
- Changes broadcast via tenant-scoped WebSocket
- All connected devices update in real time

US-14 Override Housekeeping Priority

As hotel staff, I want to flag a room as urgent so that the housekeeper's task list re-sorts in real time.

Acceptance Criteria:

- Staff selects a room with an active housekeeping task
- Marks as urgent
- Housekeeper task list updates immediately via WebSocket
- Task moves to top of priority queue

Housekeeper User Stories

US-20 Receive Auto-Prioritised Task Queue

As a housekeeper, I want an auto-prioritised task list sorted by check-out time and urgency, so that the most critical rooms appear first.

Acceptance Criteria:

- Task list scoped to housekeeper's tenant
- Sorted by: urgency flag first, then priority, then creation time
- Updates in real-time via WebSocket when staff marks a task urgent

US-21 Mark Room as Cleaned

As a housekeeper, I want to mark a room as cleaned so that the front desk is notified and the guest is alerted if applicable.

Acceptance Criteria:

- Housekeeper marks room as cleaned task status changes to DONE

- Front desk view updates instantly via WebSocket
- Guest receives early check-in notification if confirmed reservation found for today
- Room status updates to AVAILABLE

Hotel Admin User Stories

US-15 Manage Hotel Rooms

As a hotel admin, I want to add, edit, or remove rooms so that our inventory is always accurate.

Acceptance Criteria:

- Admin can add rooms with type, description, price, and photos
- Admin can edit existing room details and pricing
- Admin can delete rooms (only if no active reservations exist)
- Admin can block date ranges for maintenance periods

US-16 Manage Staff Accounts

As a hotel admin, I want to create and manage staff accounts with role-based permissions so that staff have appropriate access.

Acceptance Criteria:

- Admin can create staff with roles: Receptionist, Housekeeper, Technician, Manager
- Admin can deactivate staff accounts immediately
- Username auto-generated from name

US-17 Configure Dynamic Pricing Rules

As a hotel admin, I want to configure pricing rules (e.g. +20% above 80% occupancy) so that pricing is optimised automatically. (Premium/Custom only)

Acceptance Criteria:

- Admin sets occupancy threshold and price adjustment percent
- System applies rule nightly via node-cron
- Real-time occupancy % visible to admin
- Only available on Premium and Custom plans

US-18 View Analytics Dashboard

As a hotel admin, I want a dashboard showing total bookings, daily revenue, and occupancy so that I can monitor performance. (Premium/Custom only)

Acceptance Criteria:

- Shows: total bookings, daily revenue, occupancy %
- Filterable by date range
- Exportable as CSV (Premium/Custom plans only)
- Calendar view showing daily occupancy percentage

US-19 Manage Parking Lot

As a hotel admin, I want to manage parking spots and pricing so that parking inventory is accurately tracked.

Acceptance Criteria:

- Admin can add/remove spots with label and price
- Admin can view current occupancy
- Per-spot pricing configurable
- Changes reflected immediately on the guest-facing map

Super Admin User Stories

US-23 Manage Tenant Accounts

As a super admin, I want to view, approve, suspend, or delete hotel tenant accounts so that the platform remains healthy.

Acceptance Criteria:

- Super Admin can see all tenants across the platform
- Can approve new registrations, suspend active tenants, or delete accounts
- Changes take effect immediately suspended tenants lose access

US-24 Manage Subscription Plans

As a super admin, I want to upgrade or downgrade a tenant's subscription plan so that their feature access is updated immediately.

Acceptance Criteria:

- Super Admin can change tenant plan (Basic, Premium, Custom)
- Feature gates update immediately on plan change
- Downgraded tenants lose access to Premium/Custom features instantly

US-25 View Platform-Wide Analytics

As a super admin, I want to view platform-level metrics so that I can monitor the health of the SaaS platform.

Acceptance Criteria:

- Dashboard shows: total active tenants, total bookings across platform
- Filterable by date range and plan tier
- Does not expose individual tenant guest data

US-26 Onboard a New Hotel

As a super admin, I want to manually onboard a hotel by creating their tenant account and subdomain so that they can start using the platform immediately.

Acceptance Criteria:

- Super Admin can create tenant account with name, subdomain, and plan
- Subdomain is provisioned and available immediately
- Hotel Admin receives email invitation to set up their account

4. UML Diagrams

This chapter presents the full suite of UML diagrams for the Reservio platform. Use Case diagrams are rendered with proper UML stick-figure actors and system boundary notation. State diagrams include entry, do, and exit action compartments per UML 2.5 standard. All diagrams were rendered to PNG via Playwright/Chromium.

4.1 Use Case Diagrams

Use case diagrams identify the actors in the system and the high-level operations each role can perform. Reservio has five actor types: Guest, Receptionist/Staff, Housekeeper, Hotel Admin, and Super Admin. Each is shown with a stick-figure actor, system boundary rectangle, and use cases in ovals. Include relationships are shown with dashed arrows.

4.1.1 Guest Module

The Guest module encompasses all self-service interactions: account creation, booking, amenity reservation, service ordering, and real-time notifications.

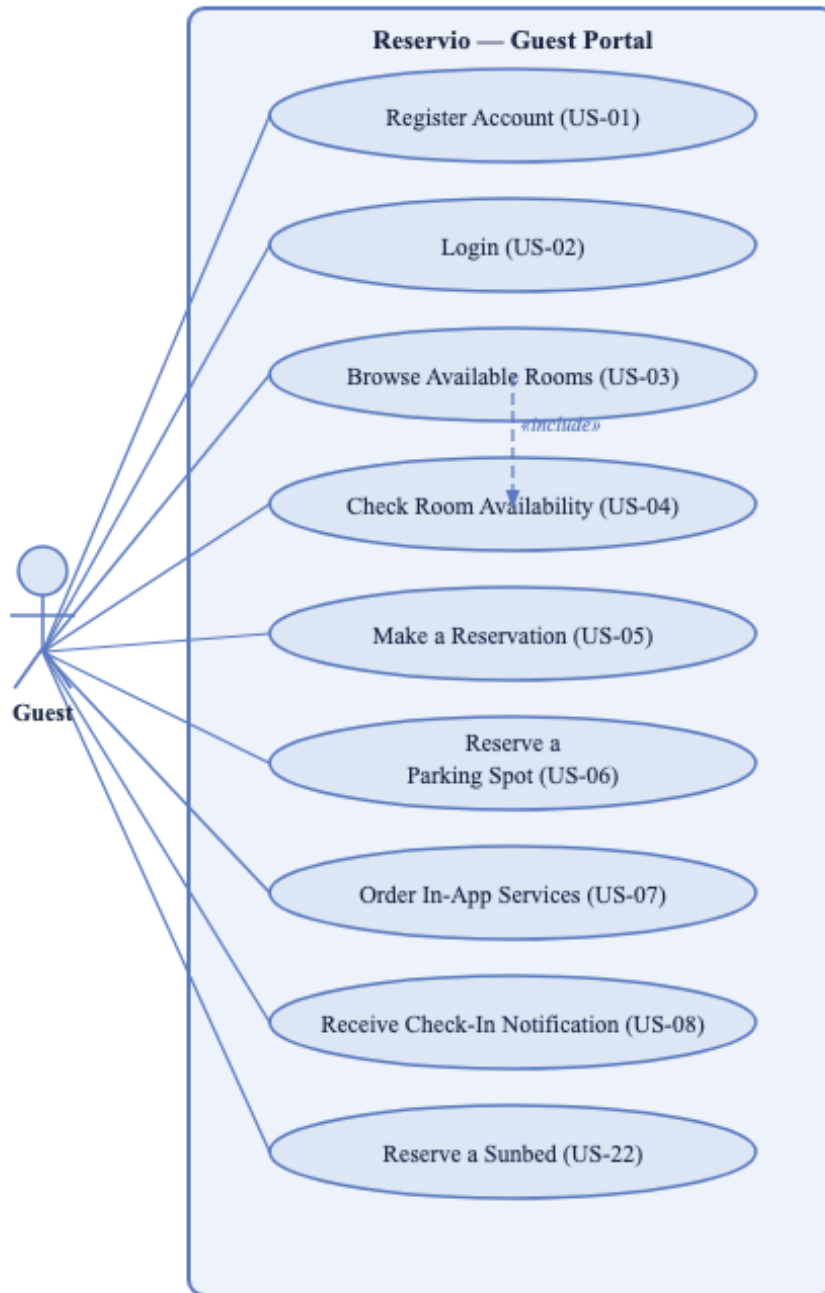


Figure 4.1 Use Case Diagram: Guest Module

4.1.2 Staff & Receptionist Module

Staff and Receptionists manage the core operational workflow from the reservation dashboard through check-in, check-out, room status management, and housekeeping coordination.

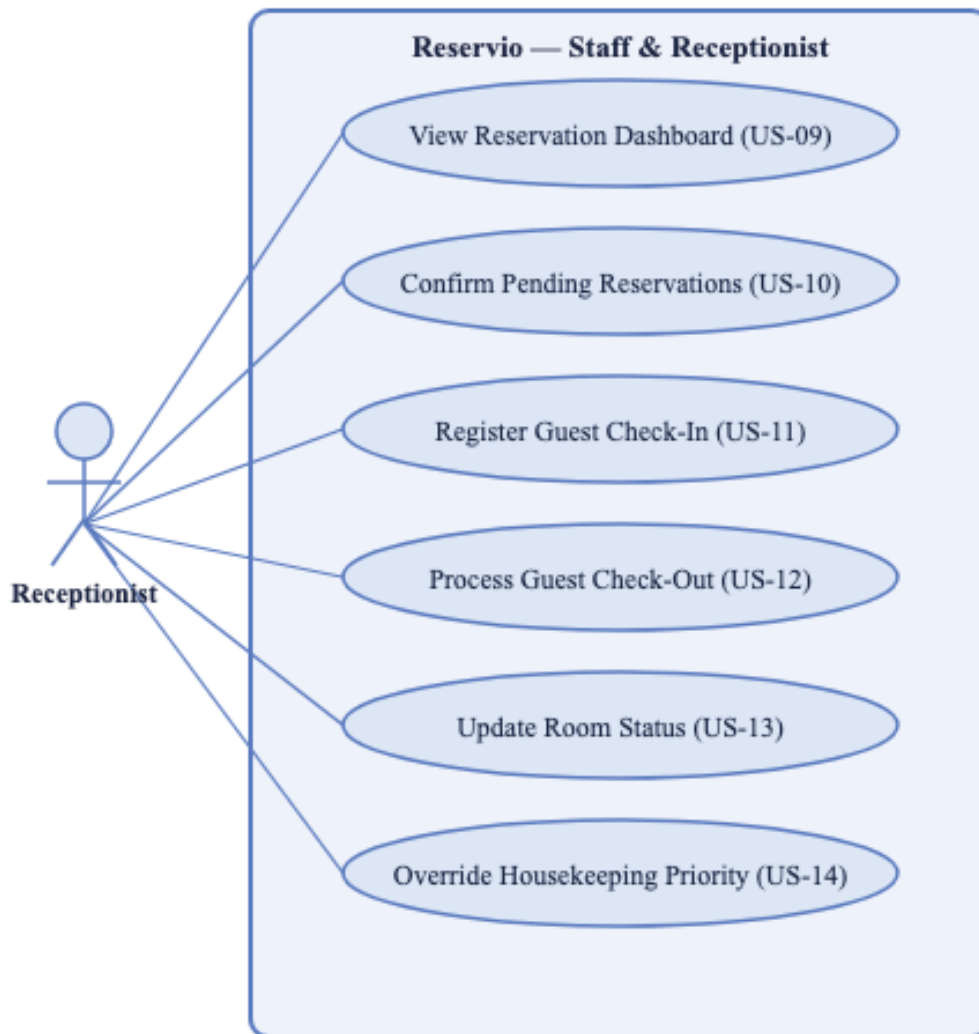


Figure 4.2 Use Case Diagram: Staff & Receptionist Module

4.1.3 Housekeeper Module

Housekeepers interact with a simplified, mobile-first task board. Their primary use cases are receiving the prioritised task queue and marking rooms as cleaned.

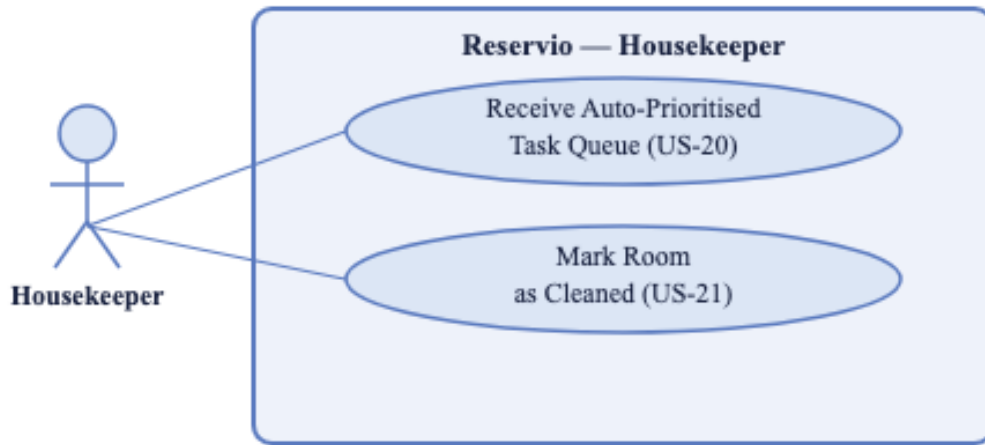


Figure 4.3 Use Case Diagram: Housekeeper Module

4.1.4 Hotel Admin Module

Hotel Administrators manage their hotel's entire workspace. Dynamic pricing and analytics are feature-gated to Premium and Custom subscription tiers.

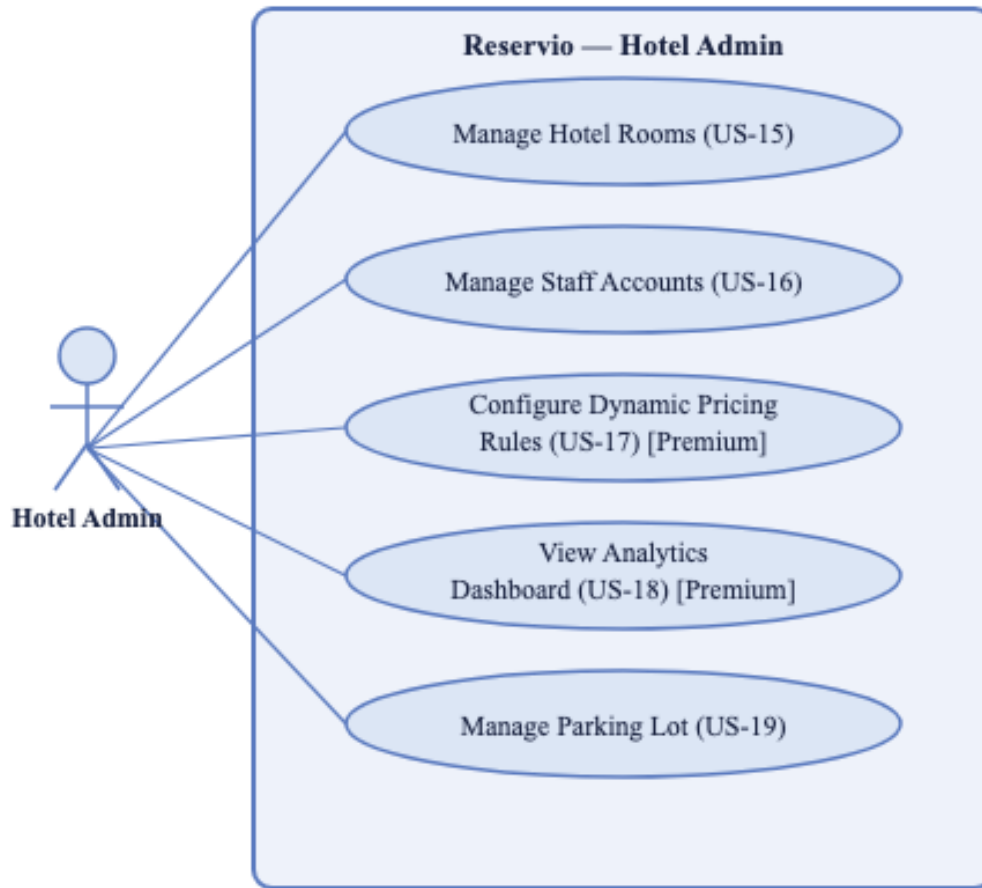


Figure 4.4 Use Case Diagram: Hotel Admin Module

4.1.5 Super Admin Module

The Super Admin operates at platform level, managing tenant lifecycle, subscription plans, and monitoring overall platform health.

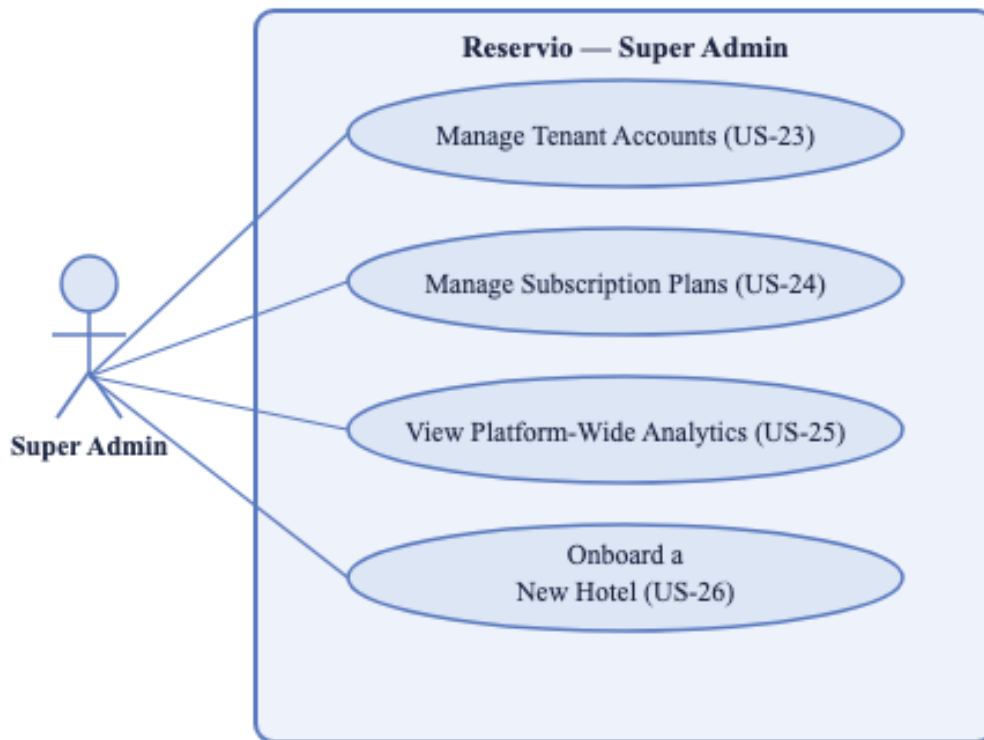


Figure 4.5 Use Case Diagram: Super Admin Module

4.2 Individual Use Case Diagrams (per User Story)

Each of the 26 user stories is represented below as an individual use case diagram, showing the specific actor, use case, and any include relationships for that story.

4.2.1 US-1 Guest Account Registration

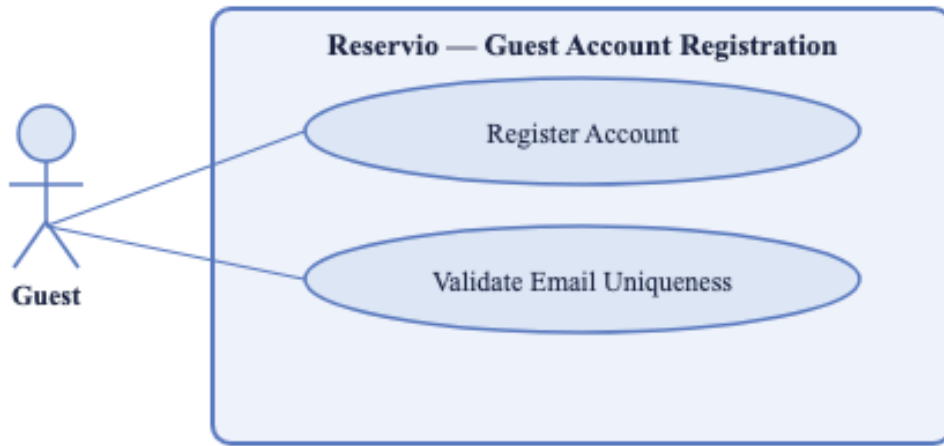


Figure 4.6 Individual Use Case: Guest Account Registration

4.2.2 US-2 Guest Login

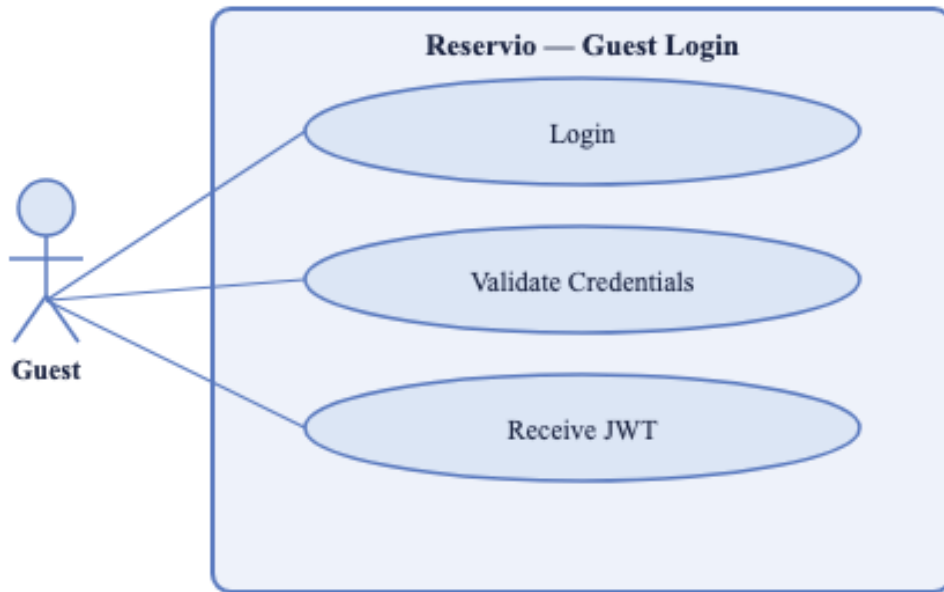


Figure 4.7 Individual Use Case: Guest Login

4.2.3 US-3 Browse Available Rooms

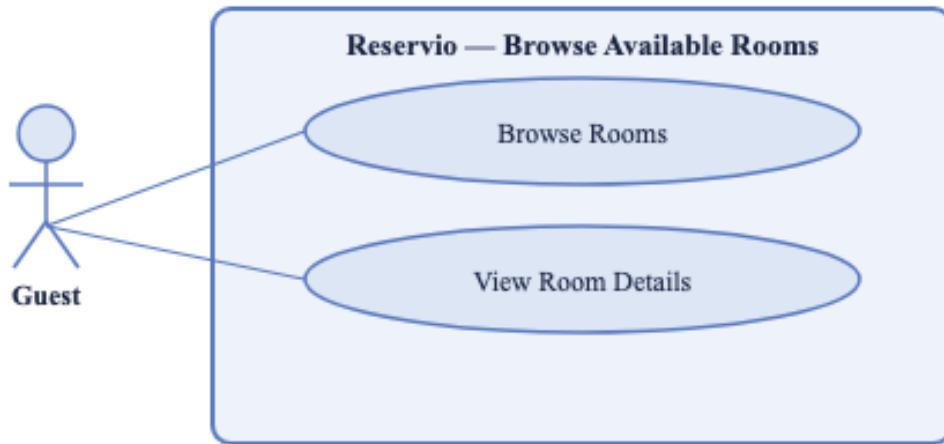


Figure 4.8 Individual Use Case: Browse Available Rooms

4.2.4 US-4 Check Room Availability

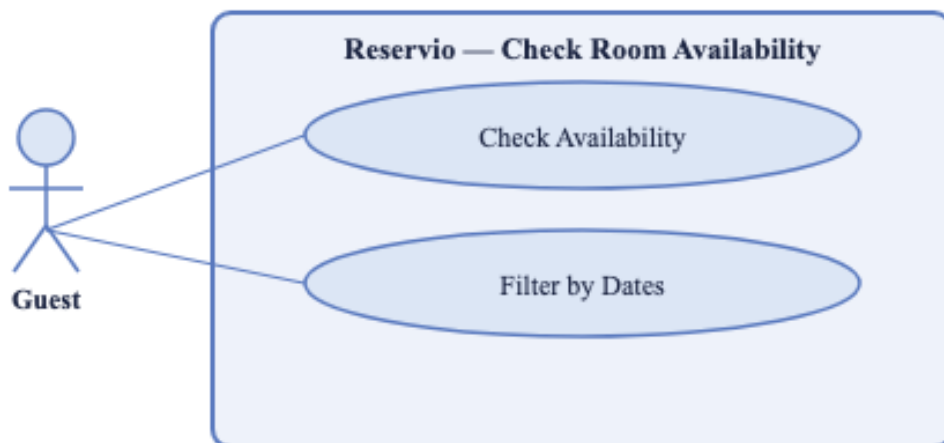


Figure 4.9 Individual Use Case: Check Room Availability

4.2.5 US-5 Make a Reservation

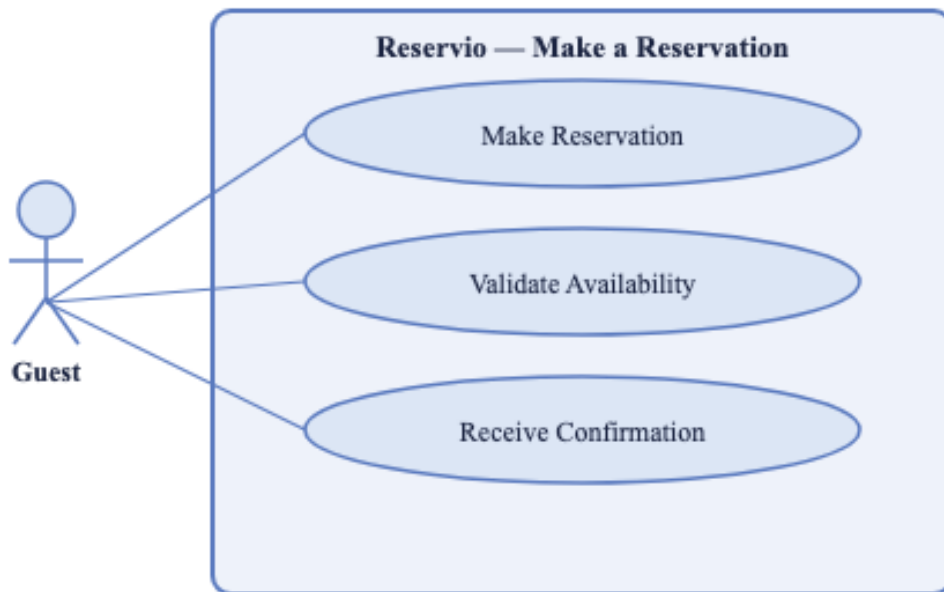


Figure 4.10 Individual Use Case: Make a Reservation

4.2.6 US-6 Reserve a Parking Spot

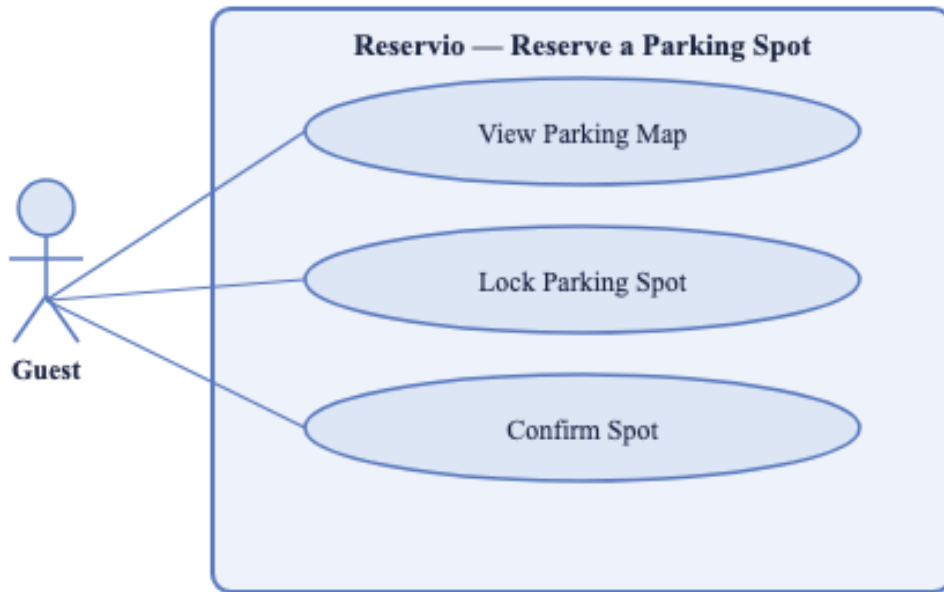


Figure 4.11 Individual Use Case: Reserve a Parking Spot

4.2.7 US-7 Order In-App Services

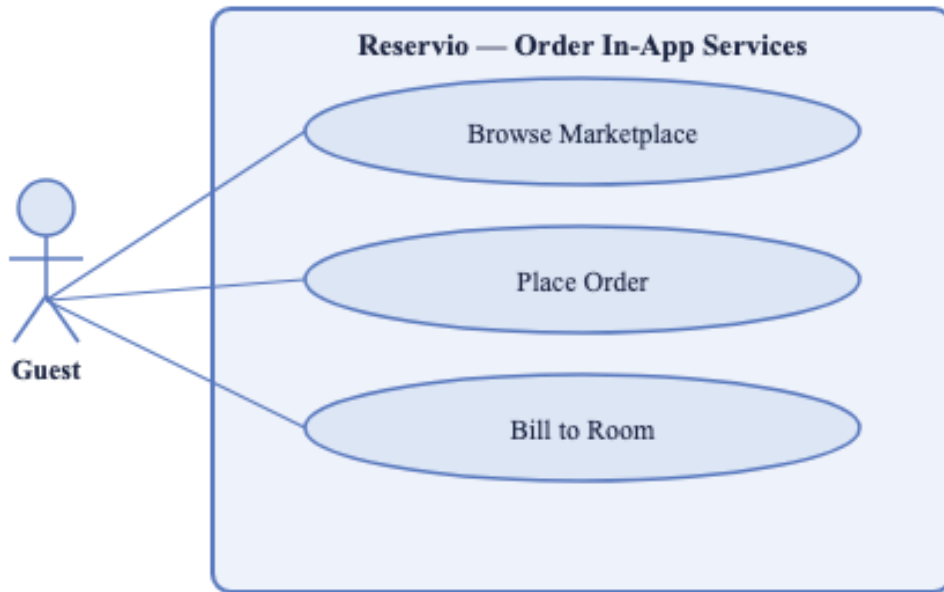


Figure 4.12 Individual Use Case: Order In-App Services

4.2.8 US-8 Receive Early Check-In Notification

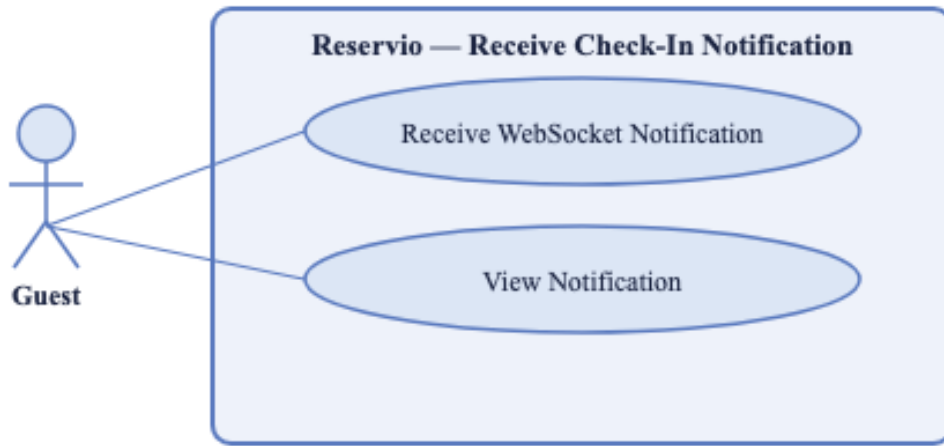


Figure 4.13 Individual Use Case: Receive Early Check-In Notification

4.2.9 US-9 View Reservation Dashboard

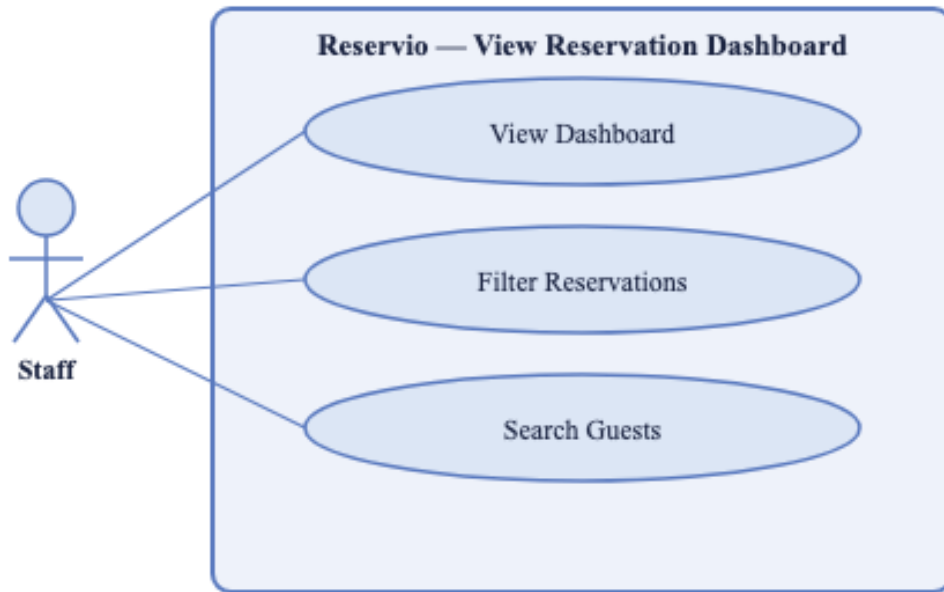


Figure 4.14 Individual Use Case: View Reservation Dashboard

4.2.10 US-10 Confirm Pending Reservations

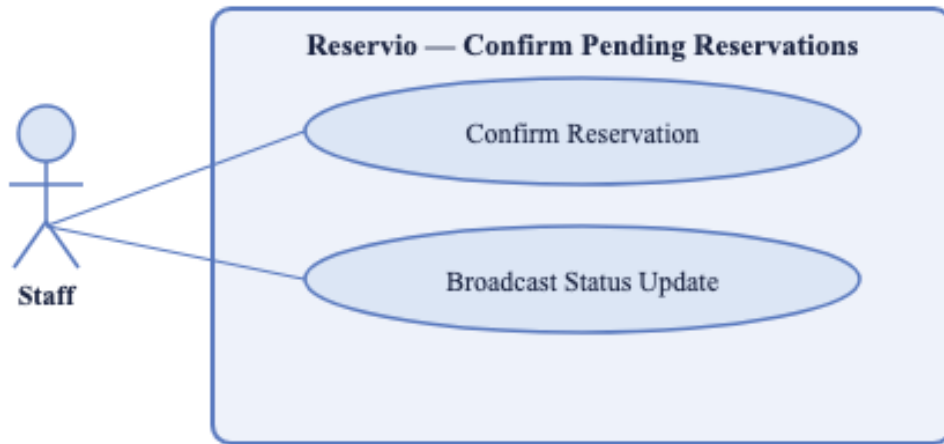


Figure 4.15 Individual Use Case: Confirm Pending Reservations

4.2.11 US-11 Register Guest Check-In

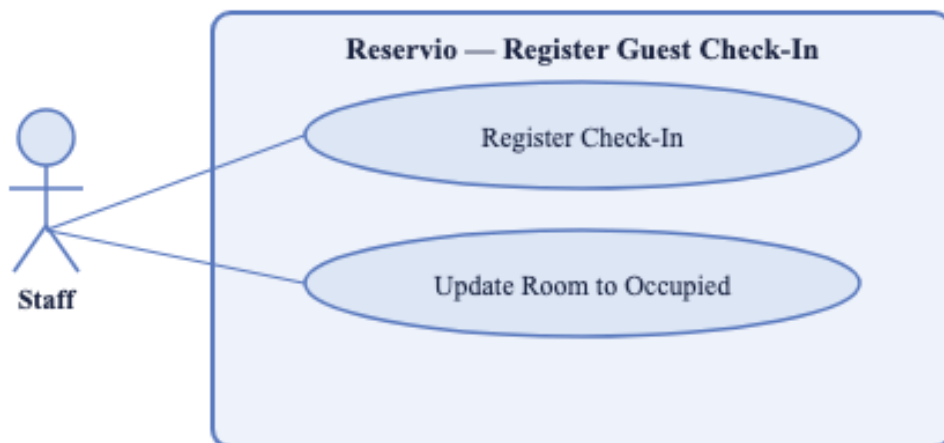


Figure 4.16 Individual Use Case: Register Guest Check-In

4.2.12 US-12 Process Guest Check-Out

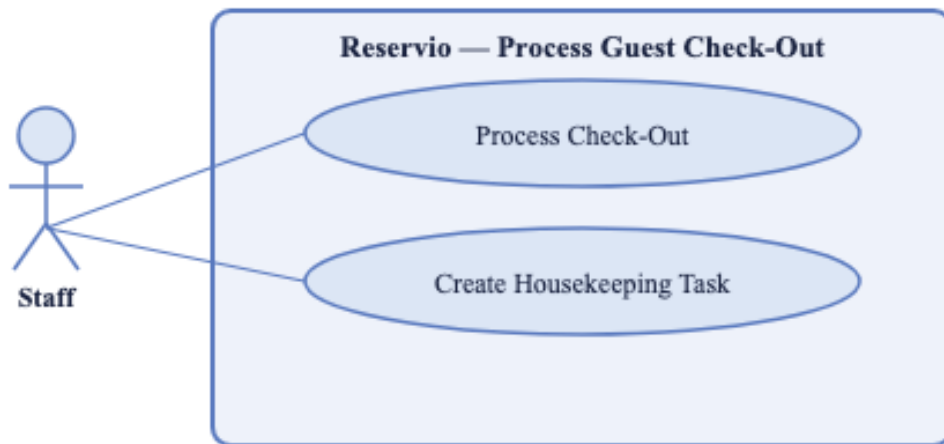


Figure 4.17 Individual Use Case: Process Guest Check-Out

4.2.13 US-13 Update Room Status

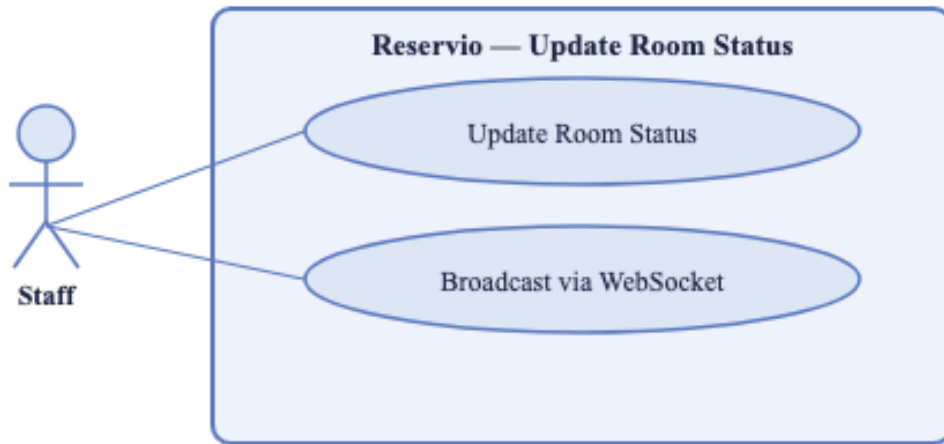


Figure 4.18 Individual Use Case: Update Room Status

4.2.14 US-14 Override Housekeeping Priority

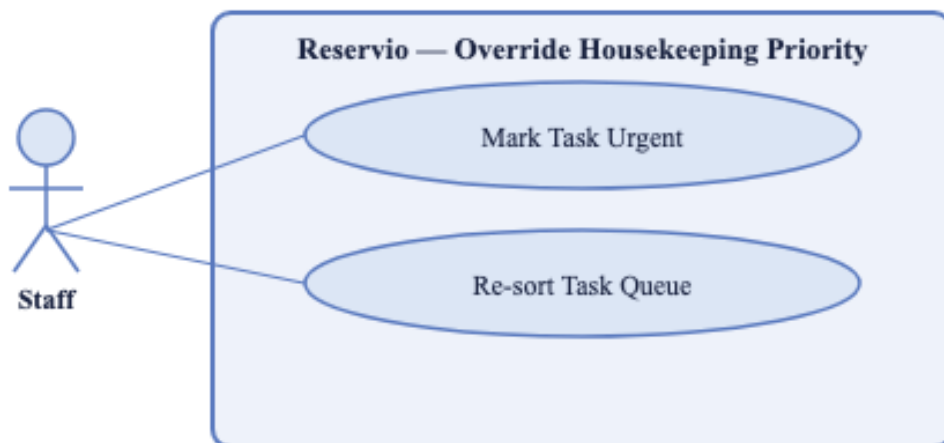


Figure 4.19 Individual Use Case: Override Housekeeping Priority

4.2.15 US-15 Manage Hotel Rooms

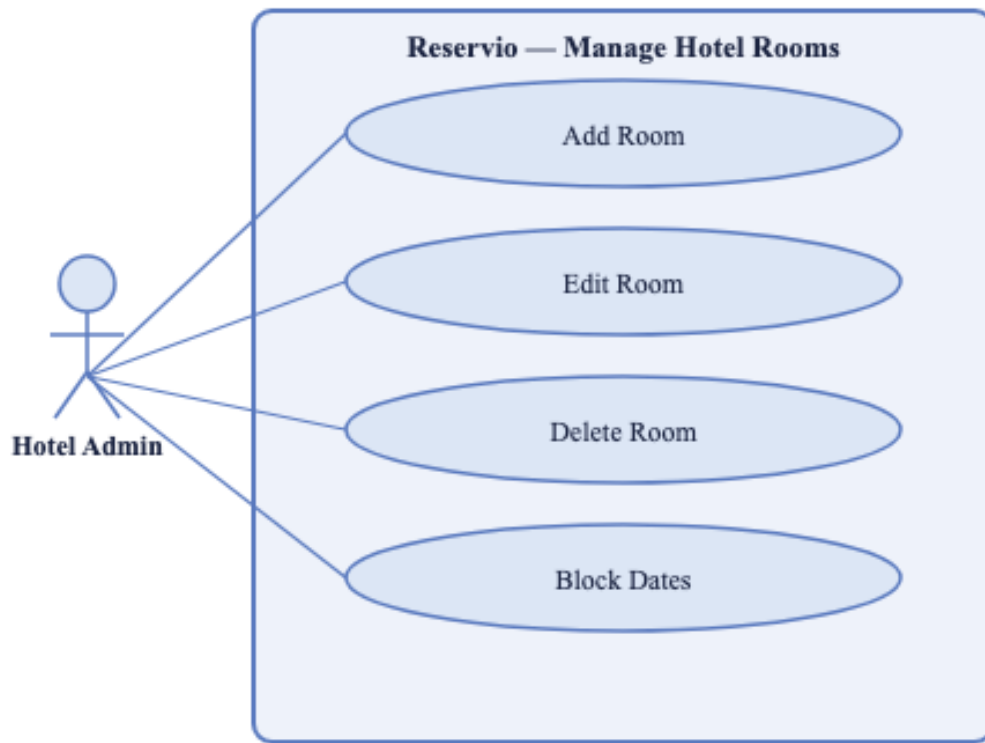


Figure 4.20 Individual Use Case: Manage Hotel Rooms

4.2.16 US-16 Manage Staff Accounts

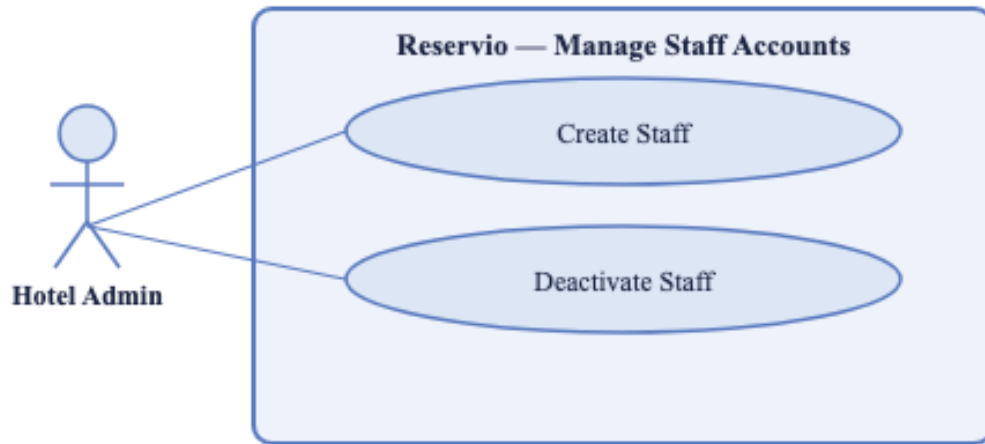


Figure 4.21 Individual Use Case: Manage Staff Accounts

4.2.17 US-17 Configure Dynamic Pricing Rules

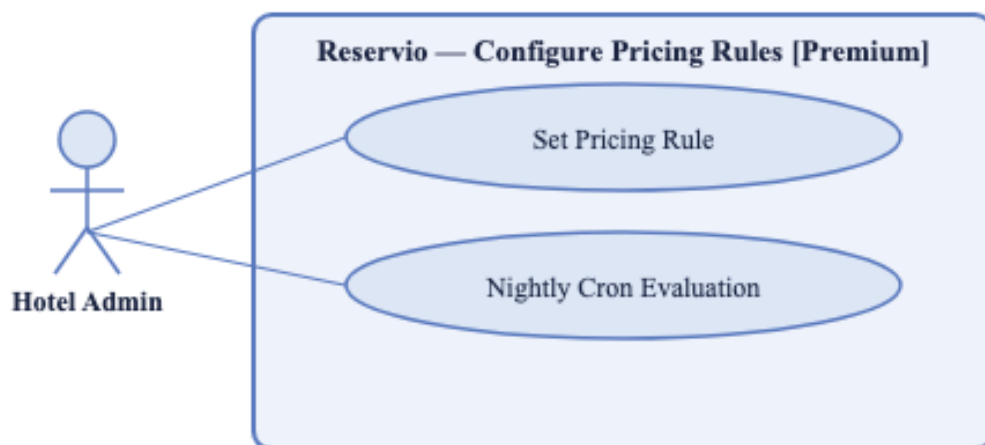


Figure 4.22 Individual Use Case: Configure Dynamic Pricing Rules

4.2.18 US-18 View Analytics Dashboard

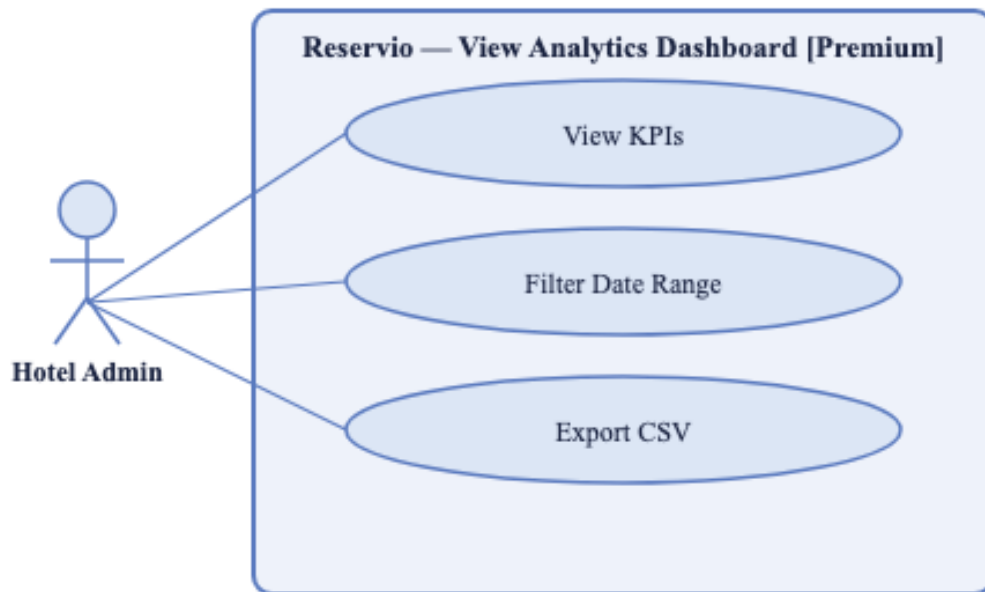


Figure 4.23 Individual Use Case: View Analytics Dashboard

4.2.19 US-19 Manage Parking Lot

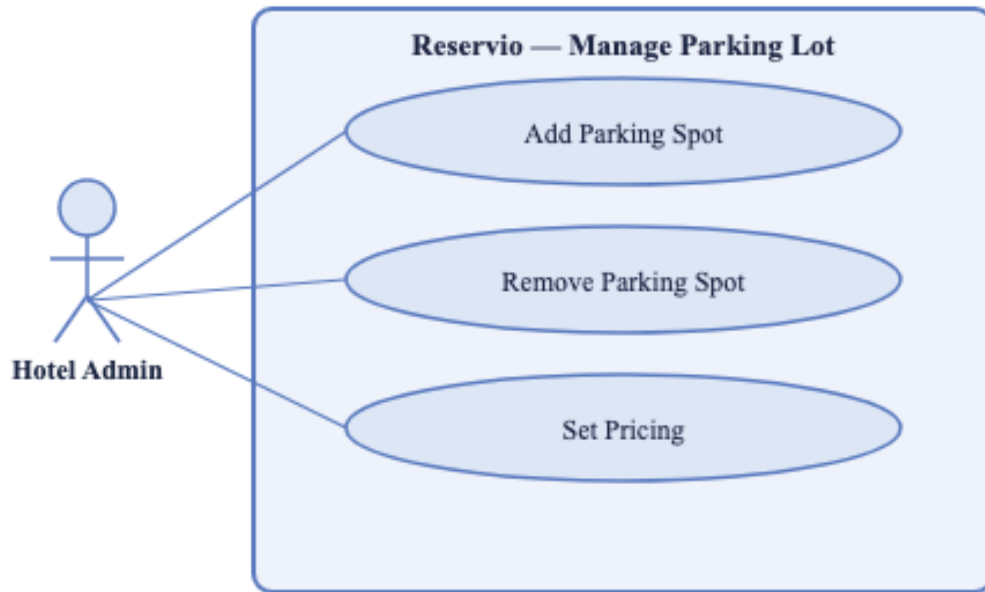


Figure 4.24 Individual Use Case: Manage Parking Lot

4.2.20 US-20 Receive Auto-Prioritised Task Queue

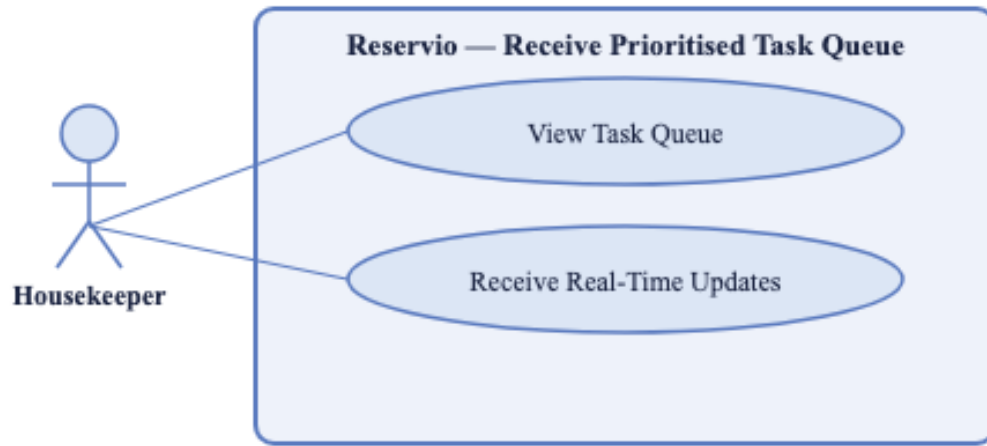


Figure 4.25 Individual Use Case: Receive Auto-Prioritised Task Queue

4.2.21 US-21 Mark Room as Cleaned

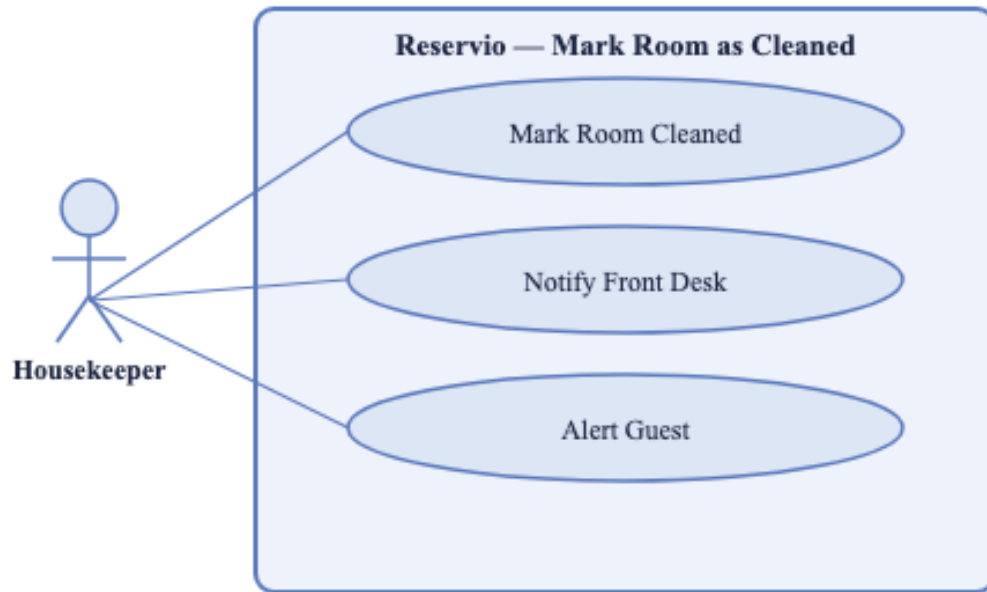


Figure 4.26 Individual Use Case: Mark Room as Cleaned

4.2.22 US-22 Reserve a Sunbed

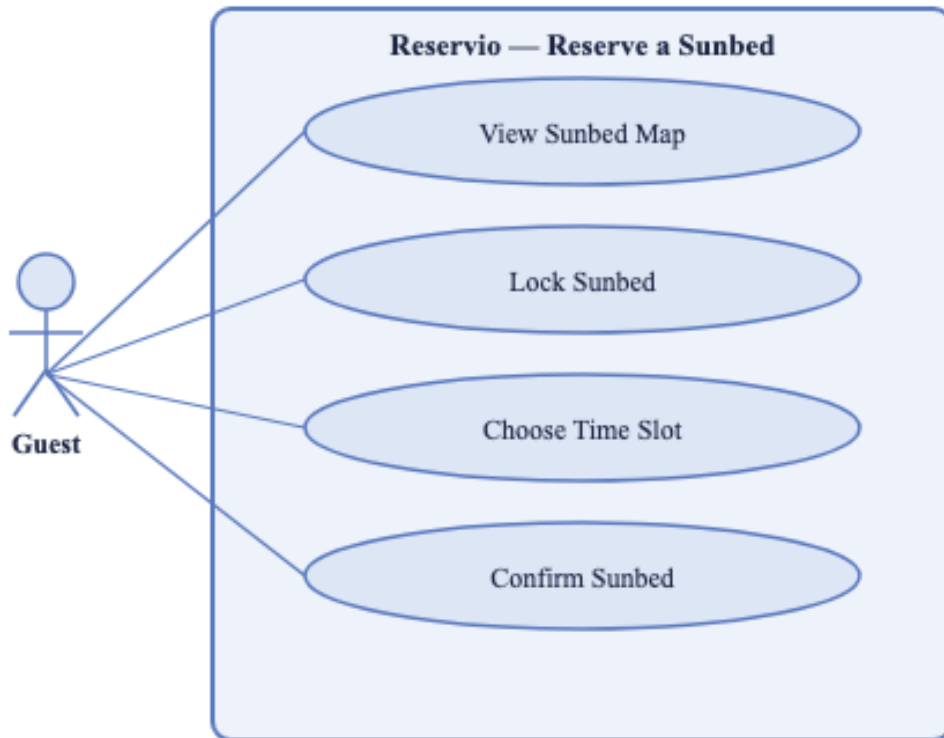


Figure 4.27 Individual Use Case: Reserve a Sunbed

4.2.23 US-23 Manage Tenant Accounts

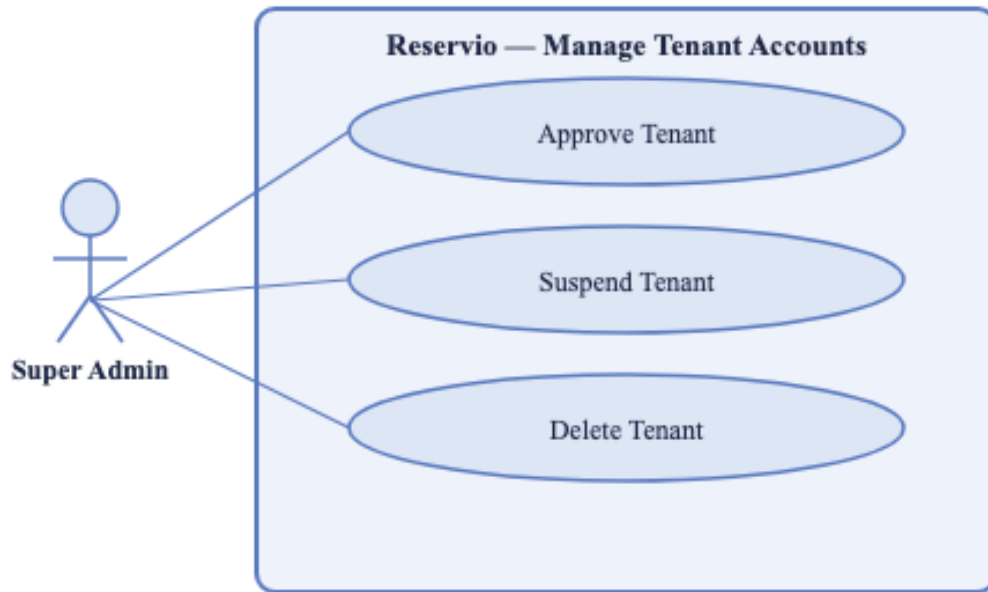


Figure 4.28 Individual Use Case: Manage Tenant Accounts

4.2.24 US-24 Manage Subscription Plans

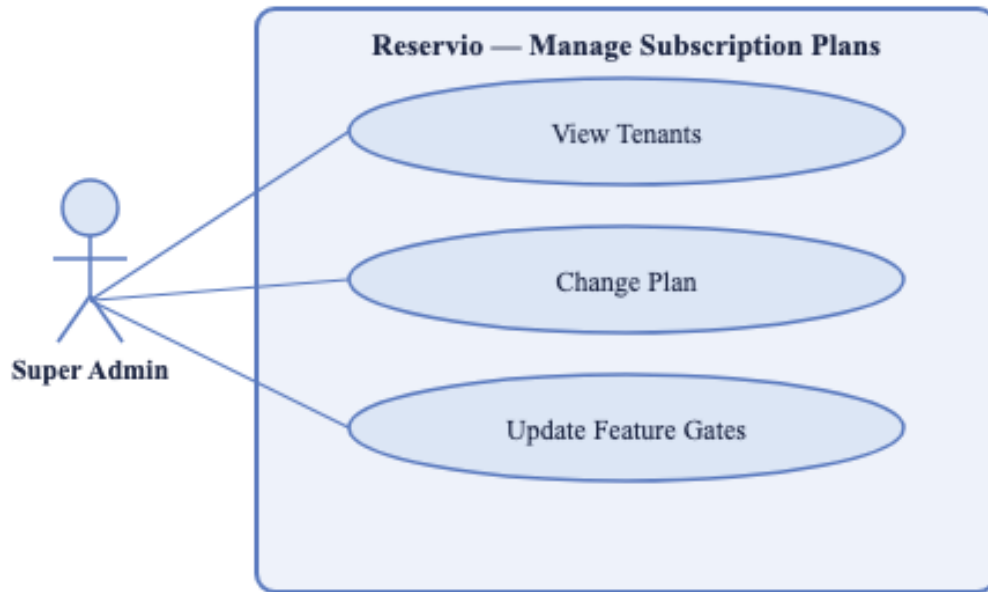


Figure 4.29 Individual Use Case: Manage Subscription Plans

4.2.25 US-25 View Platform-Wide Analytics

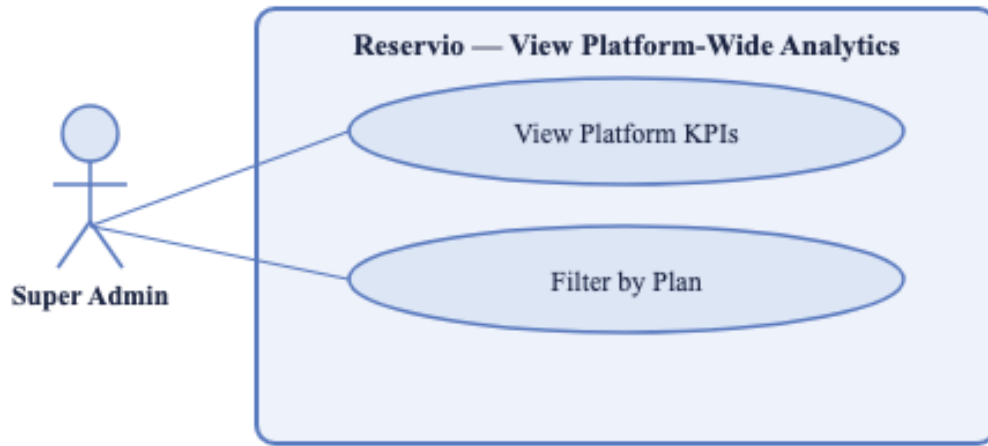


Figure 4.30 Individual Use Case: View Platform-Wide Analytics

4.2.26 US-26 Onboard a New Hotel

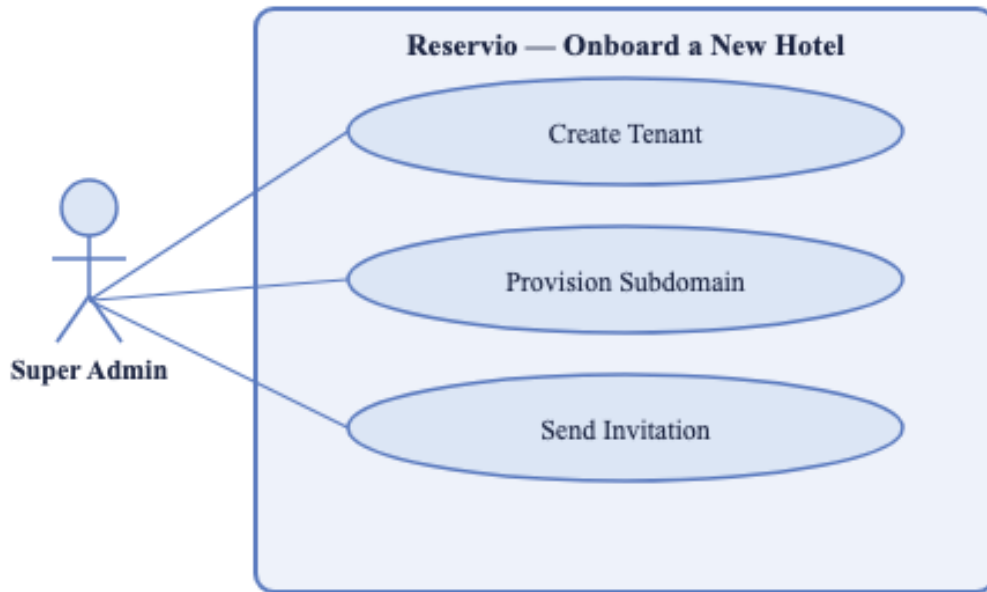


Figure 4.31 Individual Use Case: Onboard a New Hotel

4.3 Activity Diagrams

Activity diagrams model the sequential flow of actions within each user story, including decision points and error paths. Each diagram maps directly to one user story.

4.3.1 US-01 Activity Diagram

The guest navigates to the hotel subdomain, selects Register, completes the form, and the system validates uniqueness before creating an account scoped to the tenant.

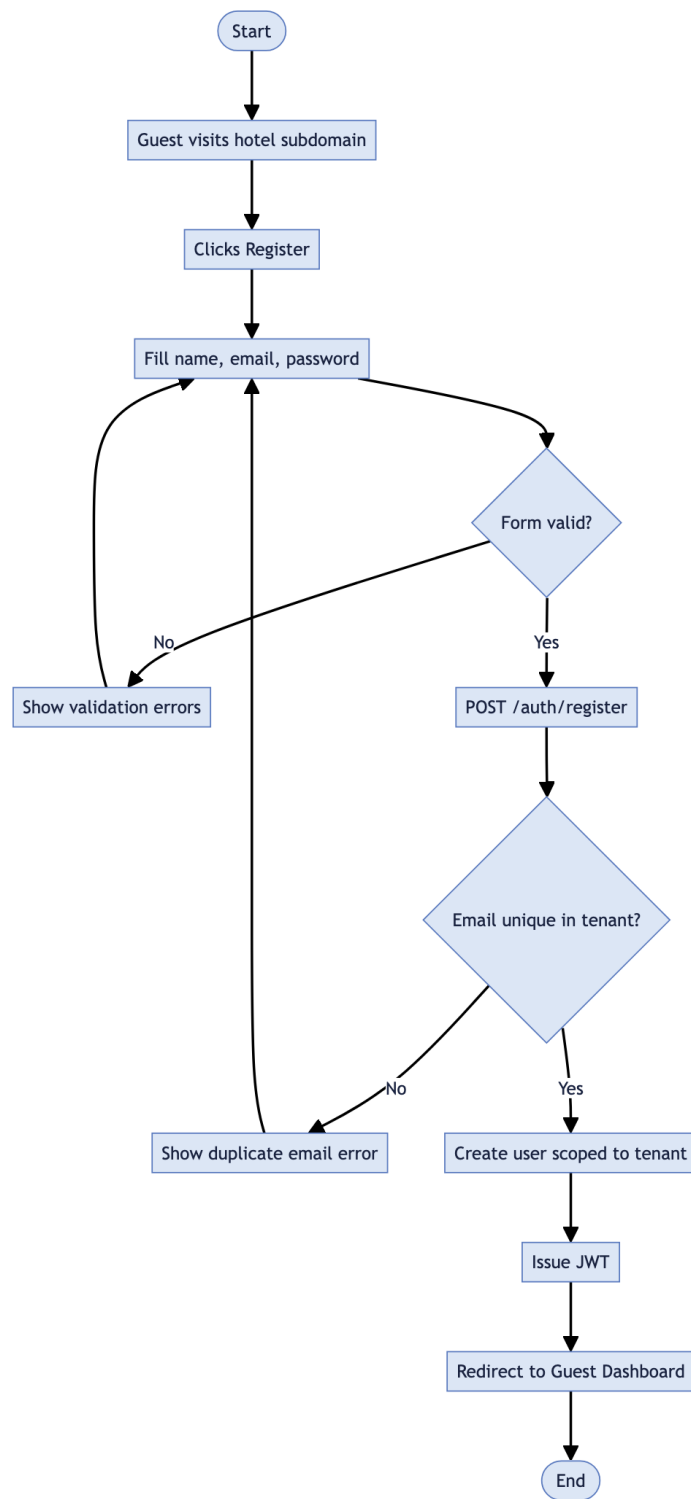


Figure 4.32 Activity Diagram: US-1

4.3.2 US-02 Activity Diagram

The guest enters email and password. The system verifies credentials against the tenant-scoped user record and issues a JWT containing userId, tenantId, and role.

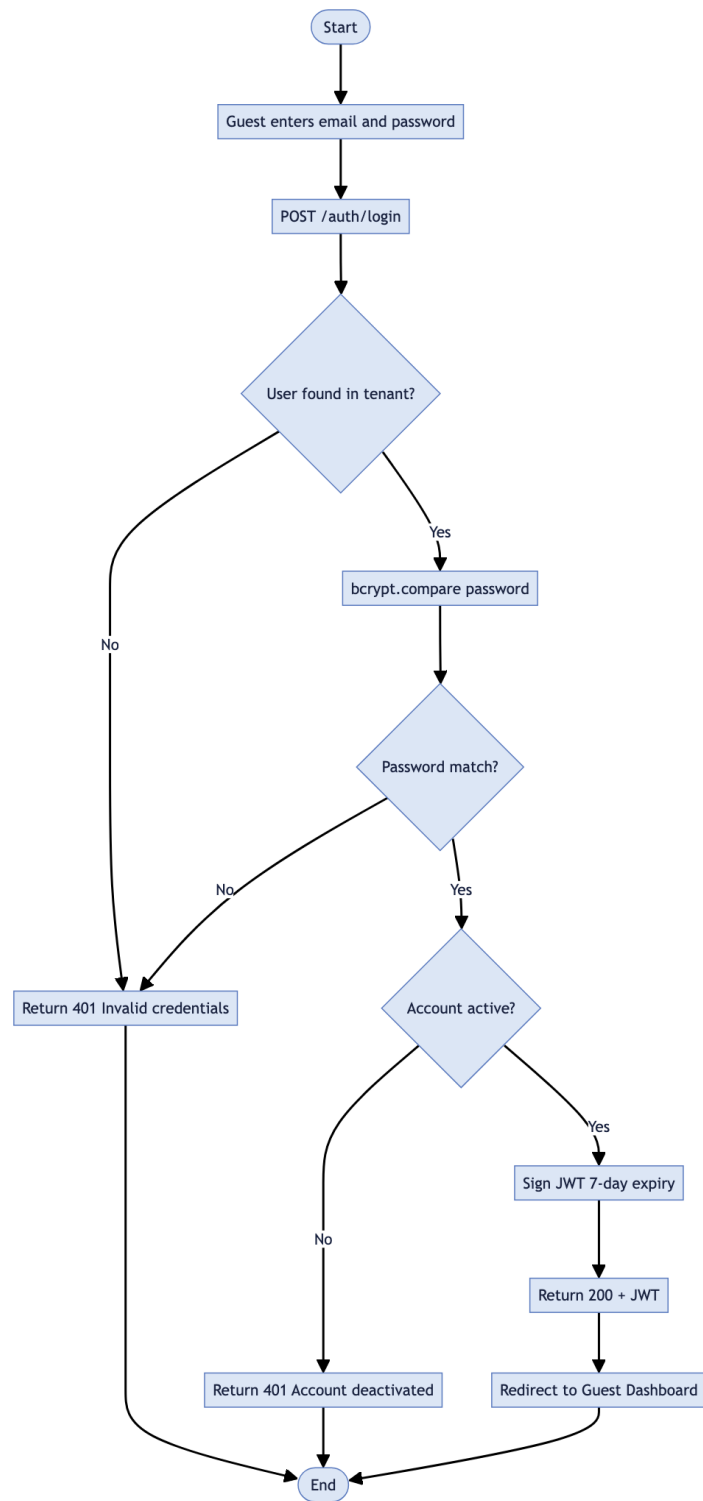


Figure 4.33 Activity Diagram: US-2

4.3.3 US-03 Activity Diagram

Guest navigates to the rooms page. The API returns all AVAILABLE rooms scoped to the tenant. Room cards display photo, type, description, and price.

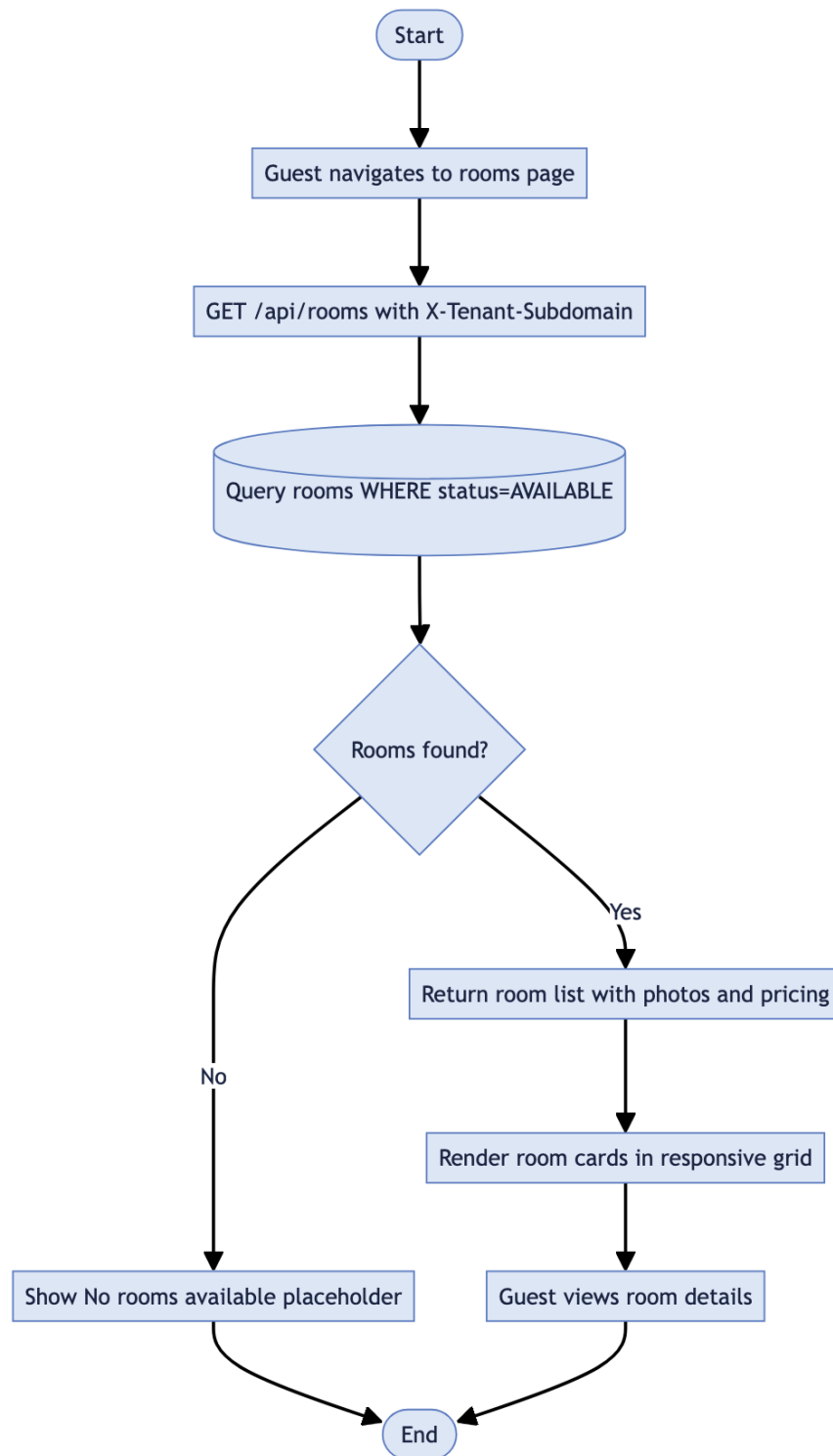


Figure 4.34 Activity Diagram: US-3

4.3.4 US-04 Activity Diagram

Guest selects check-in and check-out dates. The system queries for overlapping reservations and room blocks, then returns only conflict-free rooms.

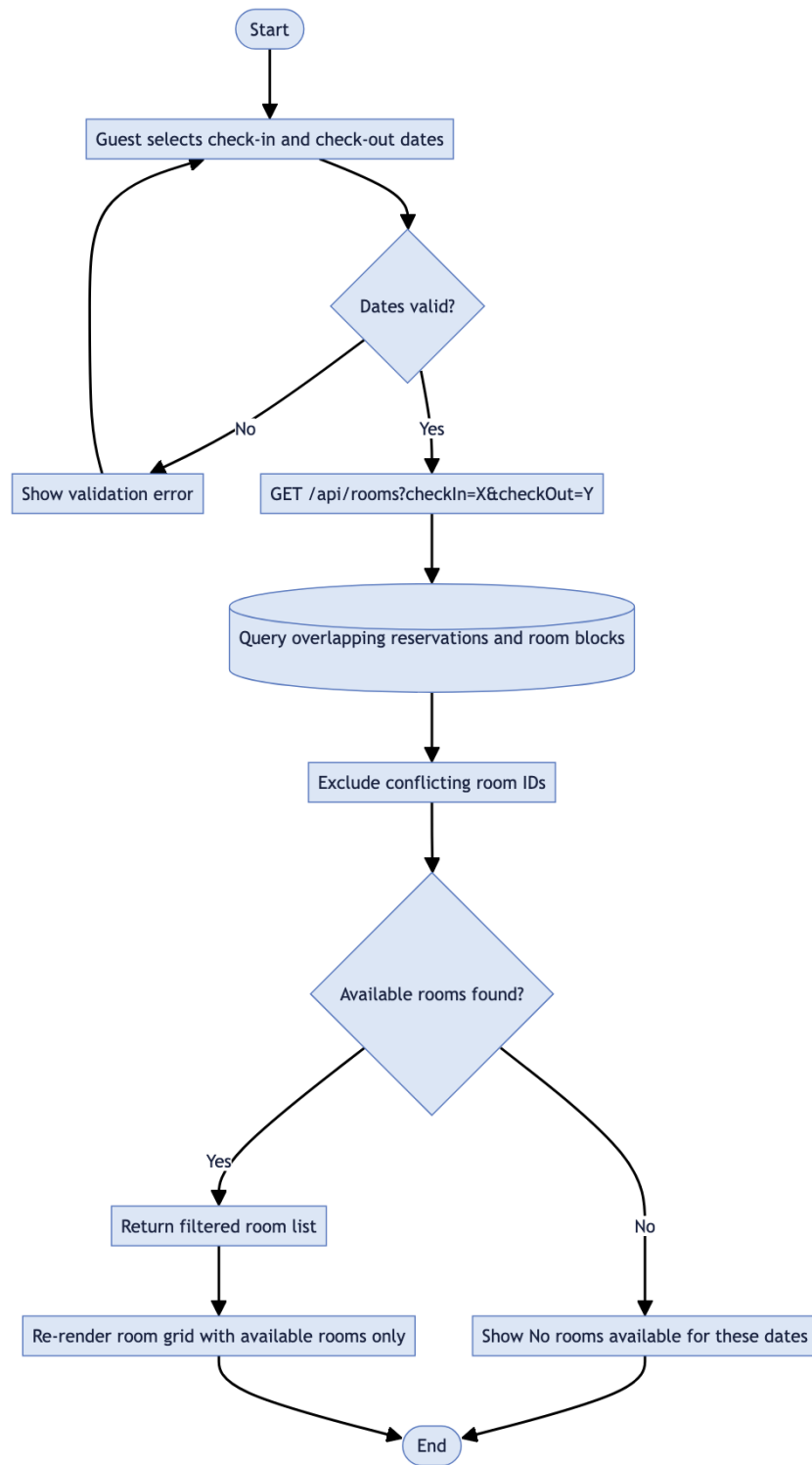


Figure 4.35 Activity Diagram: US-4

4.3.5 US-05 Activity Diagram

After selecting a room, the guest completes a 3-step flow. The system validates availability, checks blacklist status, and creates the reservation in PENDING status.

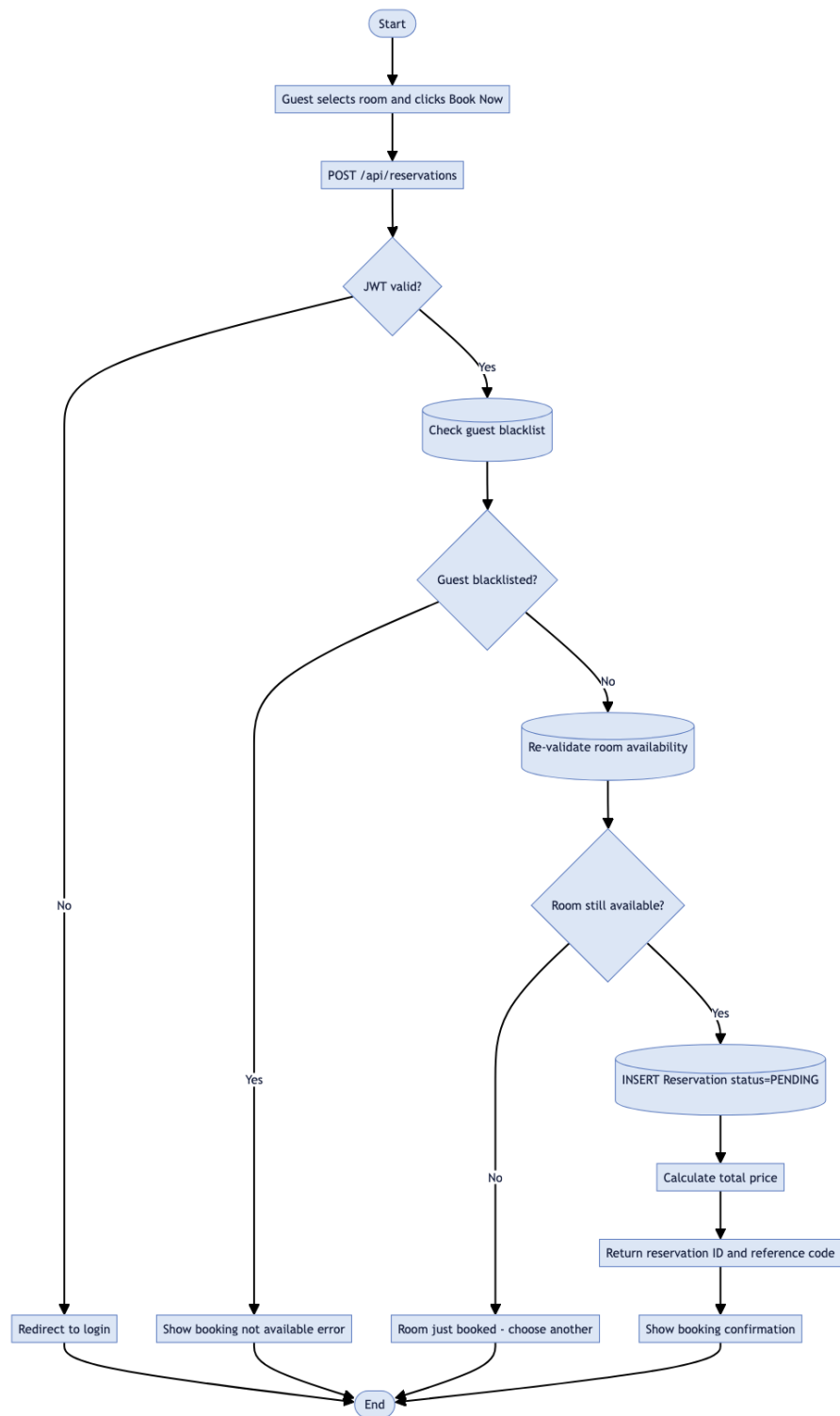


Figure 4.36 Activity Diagram: US-5

4.3.6 US-06 Activity Diagram

Guest opens the parking map. Available spots show green. Selecting a spot locks it for 5 minutes (yellow). The guest confirms or cancels within the countdown window.

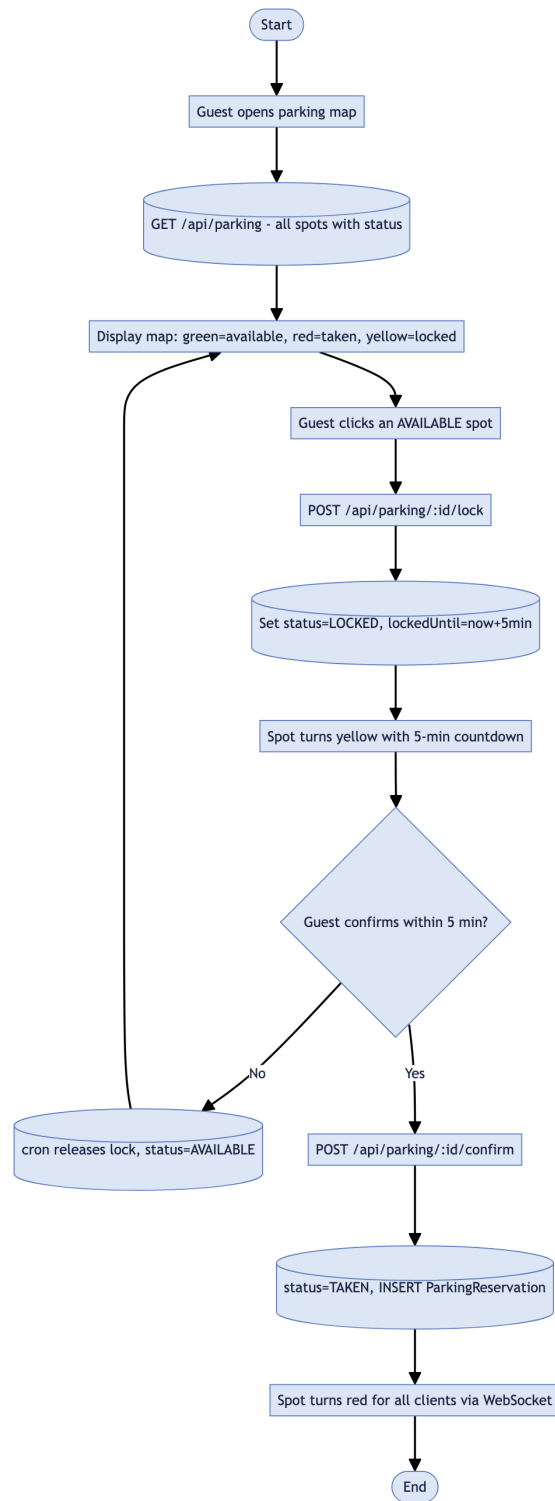


Figure 4.37 Activity Diagram: US-6

4.3.7 US-07 Activity Diagram

Guest browses service marketplace by category, adds items to cart, and bills them to their active room reservation.

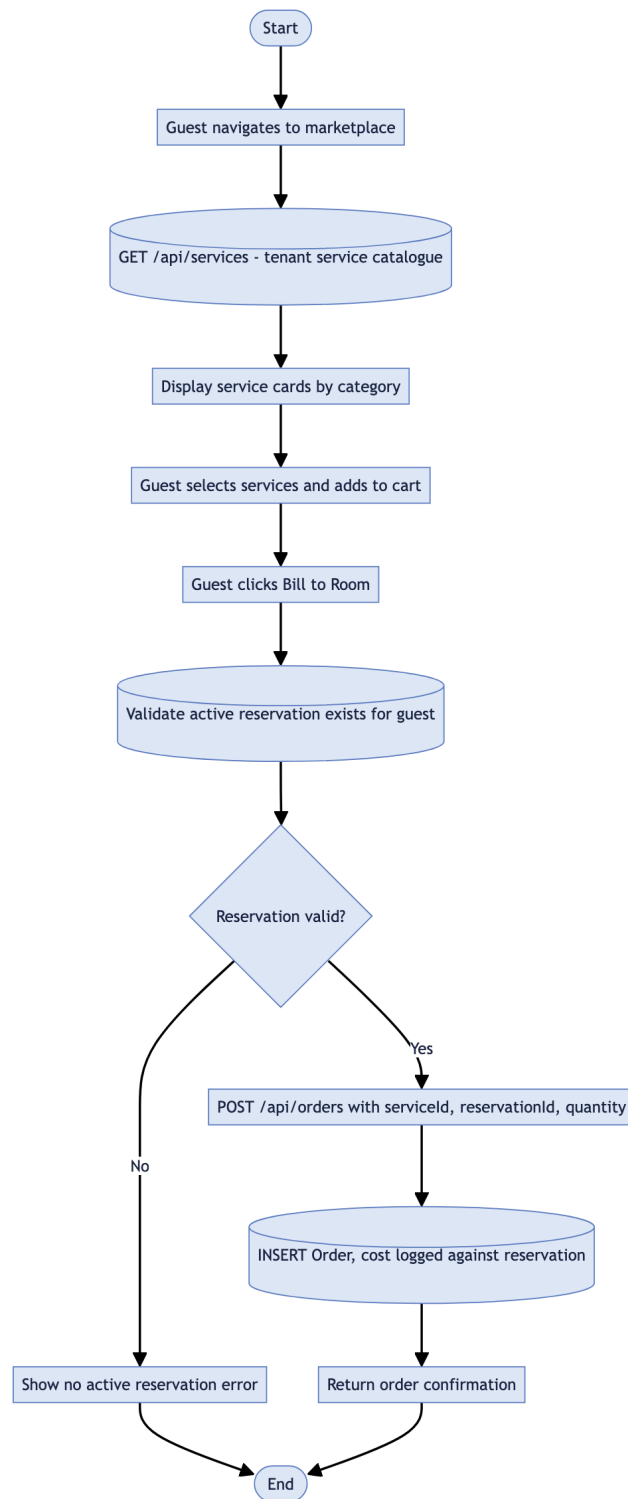


Figure 4.38 Activity Diagram: US-7

4.3.8 US-08 Activity Diagram

When a housekeeper marks a room as cleaned, the system checks for guests with same-day confirmed reservations and emits a WebSocket notification to those guests.

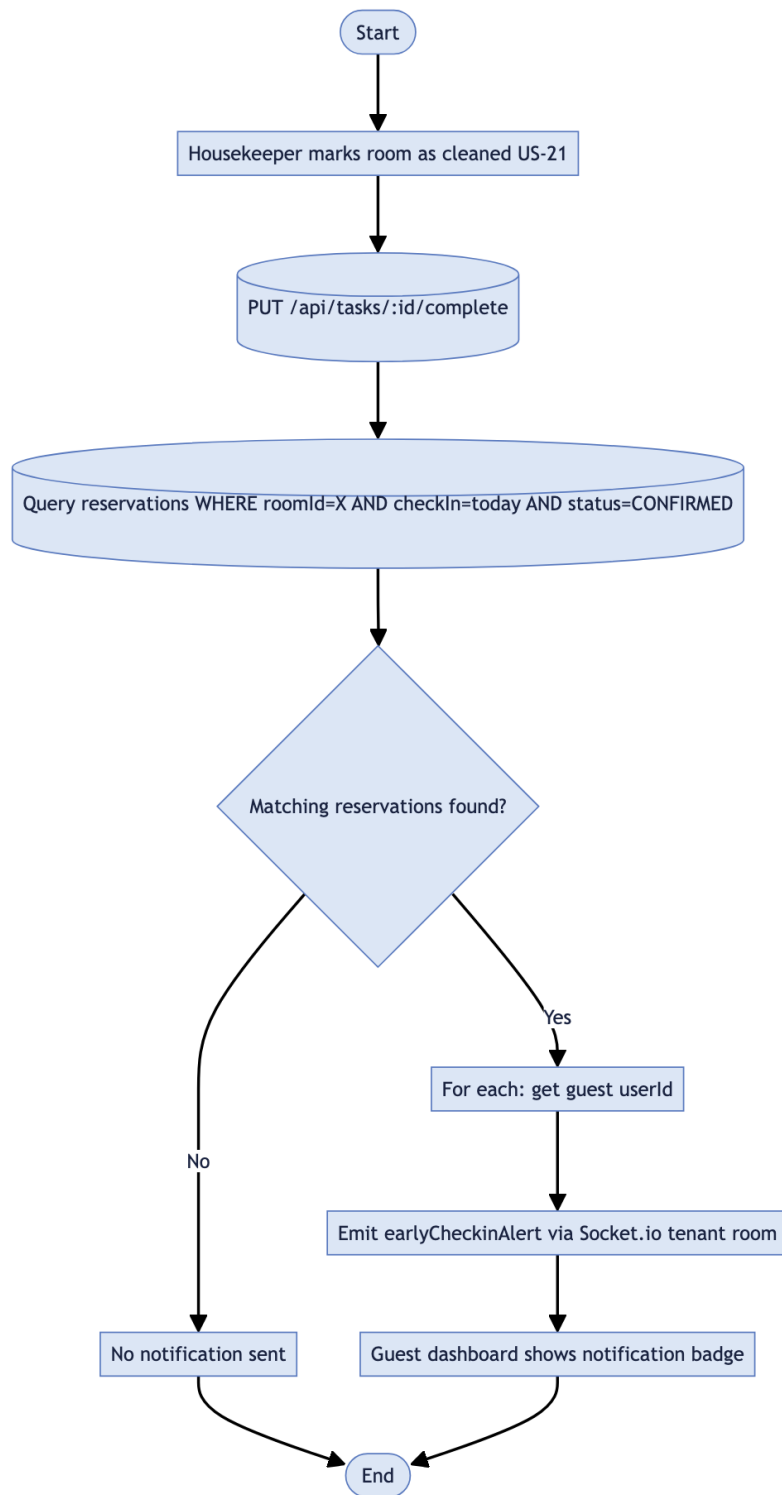


Figure 4.39 Activity Diagram: US-8

4.3.9 US-09 Activity Diagram

Staff navigates to the reservation dashboard. All reservations for the tenant are fetched with guest and room details, and can be filtered by status or date. ■ ■ AD: Filtering reservation Staff

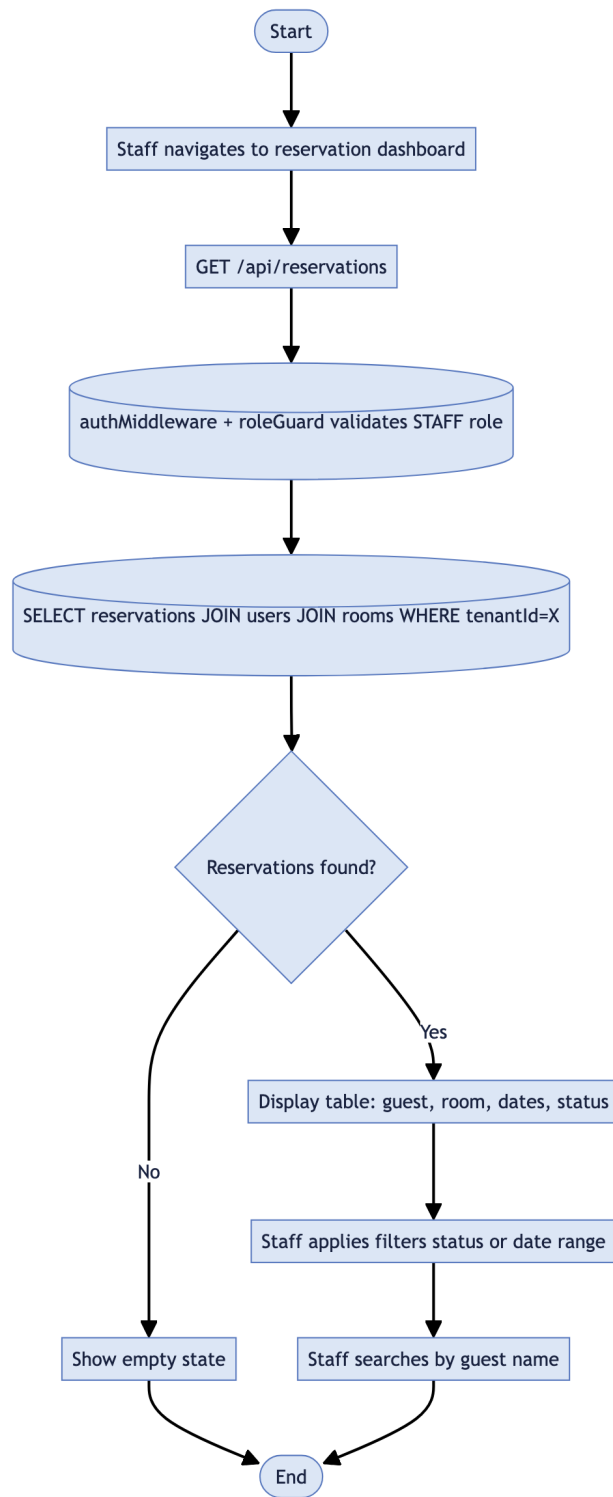


Figure 4.40 Activity Diagram: US-9

4.3.10 US-10 Activity Diagram

Staff clicks Confirm on a pending reservation. Status changes to CONFIRMED immediately and a WebSocket event propagates the change to all connected staff.

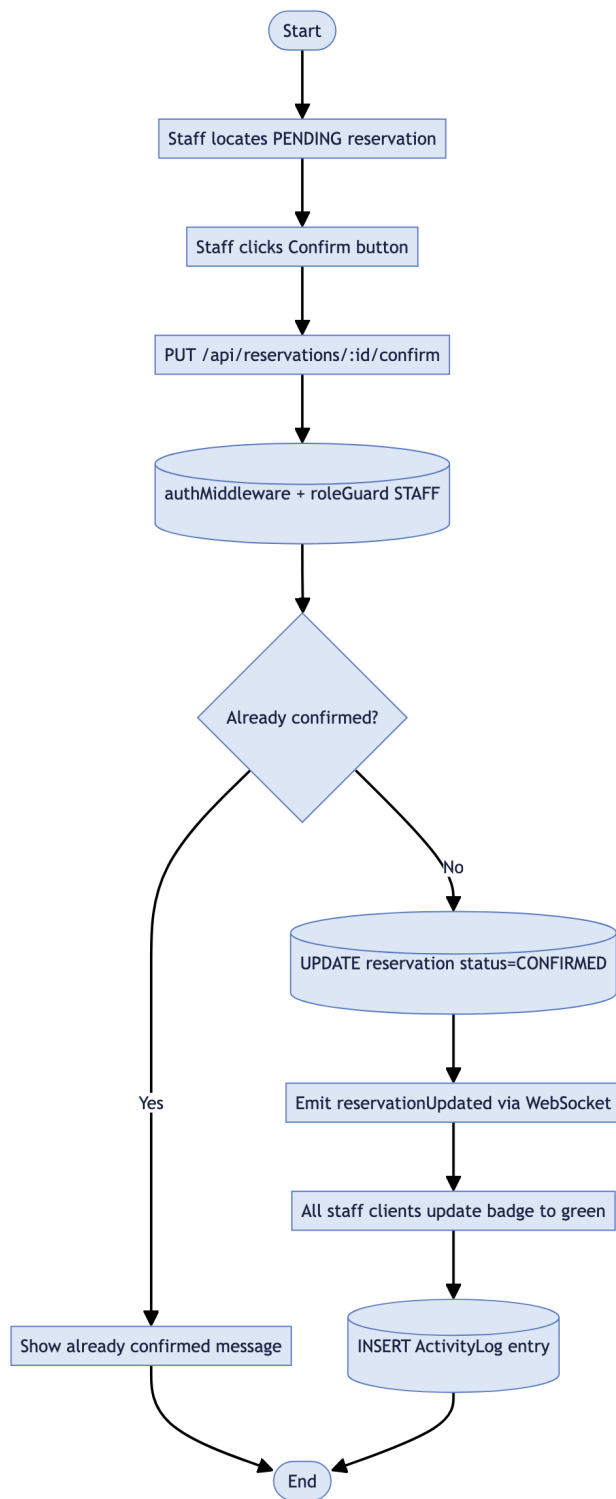


Figure 4.41 Activity Diagram: US-10

4.3.11 US-11 Activity Diagram

Staff selects a confirmed reservation for today and registers the guest as checked in. Room status updates to OCCUPIED and is broadcast via WebSocket.

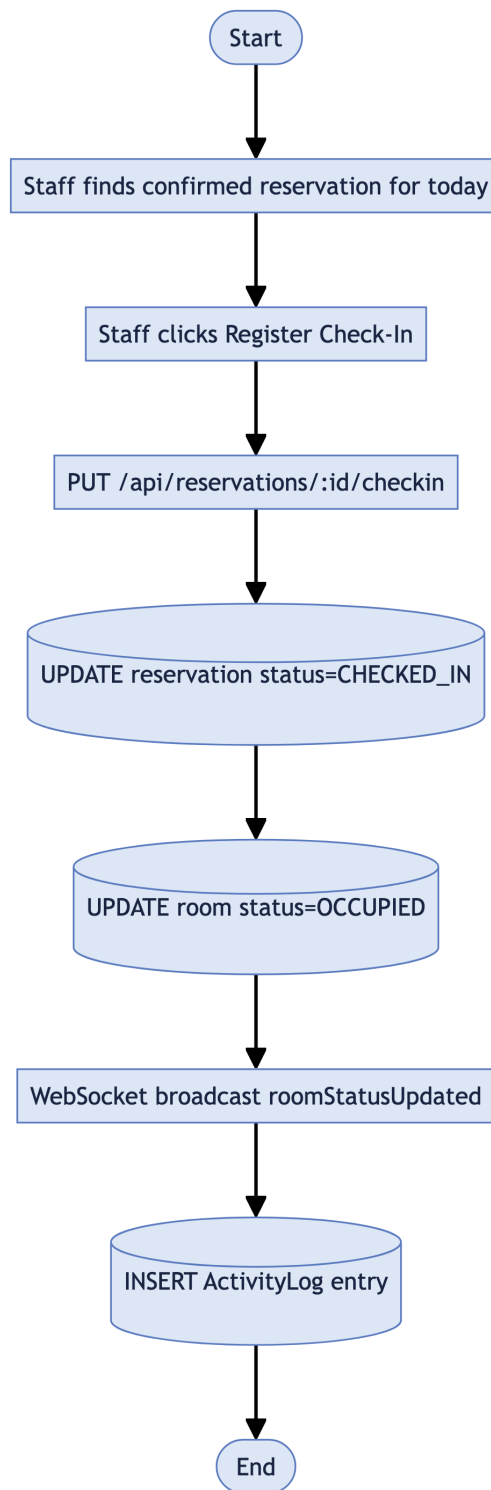


Figure 4.42 Activity Diagram: US-11

4.3.12 US-12 Activity Diagram

Staff processes check-out for a checked-in guest. Room is freed, a housekeeping task is auto-created, and the reservation is archived.

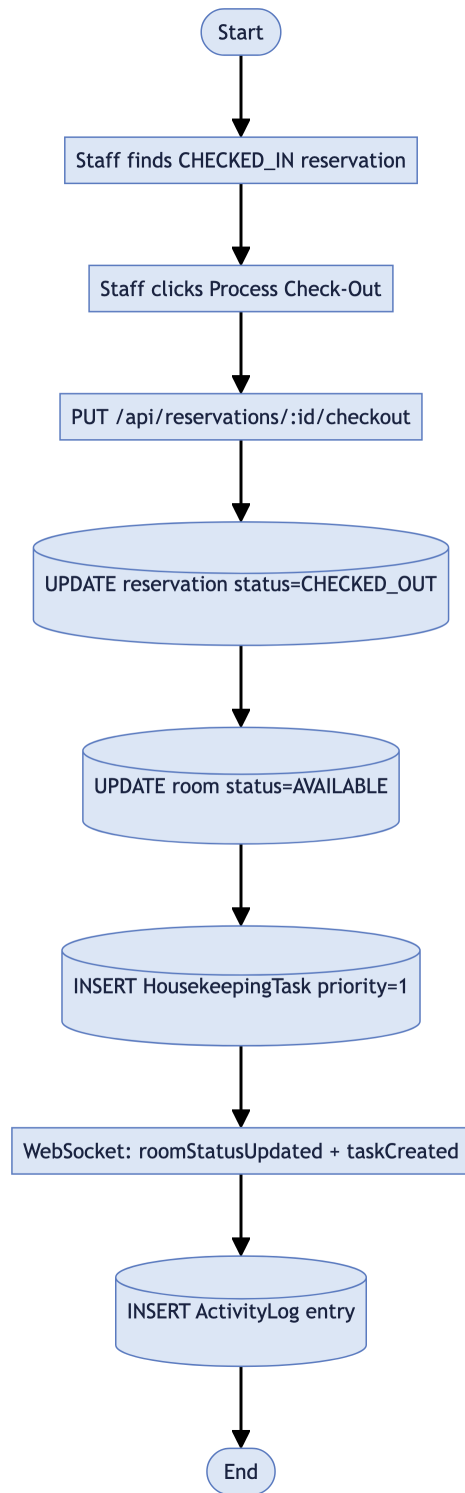


Figure 4.43 Activity Diagram: US-12

4.3.13 US-13 Activity Diagram

Staff selects a new status for a room. The change is persisted and broadcast to all connected staff and admin devices via WebSocket.

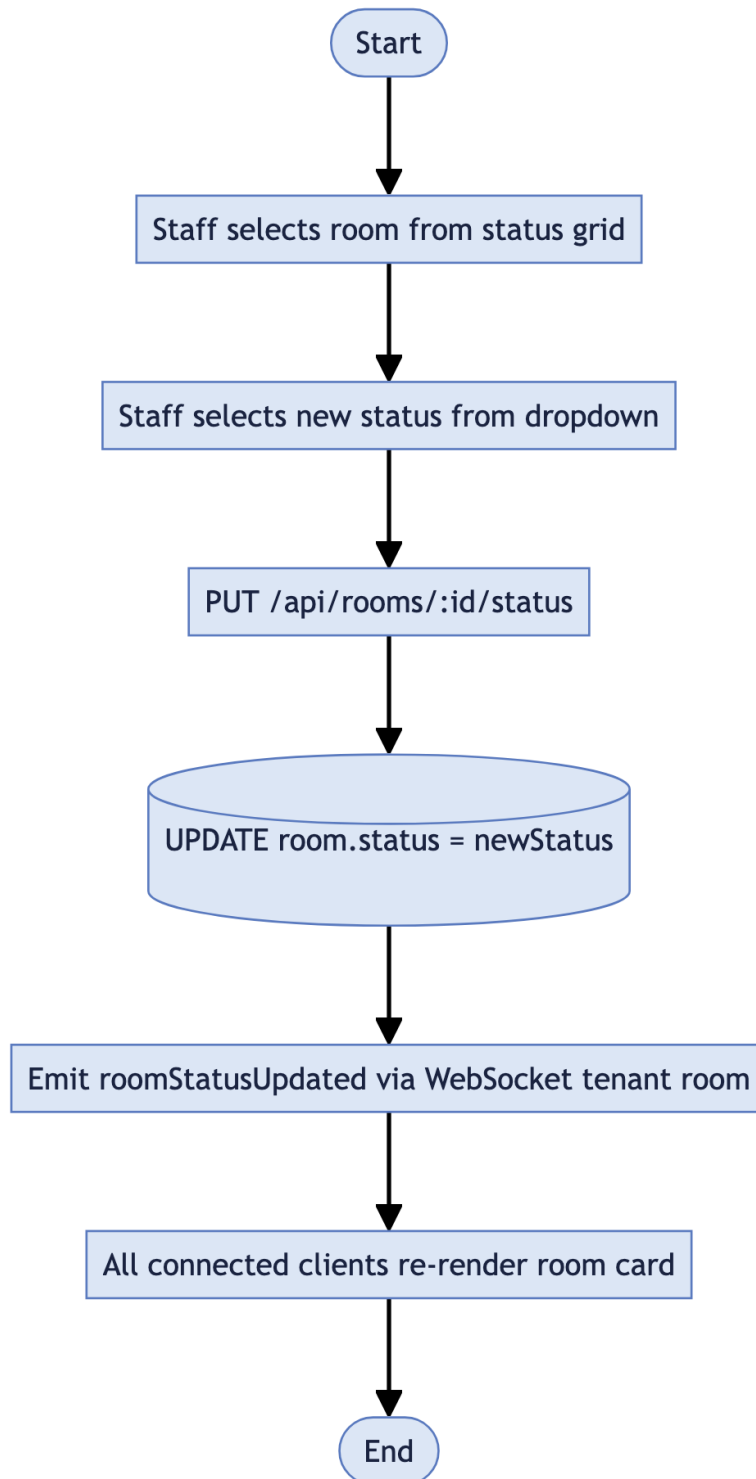


Figure 4.44 Activity Diagram: US-13

4.3.14 US-14 Activity Diagram

Staff clicks Mark Urgent on a housekeeping task. The urgency flag is set, the task list is re-sorted, and the update broadcasts to all housekeepers.

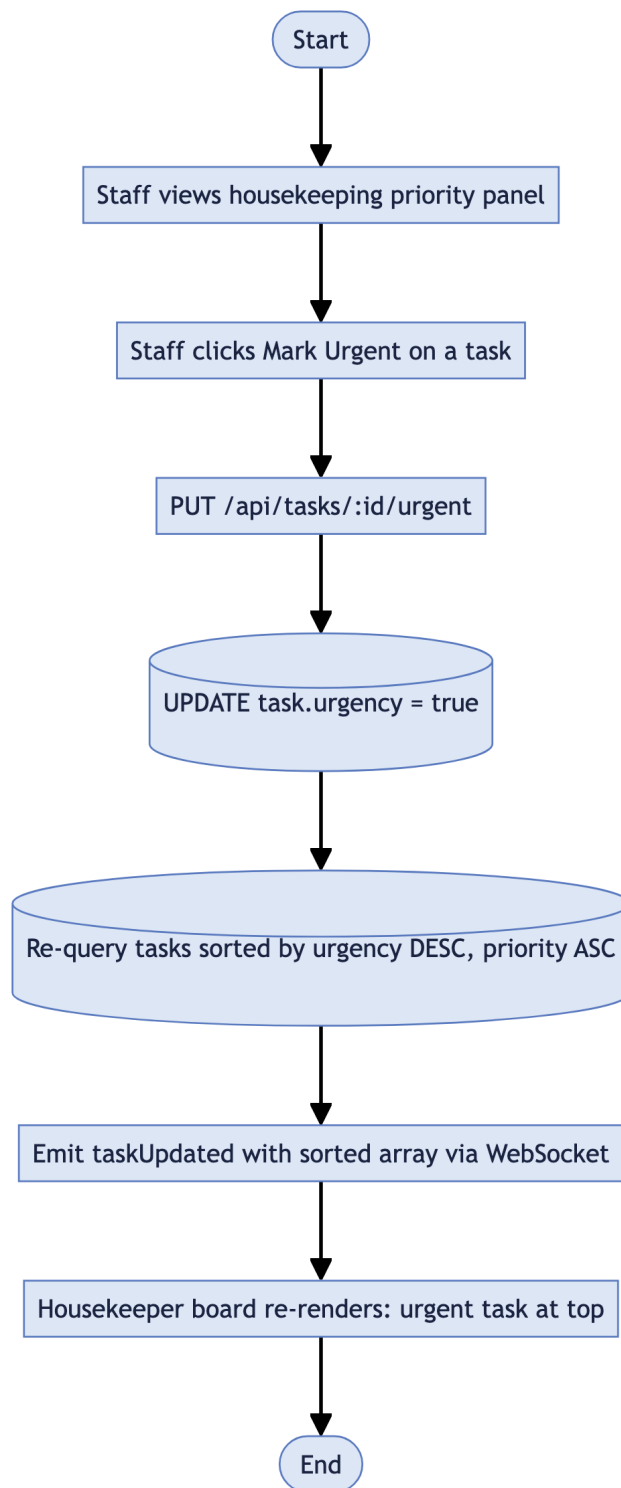


Figure 4.45 Activity Diagram: US-14

4.3.15 US-15 Activity Diagram

Hotel Admin adds, edits, or removes rooms from inventory. They can also block date ranges for maintenance. All changes immediately reflected.

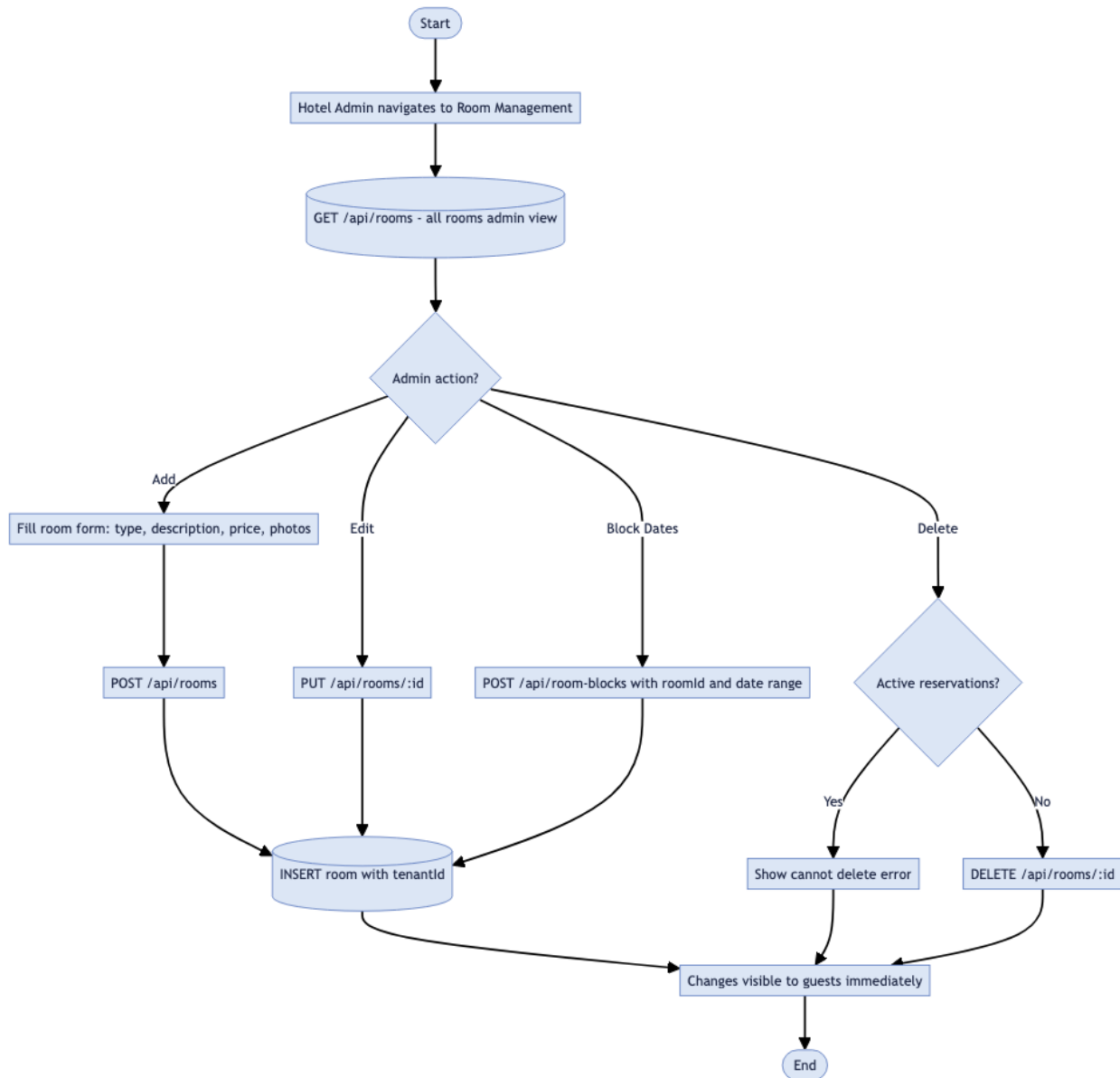


Figure 4.46 Activity Diagram: US-15

4.3.16 US-16 Activity Diagram

Hotel Admin creates staff accounts with a role assignment. The system auto-generates a username. Admin can deactivate or delete accounts.

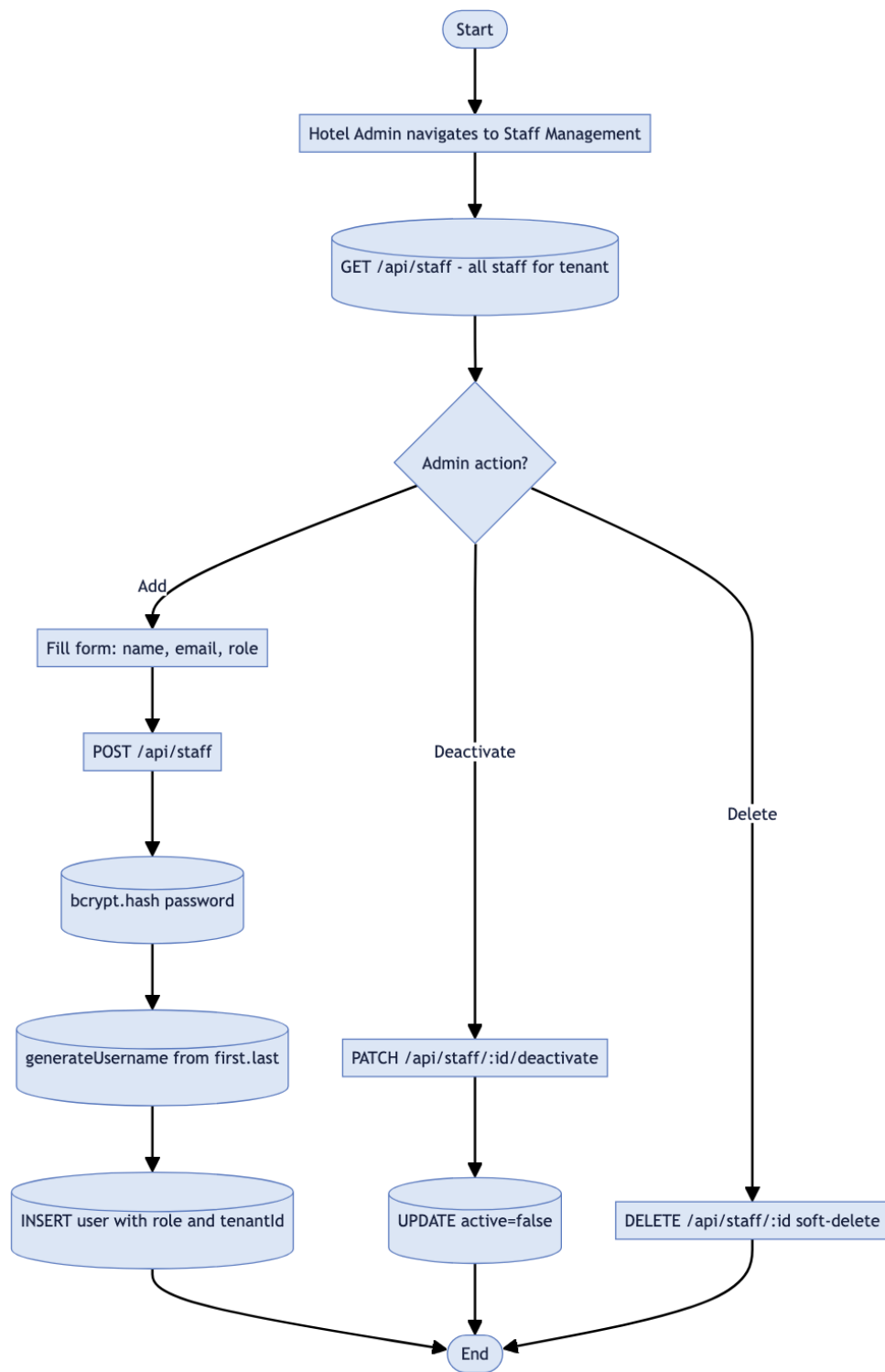


Figure 4.47 Activity Diagram: US-16

4.3.17 US-17 Activity Diagram

Hotel Admin (Premium/Custom) sets an occupancy threshold and price adjustment. The nightly cron job evaluates all active rules and adjusts room prices.

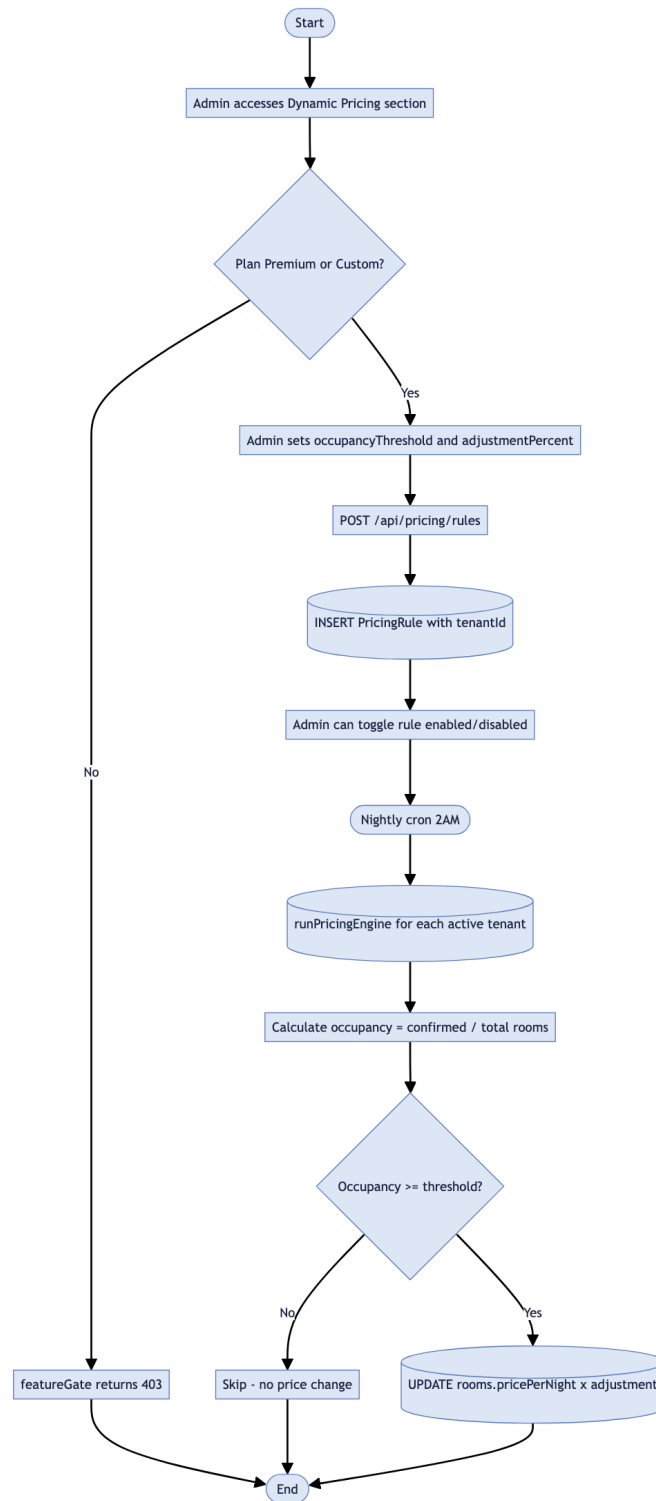


Figure 4.48 Activity Diagram: US-17

4.3.18 US-18 Activity Diagram

Hotel Admin (Premium/Custom) views KPI cards with date-range filtering. Export generates a CSV download of reservation data.

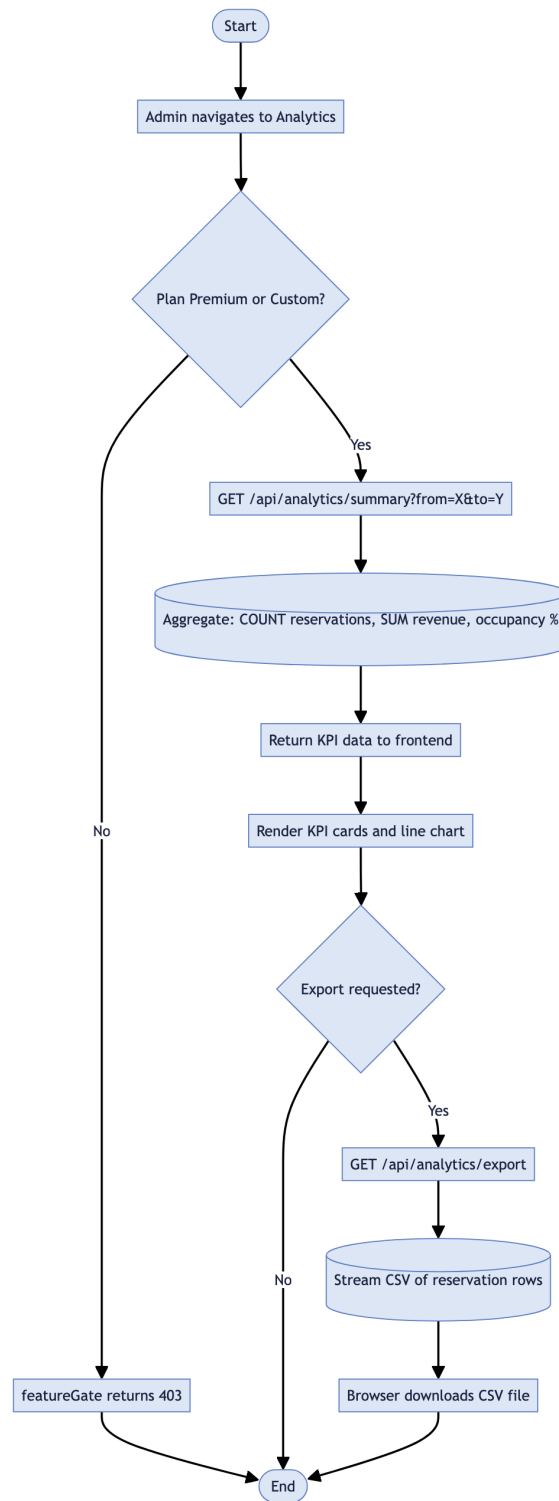


Figure 4.49 Activity Diagram: US-18

4.3.19 US-19 Activity Diagram

Hotel Admin adds parking spots with labels and per-spot pricing, views occupancy, and removes spots. Changes immediately visible to guests on the map.

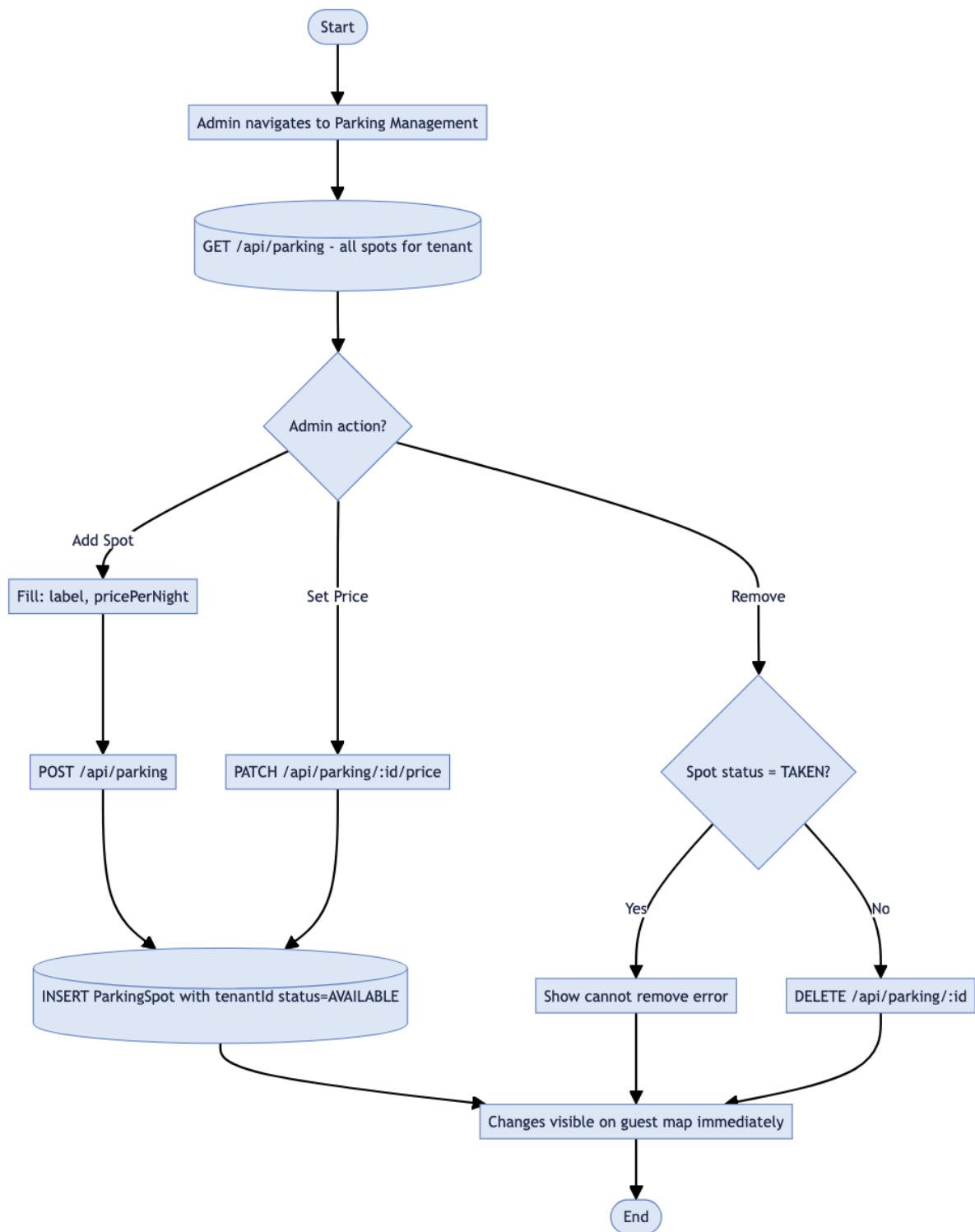


Figure 4.50 Activity Diagram: US-19

4.3.20 US-20 Activity Diagram

Housekeepers view a sorted task list urgency-flagged rooms appear first, then by priority, then creation time. The list updates in real time via WebSocket.

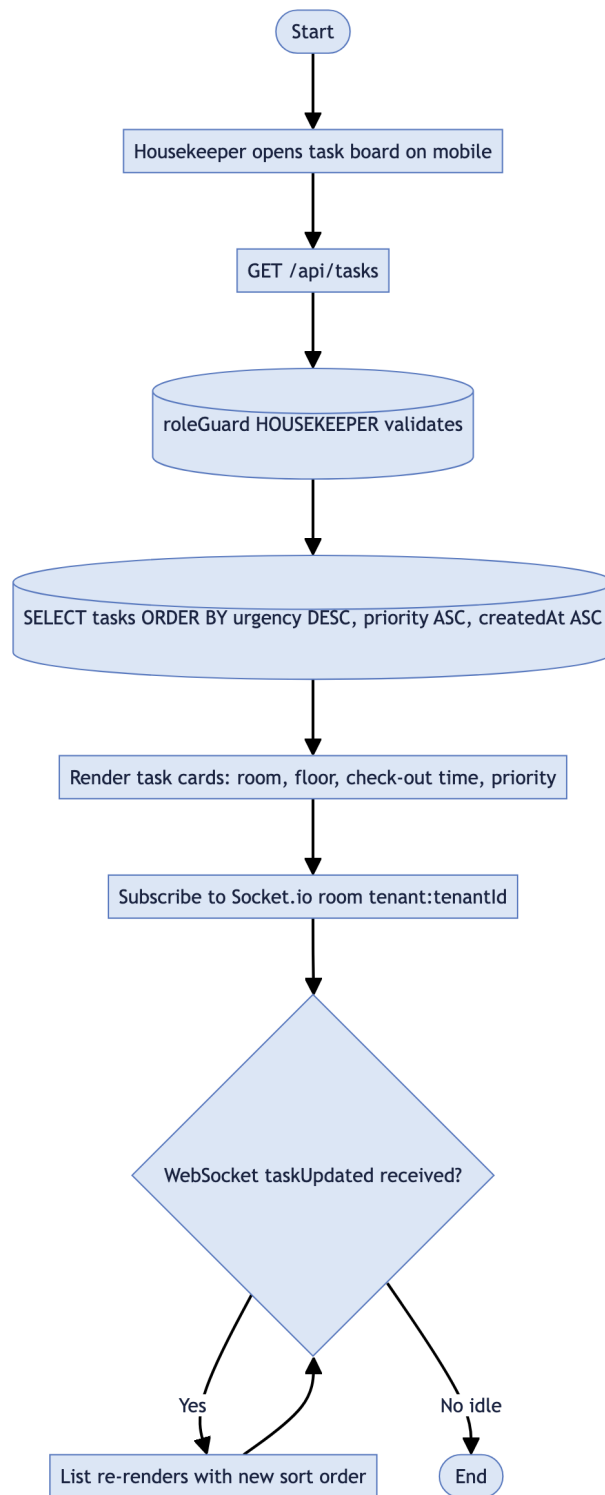


Figure 4.51 Activity Diagram: US-20

4.3.21 US-21 Activity Diagram

Housekeeper marks a room as cleaned. Task status changes to DONE, room to AVAILABLE, and notifications are sent to front desk and eligible guests.

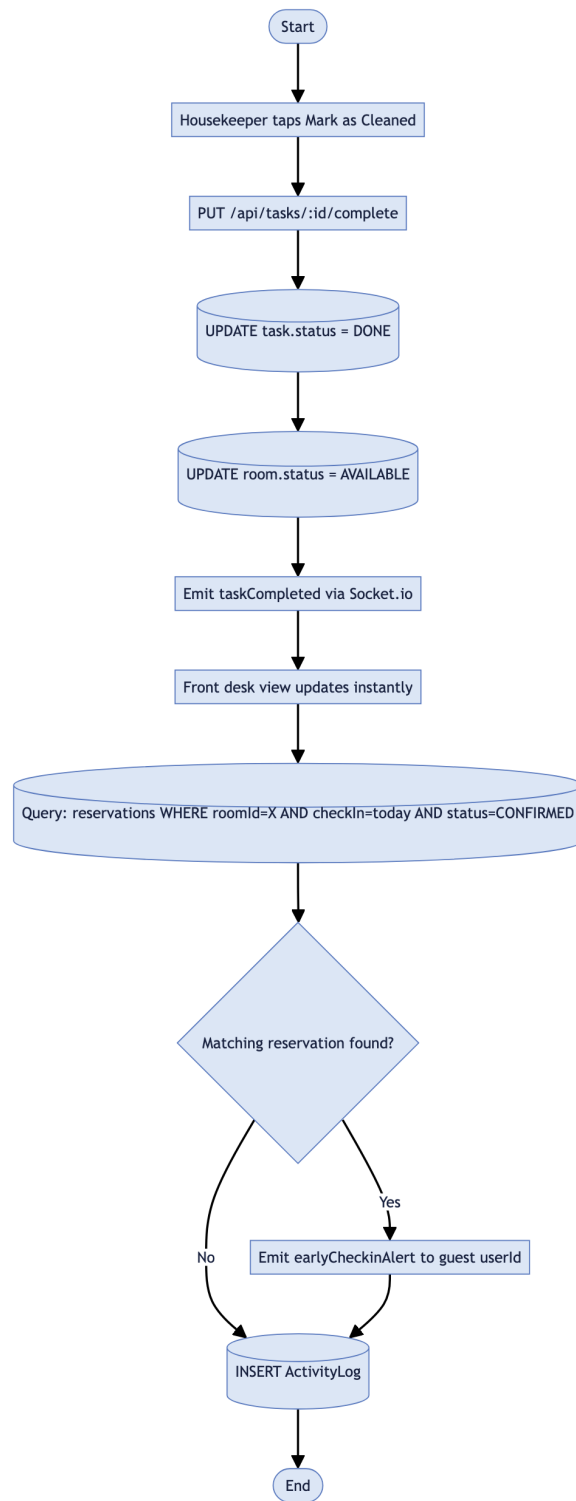


Figure 4.52 Activity Diagram: US-21

4.3.22 US-22 Activity Diagram

Guest selects a sunbed from the pool map, locks it for 5 minutes, chooses a time slot, and confirms to create a SunbedReservation record.

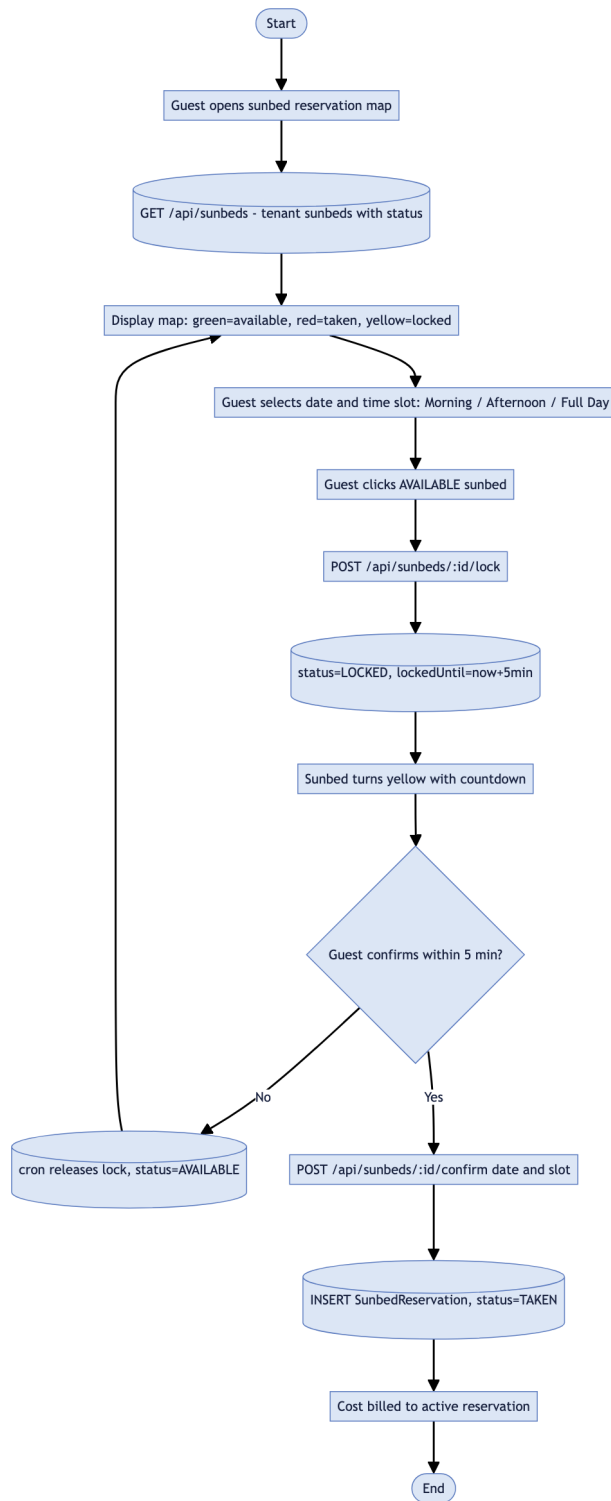


Figure 4.53 Activity Diagram: US-22

4.3.23 US-23 Activity Diagram

Super Admin views all tenants, can approve pending signups, suspend active tenants (immediately blocking access), or permanently delete tenant data.

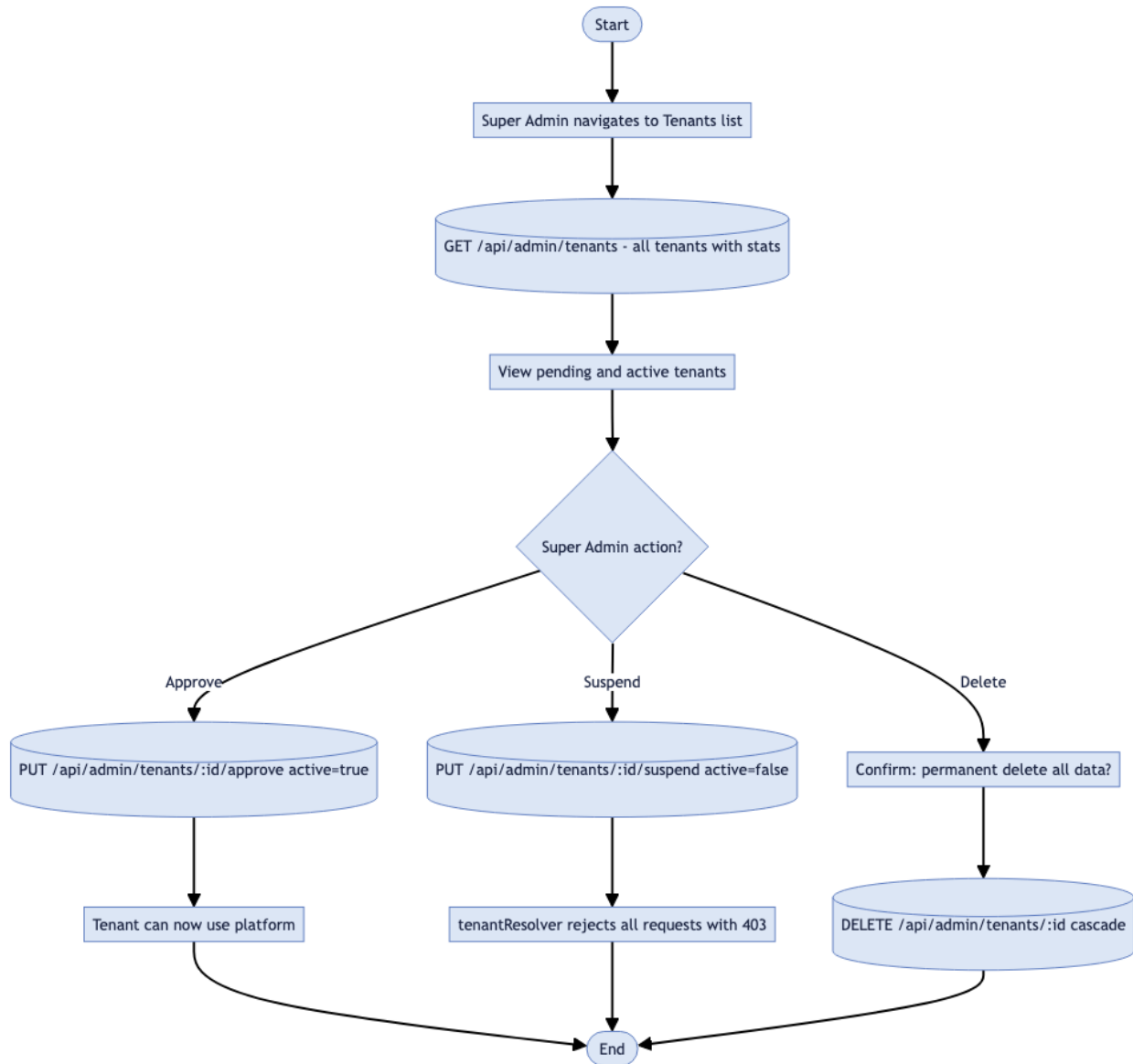


Figure 4.54 Activity Diagram: US-23

4.3.24 US-24 Activity Diagram

Super Admin changes a tenant's plan tier. The change is persisted immediately and the featureGate reads the updated plan on the next API call.

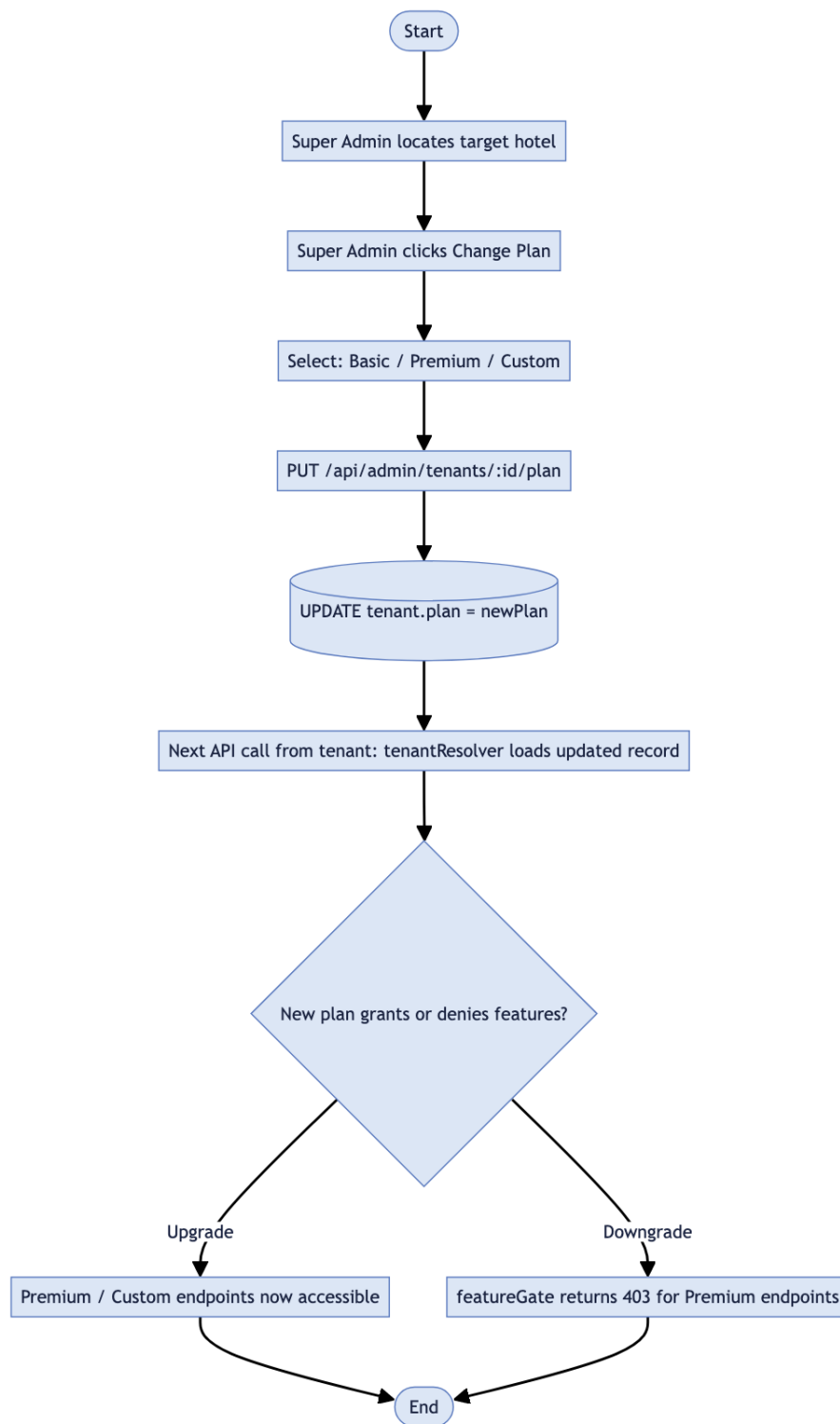


Figure 4.55 Activity Diagram: US-24

4.3.25 US-25 Activity Diagram

Super Admin views platform-level KPIs including total tenants, reservations across all hotels, and breakdown by subscription plan. No individual guest data exposed.

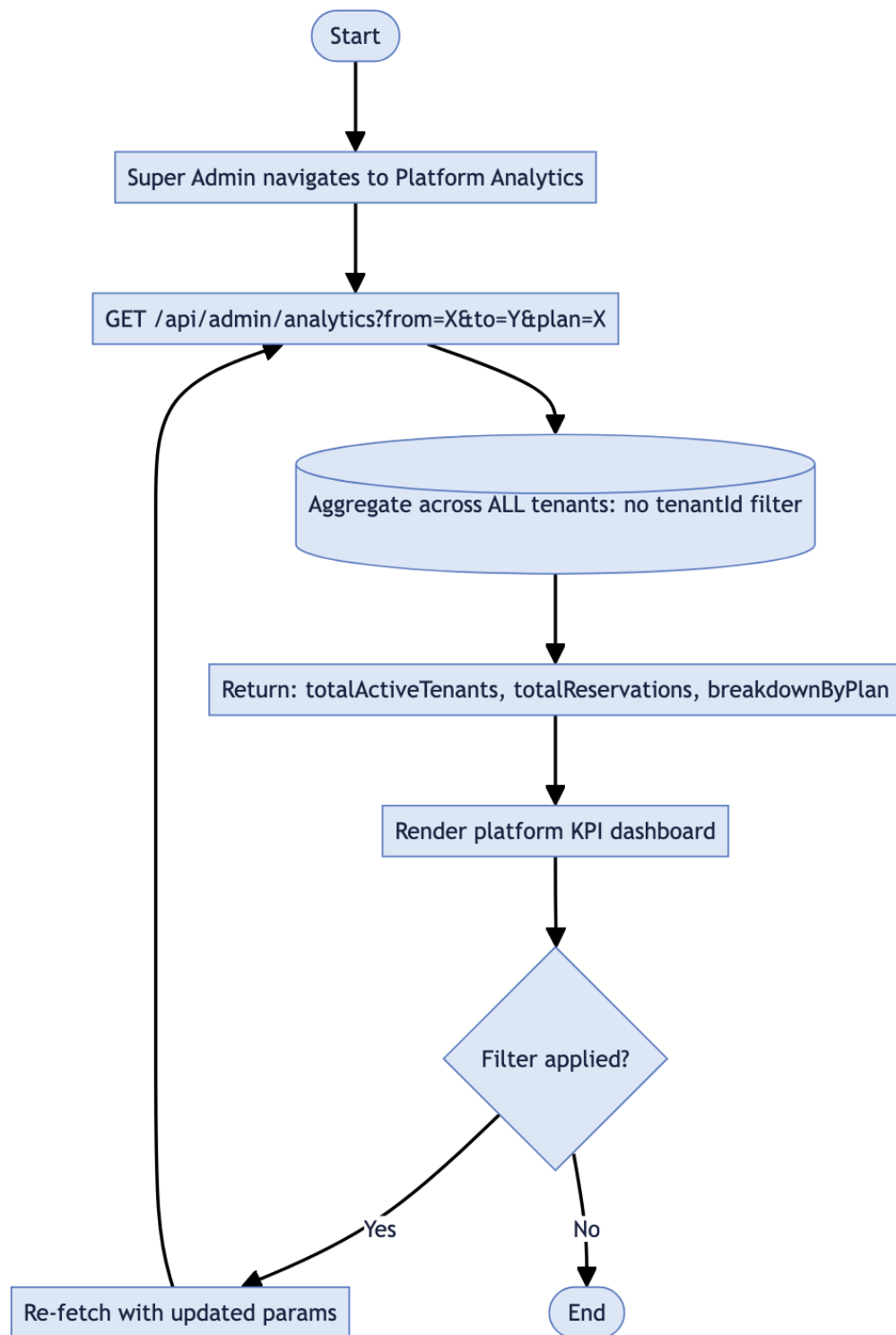


Figure 4.56 Activity Diagram: US-25

4.3.26 US-26 Activity Diagram

Super Admin fills out the onboard form. The system provisions the tenant record and subdomain immediately. An invitation email is sent to the hotel owner.

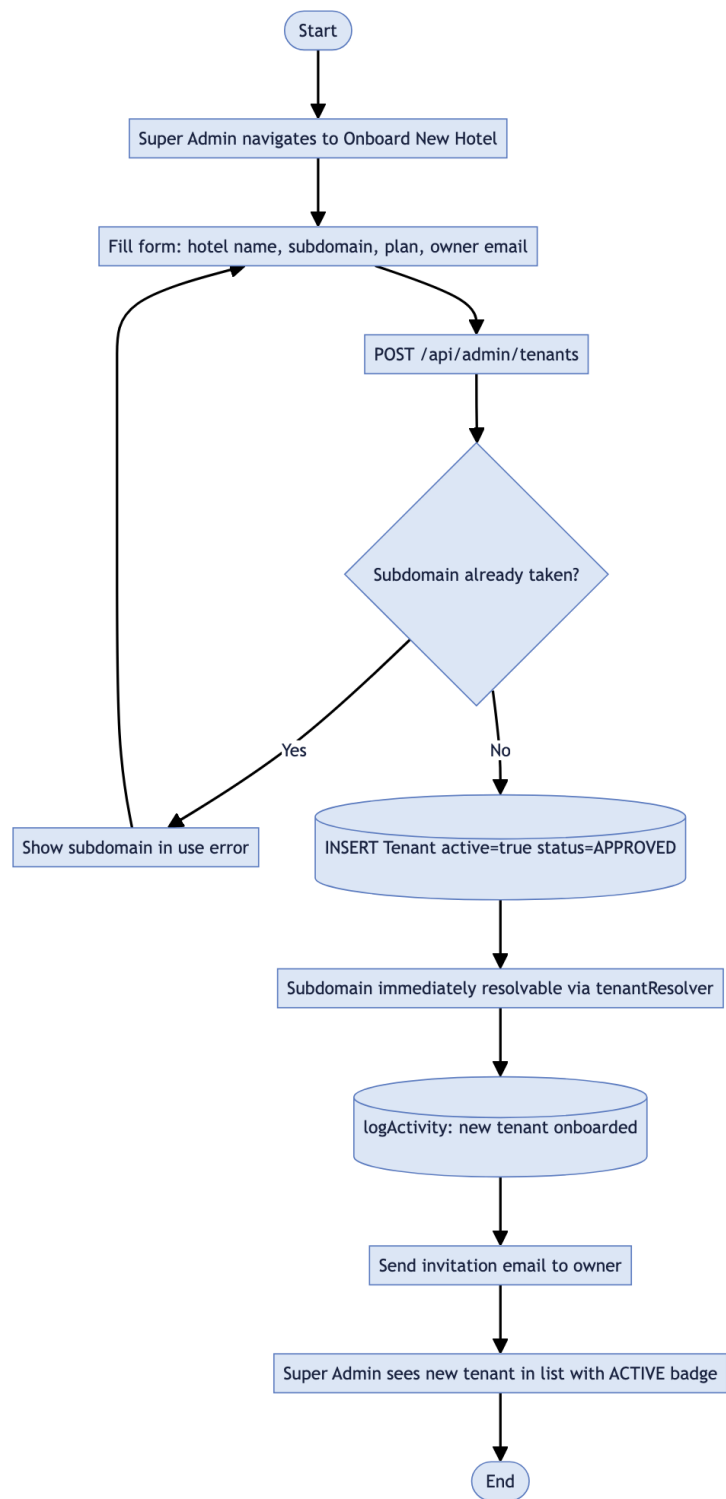


Figure 4.57 Activity Diagram: US-26

4.4 Sequence Diagrams

Sequence diagrams show the precise message exchange between actors, frontend, API middleware stack, database, and Socket.io. They illustrate how the multi-tenant middleware chain processes each request before reaching business logic.

4.4.1 US-1 Sequence Diagram

The frontend POSTs to `/api/auth/register`. `tenantResolver` resolves the subdomain to a `tenantId`. The Prisma tenant client checks email uniqueness within the tenant, hashes the password with `bcrypt`, creates the `User` record, and returns a `JWT`.

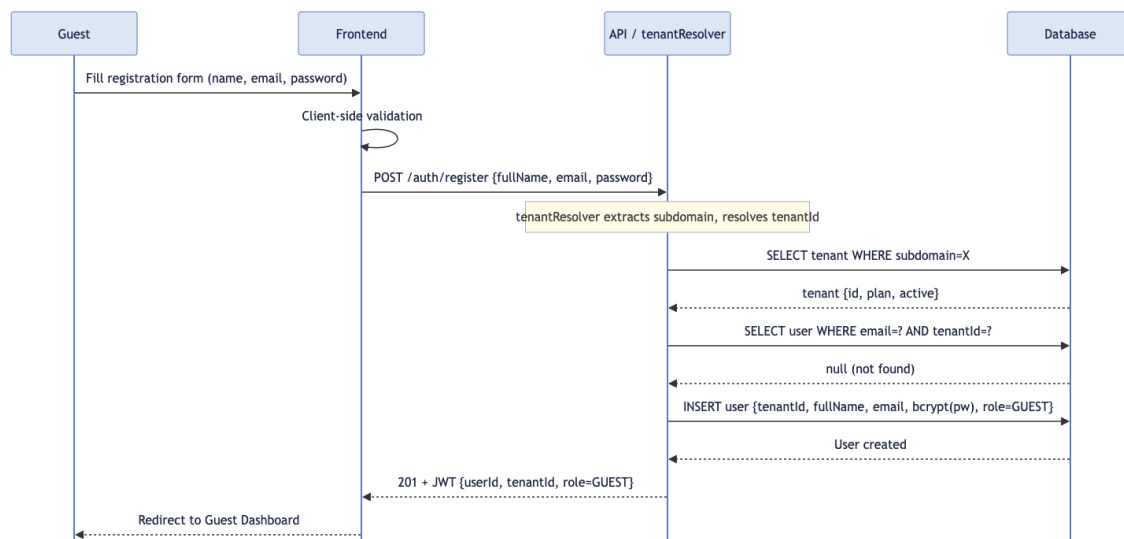


Figure 4.58 Sequence Diagram: US-1

4.4.2 US-2 Sequence Diagram

POST `/api/auth/login`. `tenantResolver` attaches tenant context. `bcrypt.compare` validates the password. A signed `JWT` with 7-day expiry is returned and stored client-side.

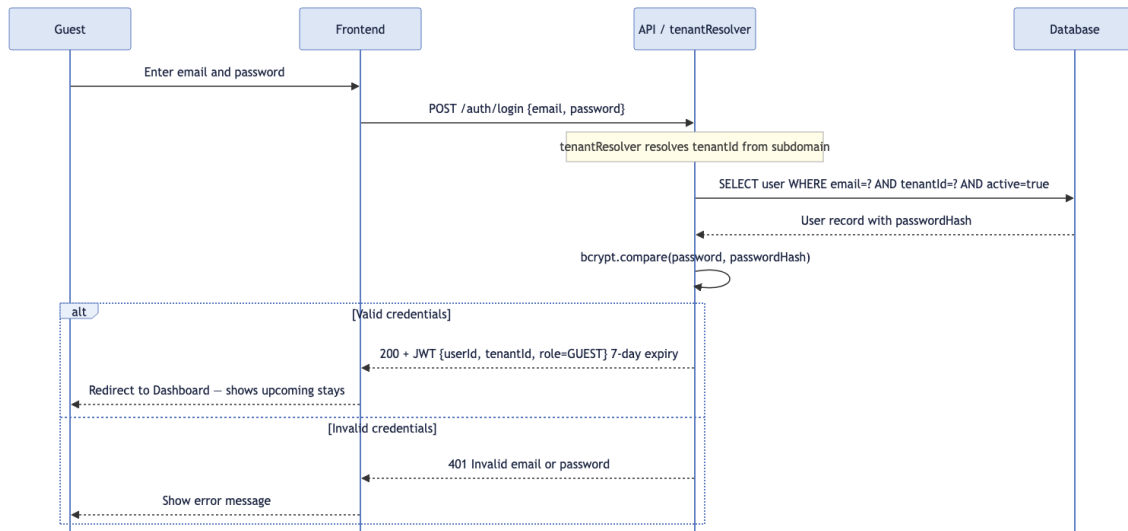


Figure 4.59 Sequence Diagram: US-2

4.4.3 US-3 Sequence Diagram

GET /api/rooms with X-Tenant-Subdomain header. No auth required for browsing. Prisma tenant query filters status=AVAILABLE and returns matching rooms.

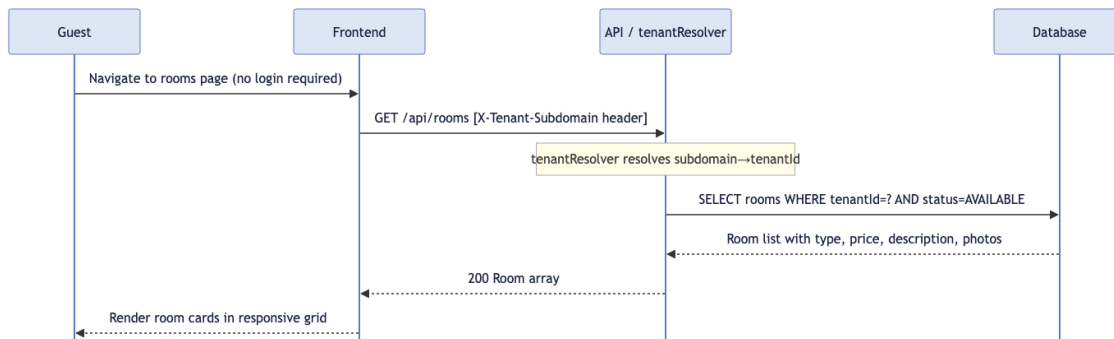


Figure 4.60 Sequence Diagram: US-3

4.4.4 US-4 Sequence Diagram

GET /api/rooms?checkIn=X&checkOut=Y. API queries for overlapping PENDING/CONFIRMED/CHECKED_IN reservations and active room blocks, excludes those room IDs, and returns available rooms.

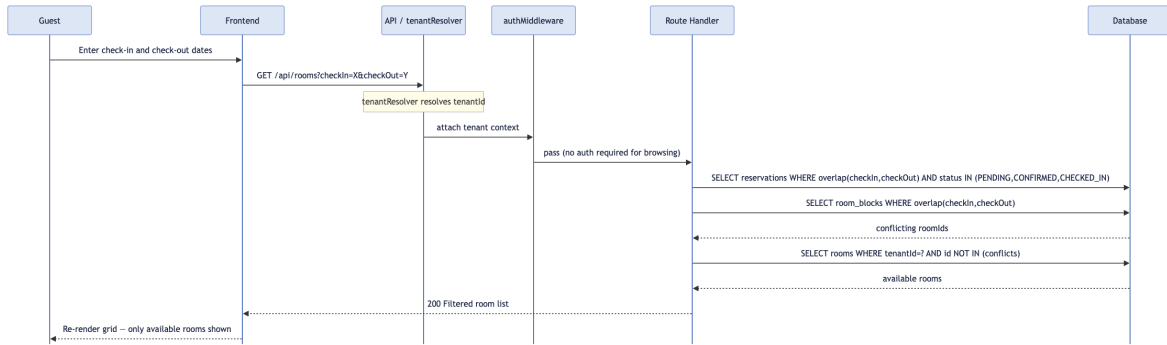


Figure 4.61 Sequence Diagram: US-4

4.4.5 US-5 Sequence Diagram

POST /api/reservations. authMiddleware verifies GUEST role. System checks guest blacklist, validates no overlap, then INSERTs the Reservation record with status=PENDING.

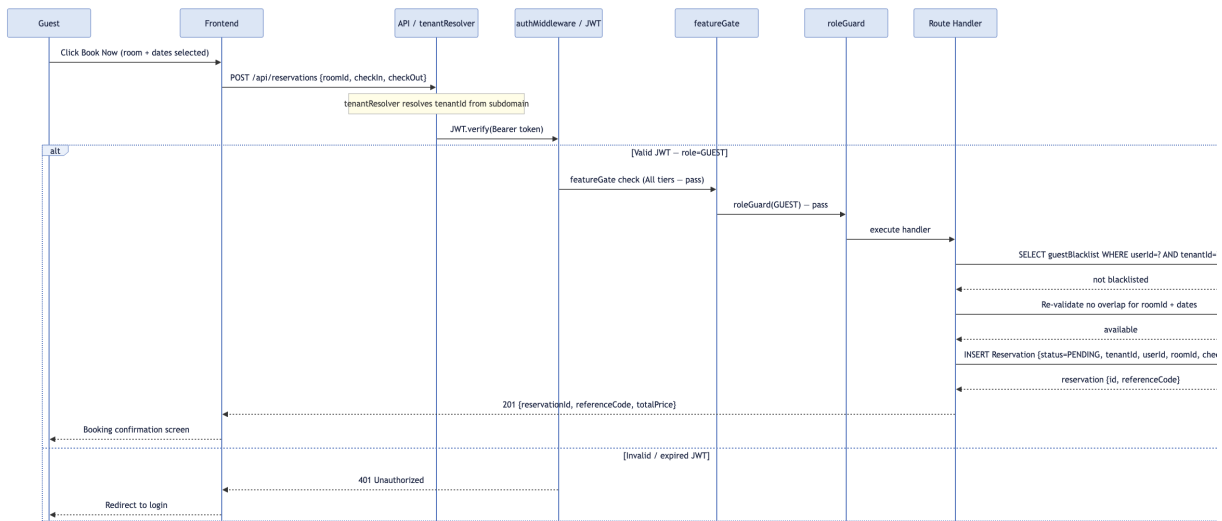


Figure 4.62 Sequence Diagram: US-5

4.4.6 US-6 Sequence Diagram

POST /api/parking/:id/lock sets status=LOCKED and lockedUntil=now+5min. node-cron releases expired locks every minute. POST /api/parking/:id/confirm sets status=TAKEN.

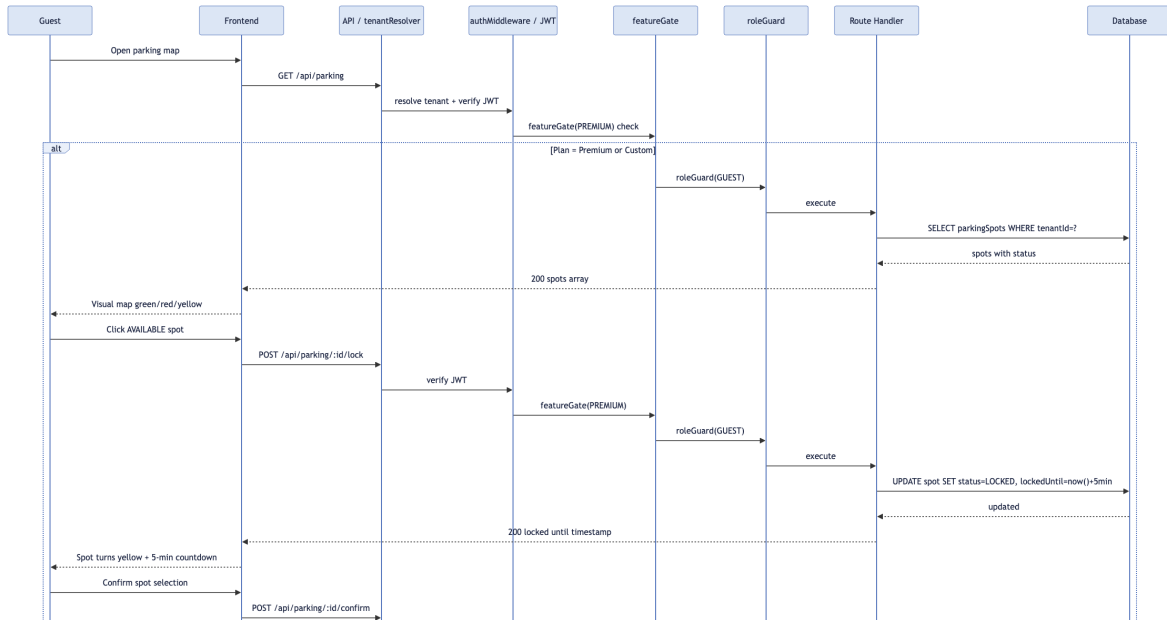


Figure 4.63 Sequence Diagram: US-6

4.4.7 US-7 Sequence Diagram

GET /api/services returns tenant services. POST /api/orders validates the reservationId belongs to the authenticated guest, then creates the Order record.

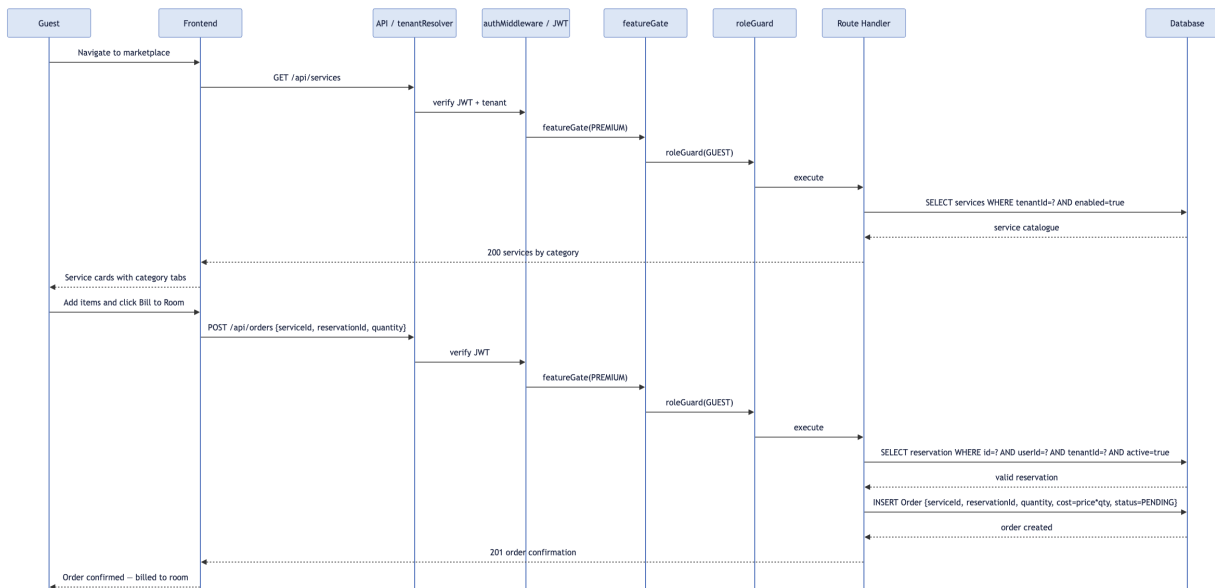


Figure 4.64 Sequence Diagram: US-7

4.4.8 US-8 Sequence Diagram

PUT /api/tasks/:id/complete → DB update → query today's CONFIRMED reservations for that room → emit earlyCheckinAlert via Socket.io tenant room.

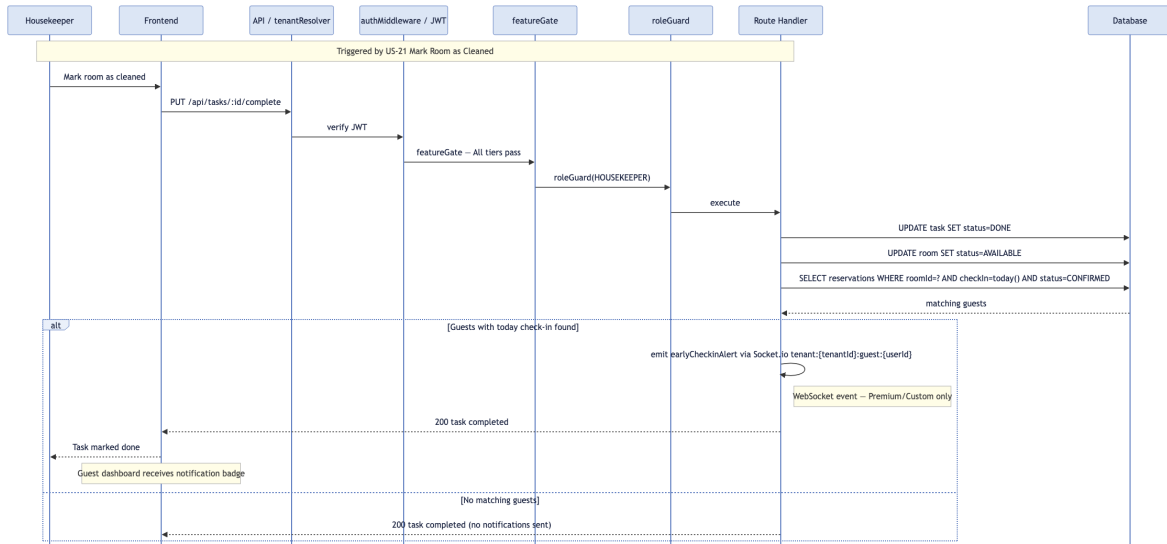


Figure 4.65 Sequence Diagram: US-8

4.4.9 US-9 Sequence Diagram

GET /api/reservations. authMiddleware + roleGuard(STAFF). Prisma tenant client JOINS users and rooms, returns full reservation list.

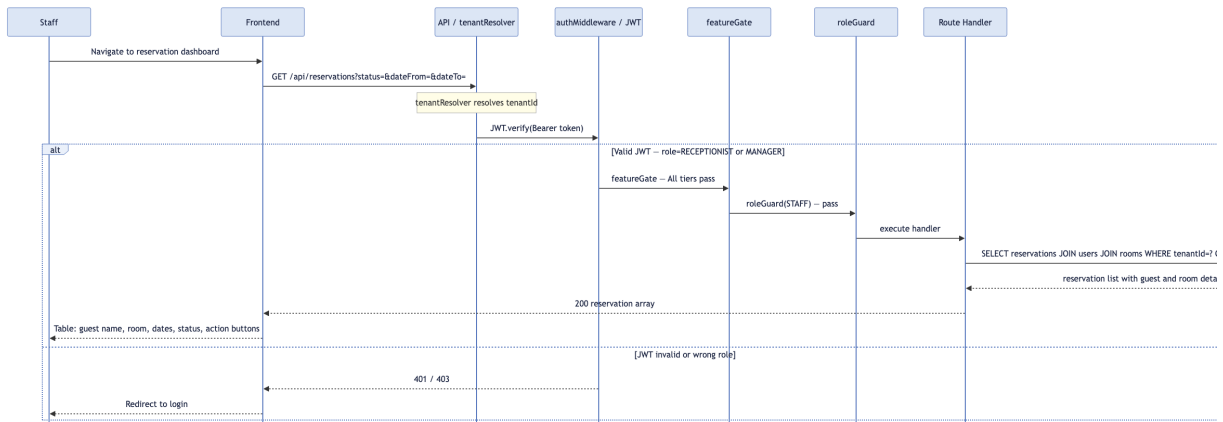


Figure 4.66 Sequence Diagram: US-9

4.4.10 US-10 Sequence Diagram

PUT /api/reservations/:id/confirm → DB update status=CONFIRMED → emit reservationUpdated via Socket.io → activity log entry.

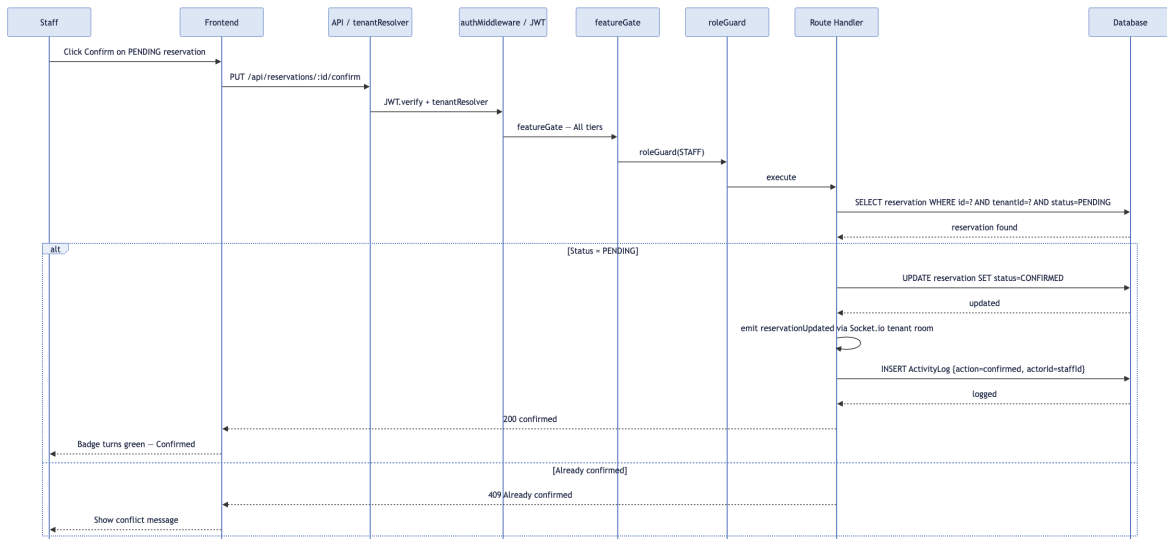


Figure 4.67 Sequence Diagram: US-10

4.4.11 US-11 Sequence Diagram

PUT /api/reservations/:id/checkin → reservation=CHECKED_IN → room=OCCUPIED → Socket.io broadcast → activity log.

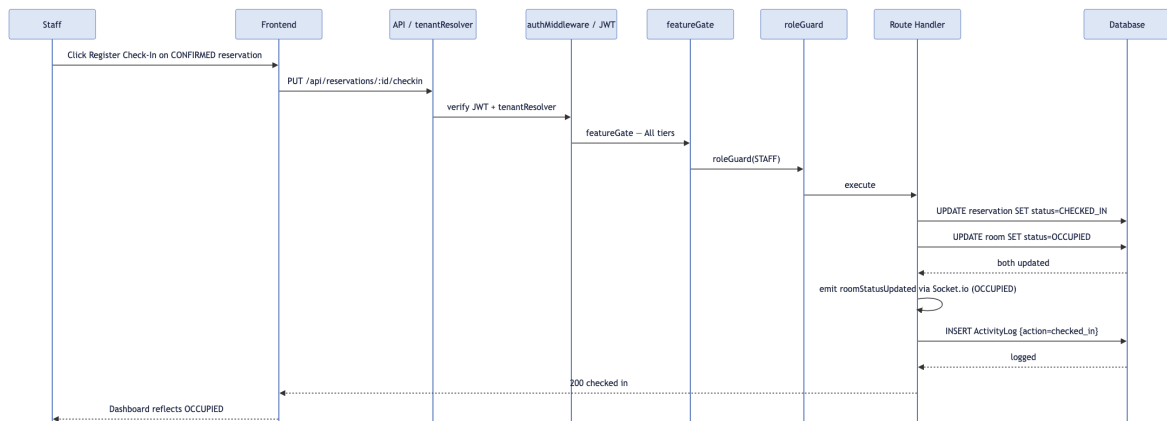


Figure 4.68 Sequence Diagram: US-11

4.4.12 US-12 Sequence Diagram

PUT /api/reservations/:id/checkout → reservation=CHECKED_OUT → room=AVAILABLE → INSERT HousekeepingTask priority=1 → Socket.io broadcast.

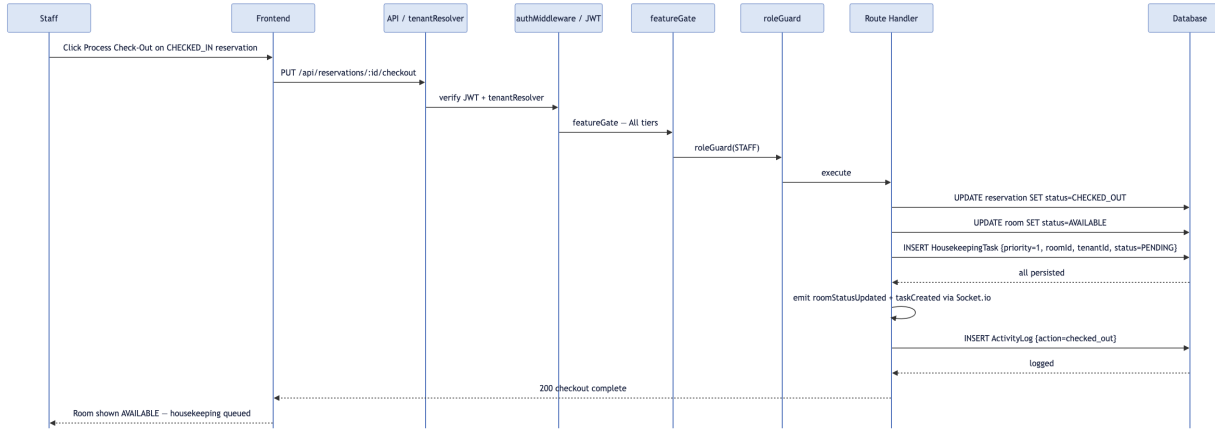


Figure 4.69 Sequence Diagram: US-12

4.4.13 US-13 Sequence Diagram

PUT /api/rooms/:id/status → DB update room.status → emit roomStatusUpdated via Socket.io
tenant room → all subscribed clients update.

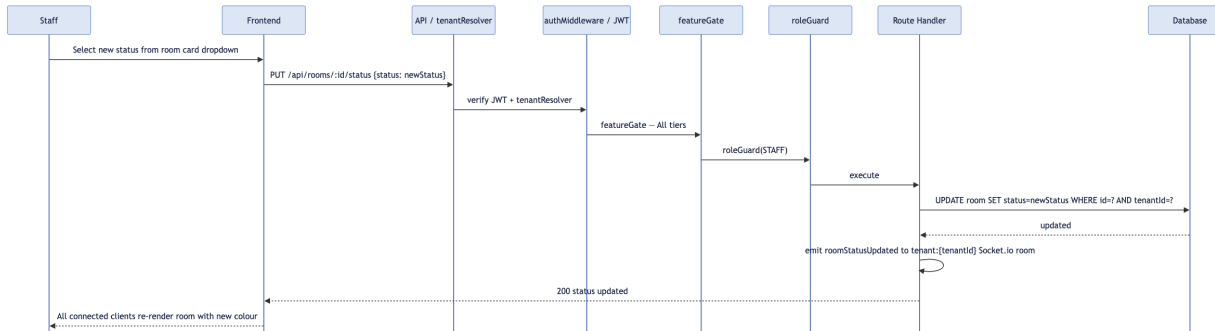


Figure 4.70 Sequence Diagram: US-13

4.4.14 US-14 Sequence Diagram

PUT /api/tasks/:id/urgent → DB update urgency=true → re-query sorted tasks → emit taskUpdated with sorted array → housekeeper board re-renders.

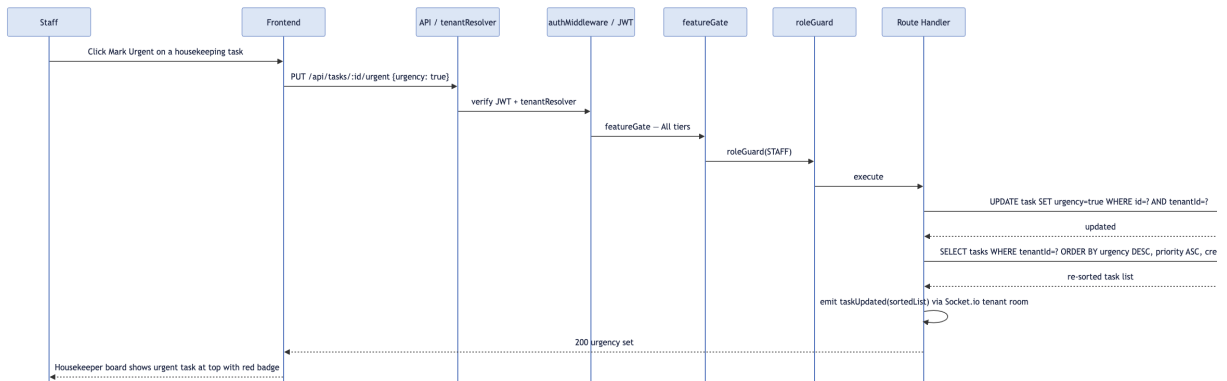


Figure 4.71 Sequence Diagram: US-14

4.4.15 US-15 Sequence Diagram

POST /api/rooms → create room. PUT /api/rooms/:id → update. DELETE /api/rooms/:id (if no active reservations). POST /api/room-blocks → block dates.

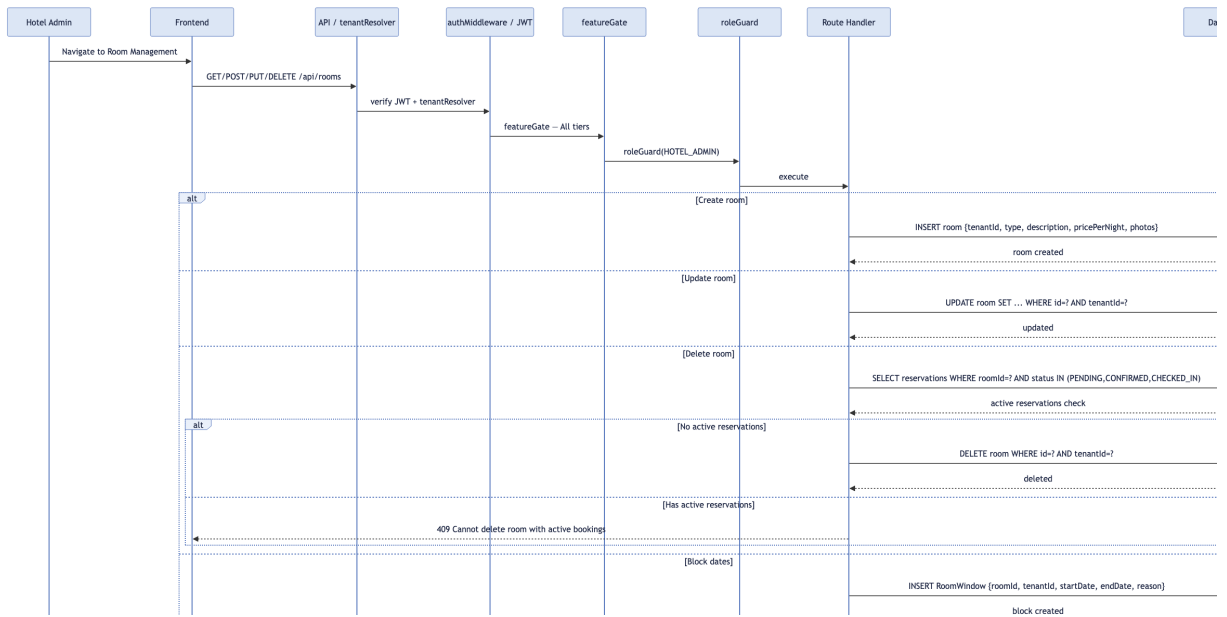


Figure 4.72 Sequence Diagram: US-15

4.4.16 US-16 Sequence Diagram

POST /api/staff → bcrypt.hash password → generateUsername → INSERT user with role. PATCH /api/staff/:id/deactivate → active=false.

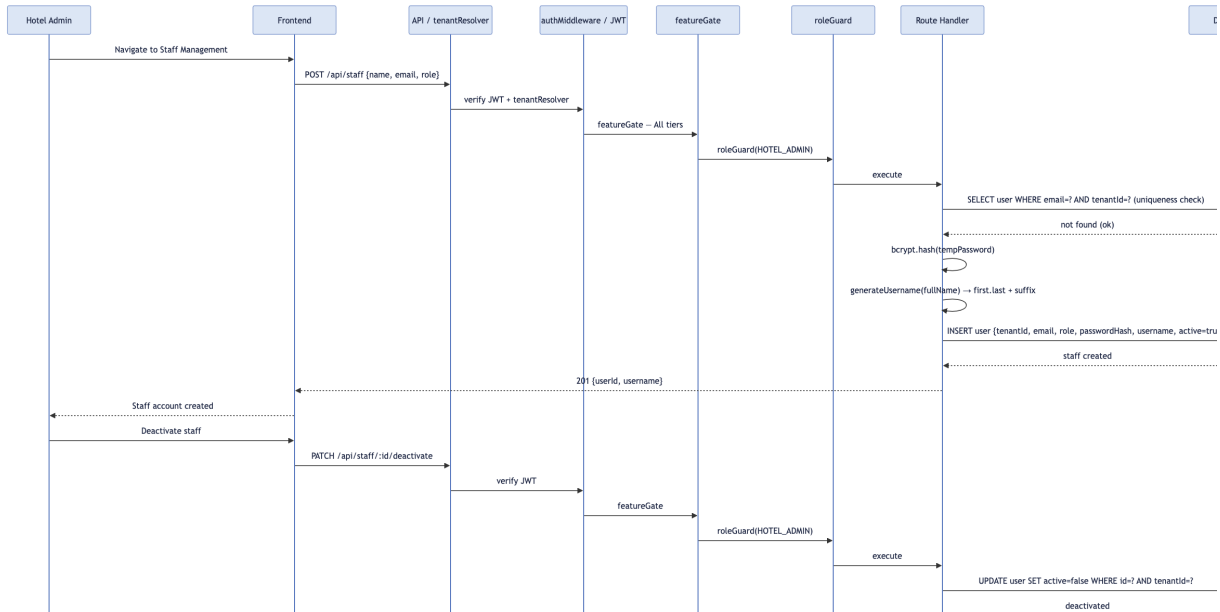


Figure 4.73 Sequence Diagram: US-16

4.4.17 US-17 Sequence Diagram

POST /api/pricing/rules → featureGate(PREMIUM) → INSERT PricingRule. Nightly cron → runPricingEngine() per tenant → occupancy calc → if threshold met → UPDATE room prices.

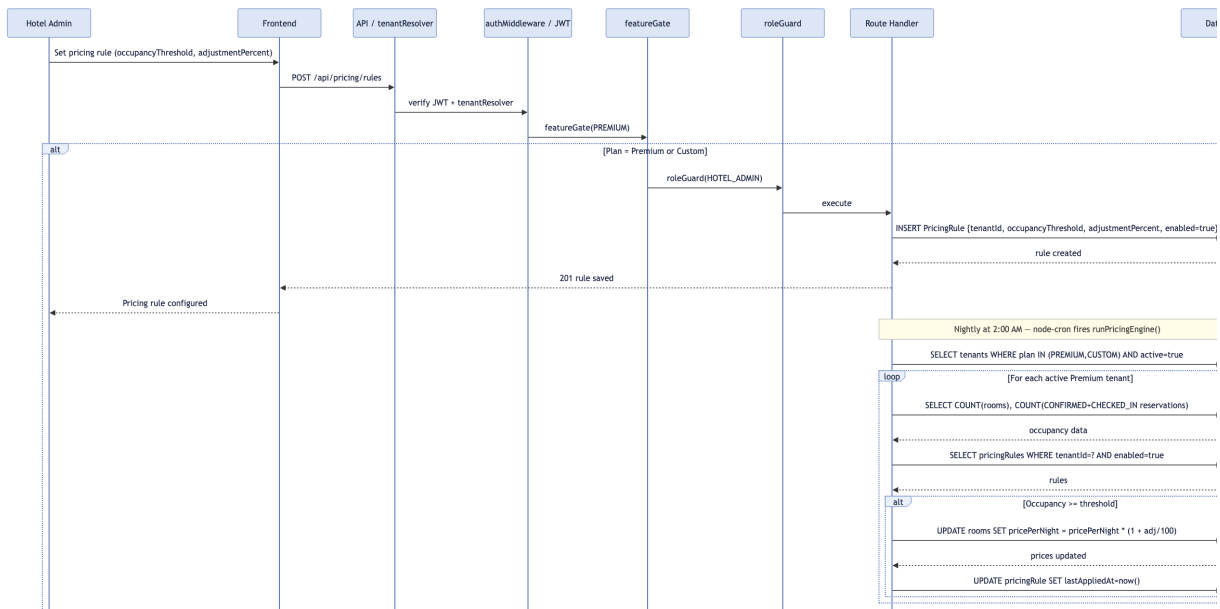


Figure 4.74 Sequence Diagram: US-17

4.4.18 US-18 Sequence Diagram

GET /api/analytics/summary → featureGate(PREMIUM) → COUNT reservations, SUM revenue, occupancy % → return stats. GET /api/analytics/export → CSV stream.

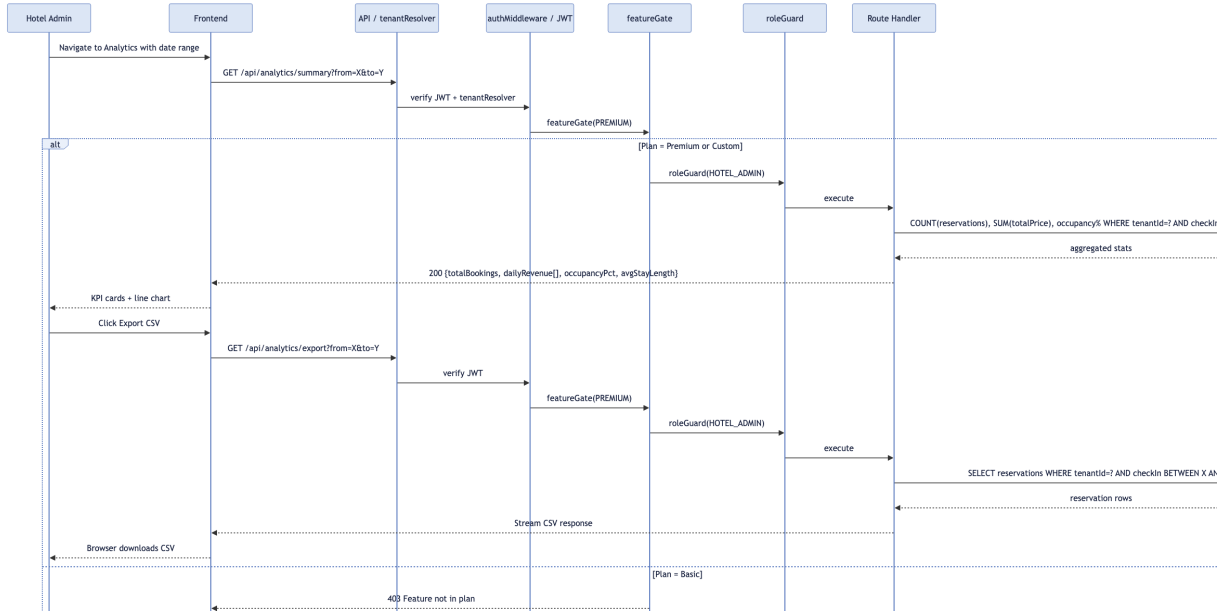


Figure 4.75 Sequence Diagram: US-18

4.4.19 US-19 Sequence Diagram

POST /api/parking → INSERT ParkingSpot. DELETE /api/parking/:id. PATCH /api/parking/:id/price → update. GET /api/parking → all spots with status.

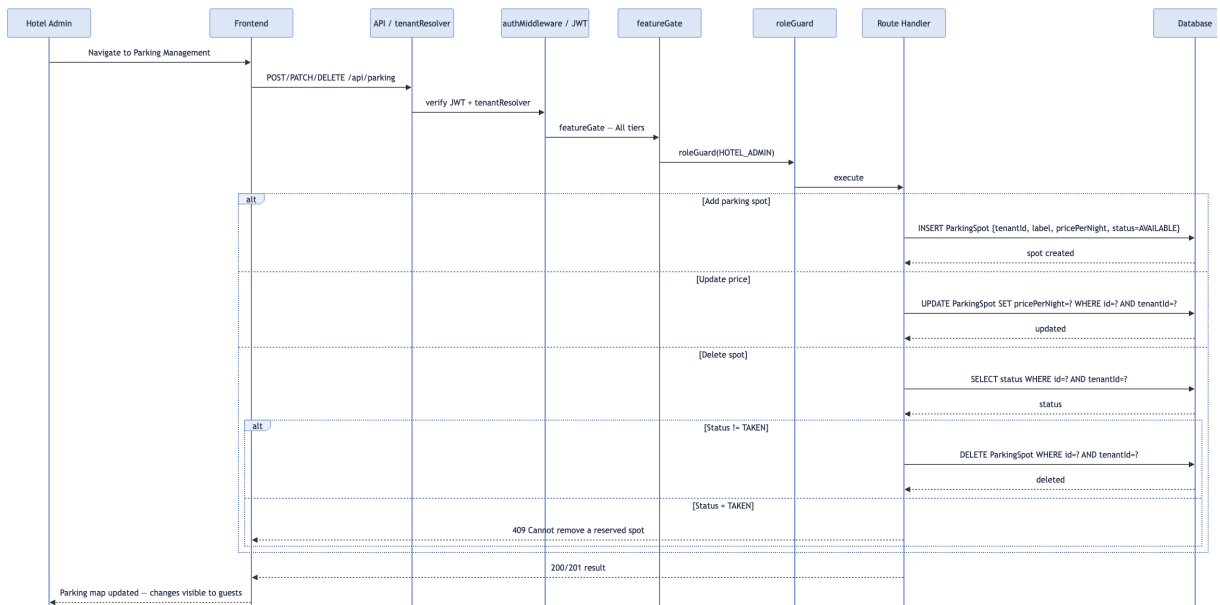


Figure 4.76 Sequence Diagram: US-19

4.4.20 US-20 Sequence Diagram

GET /api/tasks → roleGuard(HOUSEKEEPER) → SELECT tasks ORDER BY urgency DESC, priority ASC. WebSocket taskUpdated event triggers re-render.

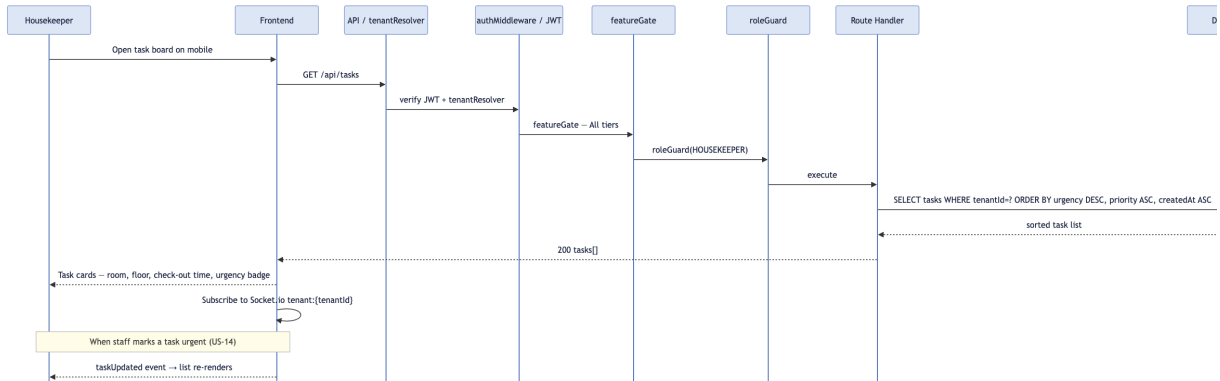


Figure 4.77 Sequence Diagram: US-20

4.4.21 US-21 Sequence Diagram

PUT /api/tasks/:id/complete → status=DONE → emit taskCompleted → query same-day confirmed reservations → emit earlyCheckinAlert to guest.

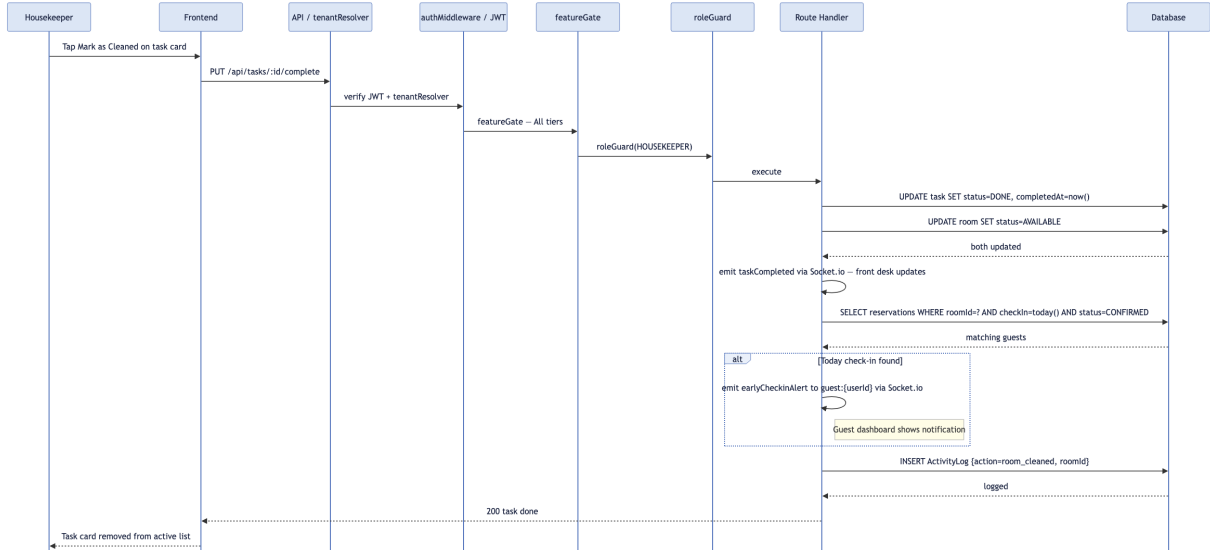


Figure 4.78 Sequence Diagram: US-21

4.4.22 US-22 Sequence Diagram

POST /api/sunbeds/:id/lock → LOCKED+lockedUntil. POST /api/sunbeds/:id/confirm {date,slot} → TAKEN + INSERT SunbedReservation. Cron releases expired locks.

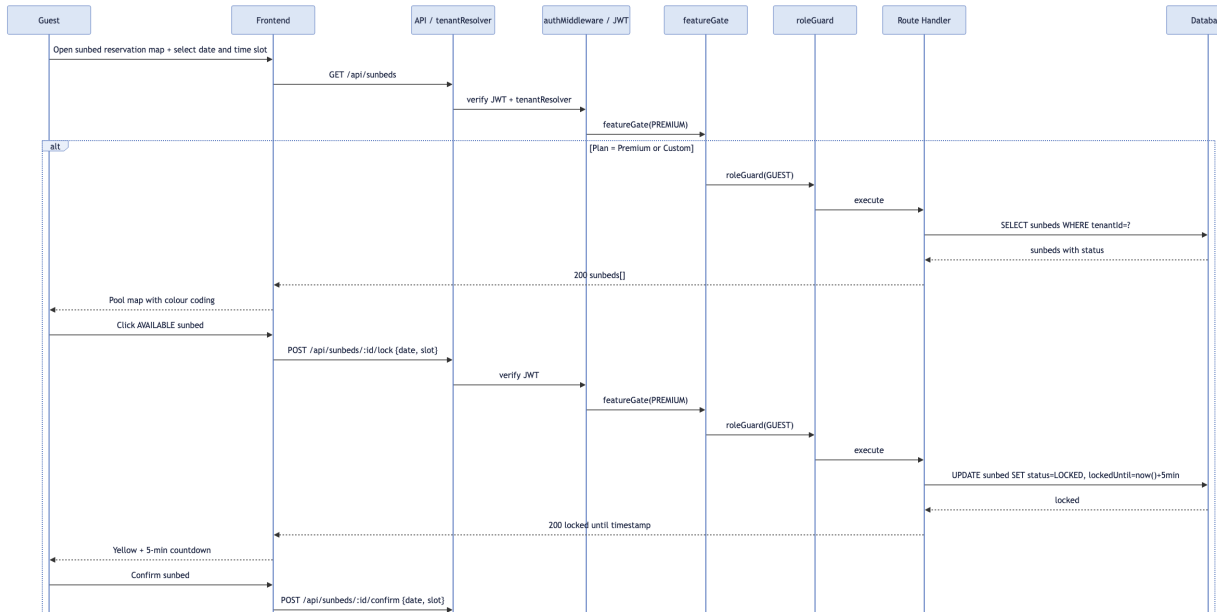


Figure 4.79 Sequence Diagram: US-22

4.4.23 US-23 Sequence Diagram

GET /api/admin/tenants → all tenants. PUT .../approve → active=true. PUT .../suspend → active=false → tenantResolver rejects future requests. DELETE → cascade delete.

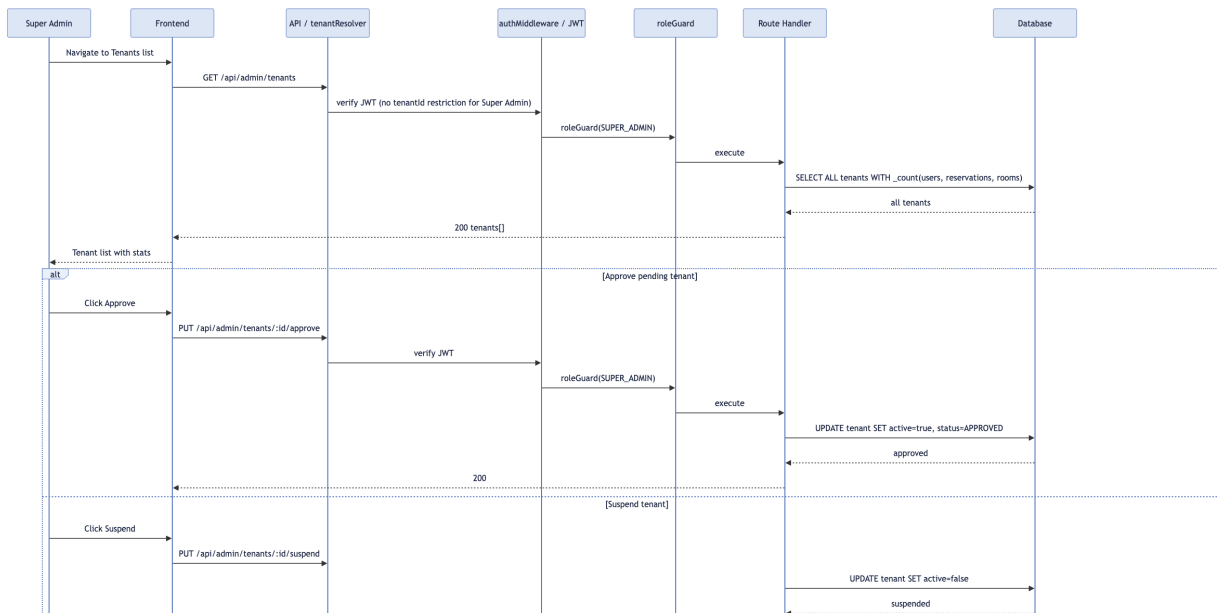


Figure 4.80 Sequence Diagram: US-23

4.4.24 US-24 Sequence Diagram

PUT /api/admin/tenants/:id/plan → UPDATE tenant.plan. Next API call from hotel → tenantResolver loads tenant → featureGate checks plan → grants/denies endpoint.

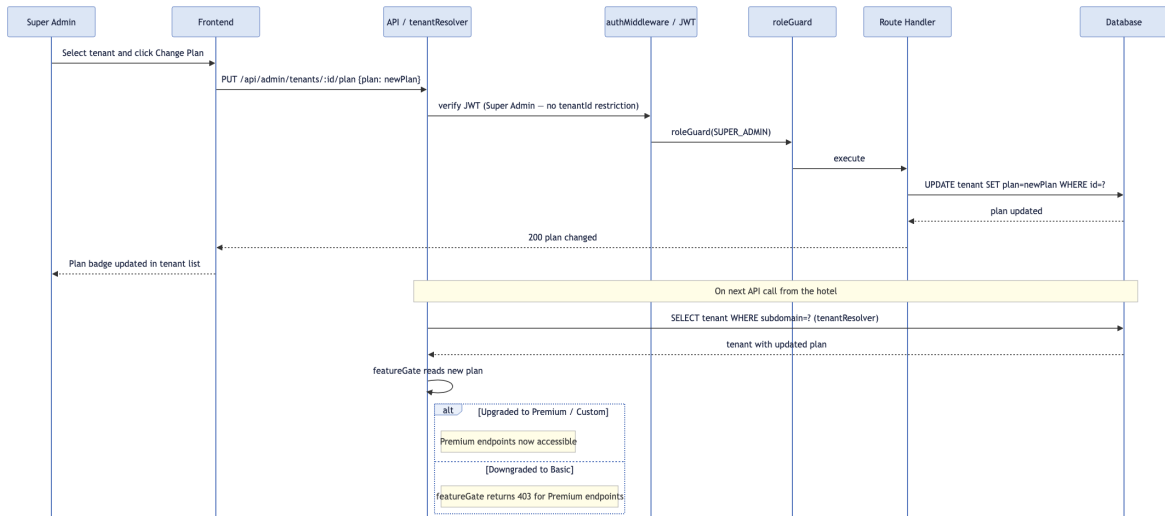


Figure 4.81 Sequence Diagram: US-24

4.4.25 US-25 Sequence Diagram

GET /api/admin/analytics → aggregate across ALL tenants: COUNT, GROUP BY plan → return totals. No cross-tenant guest data included.

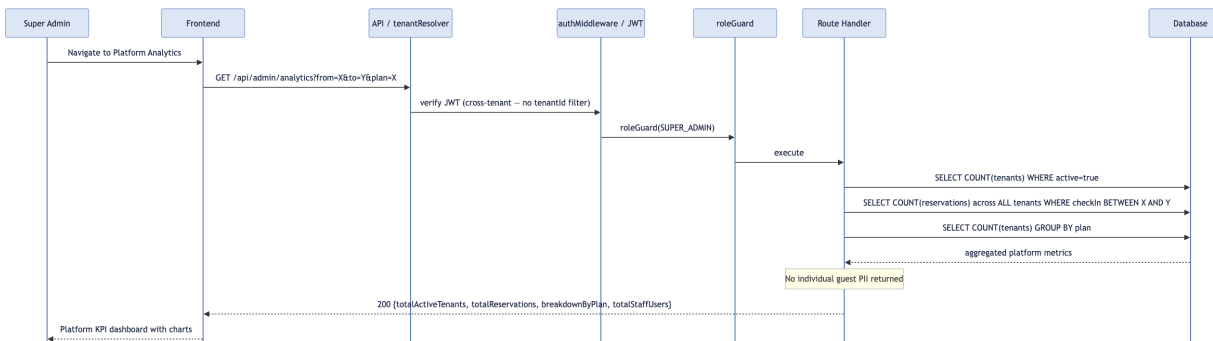


Figure 4.82 Sequence Diagram: US-25

4.4.26 US-26 Sequence Diagram

POST /api/admin/tenants → INSERT tenant → subdomain immediately resolvable via tenantResolver → logActivity → email notification (console.log in academic version).

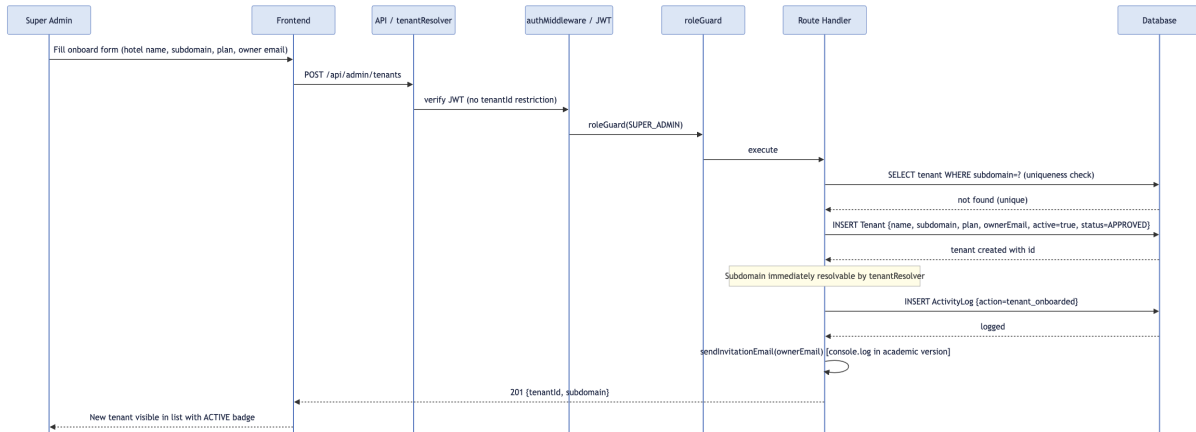


Figure 4.83 Sequence Diagram: US-26

4.5 State Diagrams

State diagrams model the lifecycle of key system entities. Each state box includes three compartments: entry actions (triggered on entering the state), do actions (ongoing while in the state), and exit actions (triggered on leaving). Initial states are shown as filled circles; final states are shown as double circles.

4.5.1 Room Status

Rooms transition between AVAILABLE, OCCUPIED, and MAINTENANCE. Check-in sets OCCUPIED; check-out returns to AVAILABLE; admin or staff can set MAINTENANCE at any time.

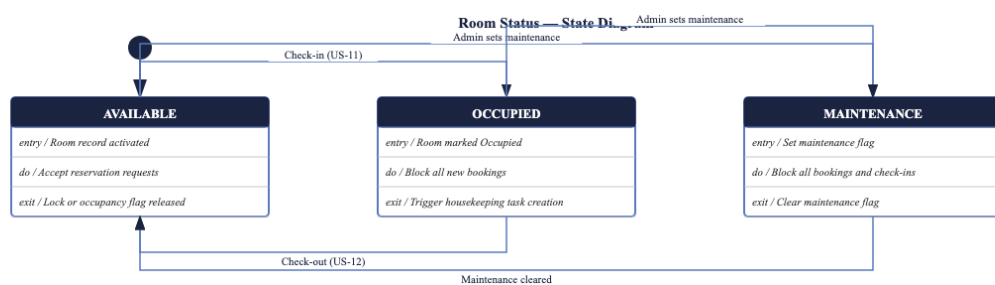


Figure 4.84 State Diagram: 4.5.1 Room Status

4.5.2 Reservation Status

Reservations progress from PENDING through CONFIRMED, CHECKED_IN, CHECKED_OUT. Cancellation is possible from PENDING or CONFIRMED. CHECKED_OUT triggers housekeeping task creation.

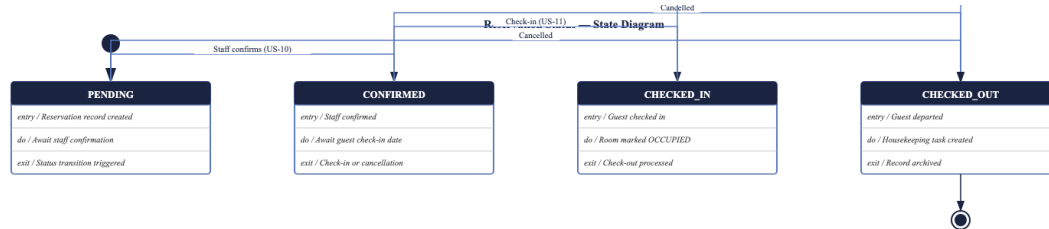


Figure 4.85 State Diagram: 4.5.2 Reservation Status

4.5.3 Housekeeping Task

Tasks are auto-created on guest check-out (PENDING). Staff can mark urgent. Housekeepers transition to DONE, triggering front-desk and guest notifications.

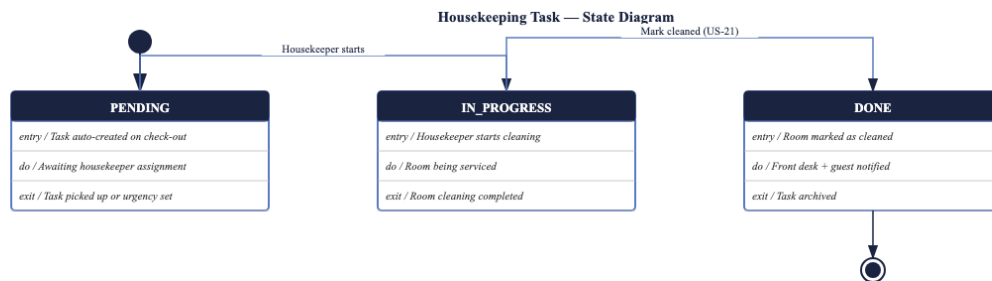


Figure 4.86 State Diagram: 4.5.3 Housekeeping Task

4.5.4 Parking Spot

Spots are AVAILABLE until selected (LOCKED for 5 minutes). The lock either expires (cron releases) or the guest confirms (TAKEN). Admin can release TAKEN spots.

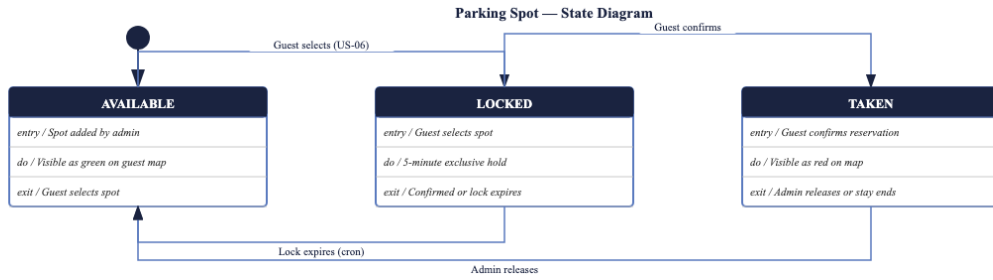


Figure 4.87 State Diagram: 4.5.4 Parking Spot

4.5.5 Tenant Account

New hotel accounts start **PENDING** (self-signup) or **ACTIVE** (Super Admin creates directly). Super Admin can approve, suspend, reactivate, or permanently delete.

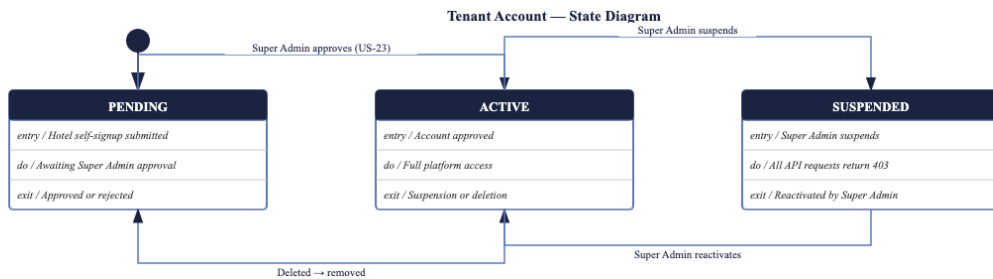


Figure 4.88 State Diagram: 4.5.5 Tenant Account

5. System Architecture Diagrams

5.1 Entity-Relationship Diagram (ERD)

The ERD models all database entities in the Reservio PostgreSQL schema (server/prisma/schema.prisma). Every entity except Tenant carries a `tenantId` foreign key, enforcing row-level multi-tenancy. The Prisma client extension auto-injects `WHERE tenantId=?` on all queries.

Key Relationships: Tenant is the root entity; all others cascade-delete on tenant removal. User owns Reservations and Orders. Room has HousekeepingTasks. ParkingSpot and Sunbed have 5-minute DB-level locks via lockedUntil field.

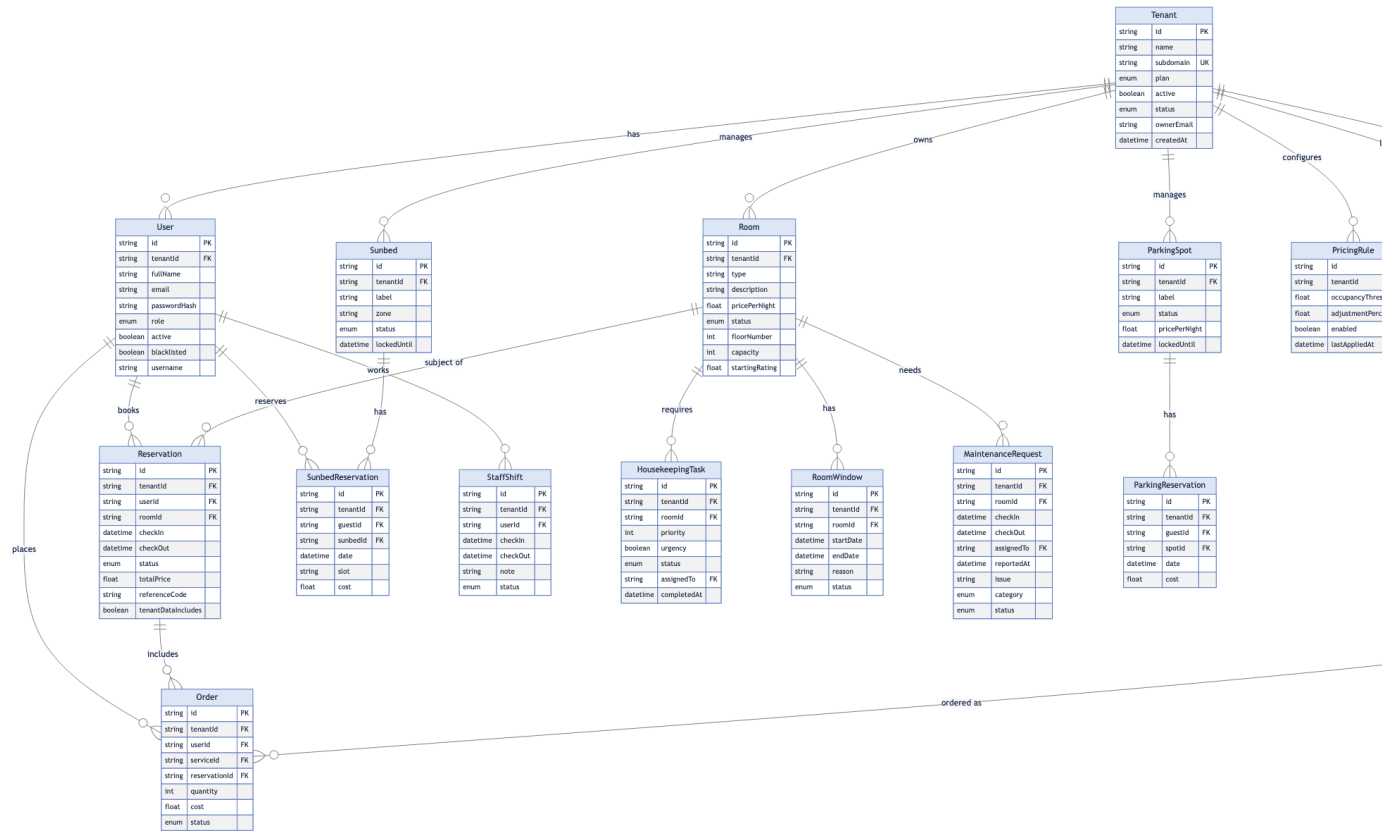


Figure 5.1 Entity-Relationship Diagram: Full Reservio Schema

5.2 Component Architecture Diagram

The component diagram illustrates system decomposition. The React SPA serves five portals from one codebase gated by JWT role. Every API request passes through tenantResolver → authMiddleware → featureGate → roleGuard before reaching handlers.

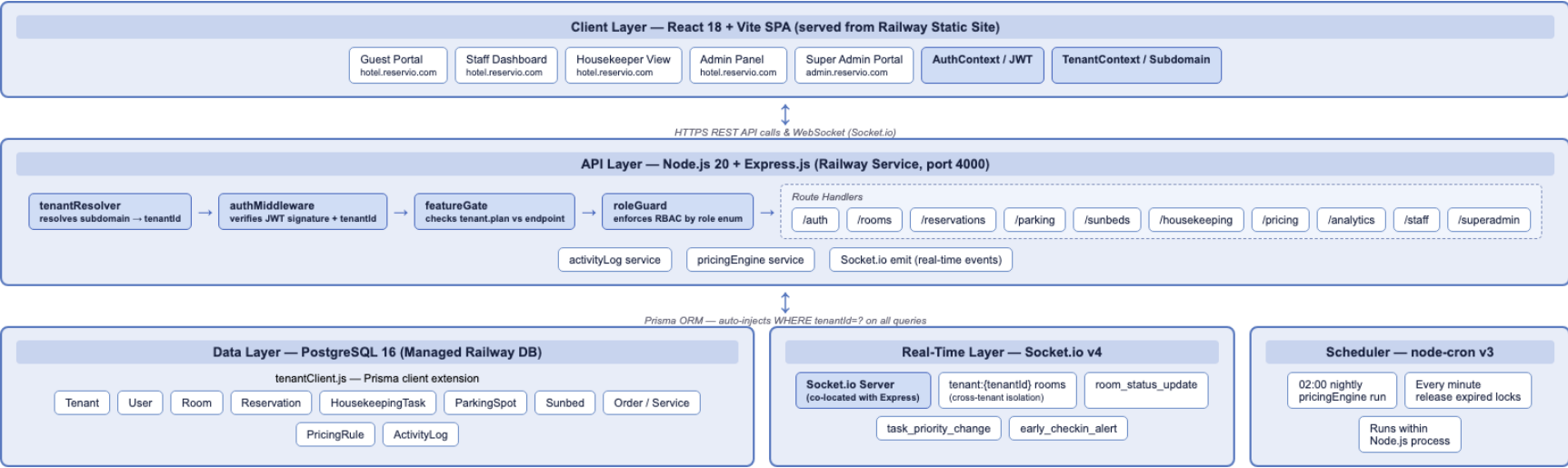


Figure 5.2 Component Architecture Diagram

5.3 Deployment Diagram

Reservio is deployed entirely on Railway Cloud PaaS. The Node.js service hosts both Express API and Socket.io WebSocket server in one process. React/Vite SPA is served as a static site. Both connect to a shared managed PostgreSQL 16 instance.

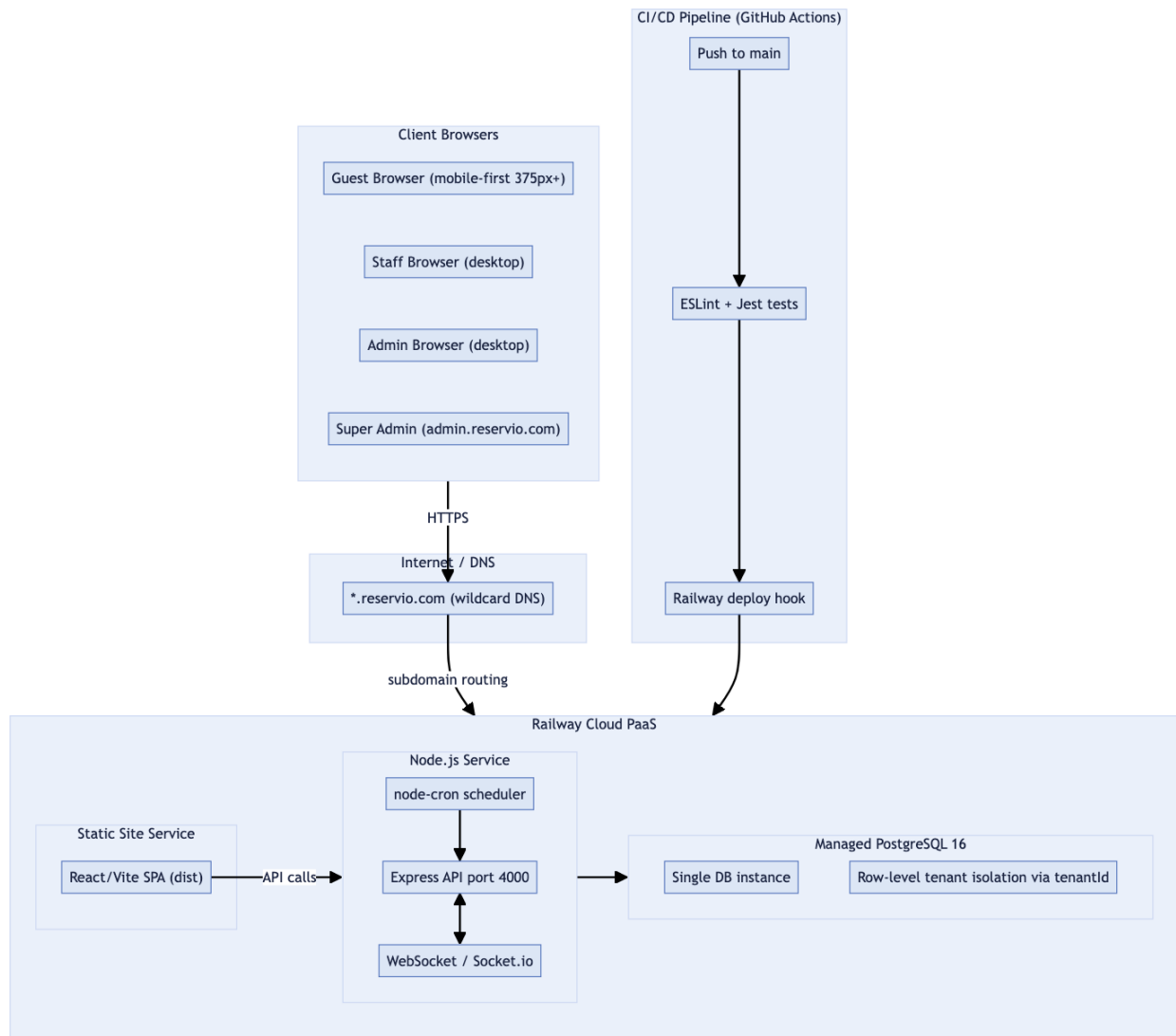


Figure 5.3 Deployment Diagram: Railway Cloud Infrastructure

5.4 Additional Architecture Diagrams

5.4.1 System Context Diagram

The system context shows how all five actor types interact with the Reservio platform and how the internal components relate to each other.

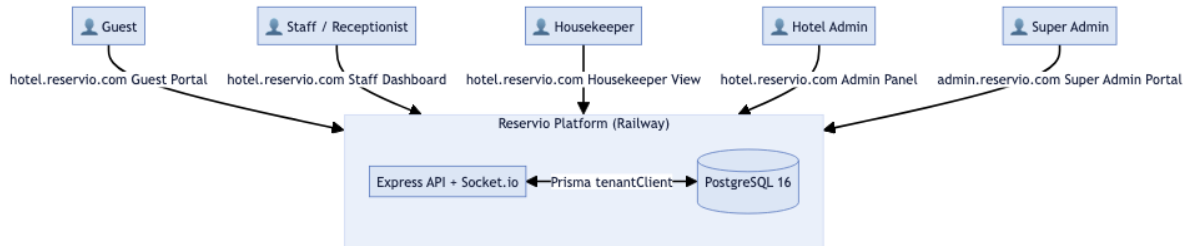


Figure 5.4 System Context Diagram

5.4.2 Request Middleware Pipeline

Every HTTP request to the Reservio API passes through four middleware layers before reaching any business logic. This diagram shows the full pipeline from request to database.



Figure 5.5 Middleware Pipeline: *tenantResolver* → *auth* → *featureGate* → *roleGuard*

5.4.3 Multi-Tenant Data Isolation

All tenants share a single PostgreSQL instance. The Prisma client extension automatically appends `WHERE tenantId=X` to every query. This diagram illustrates how the shared database serves multiple isolated hotels.

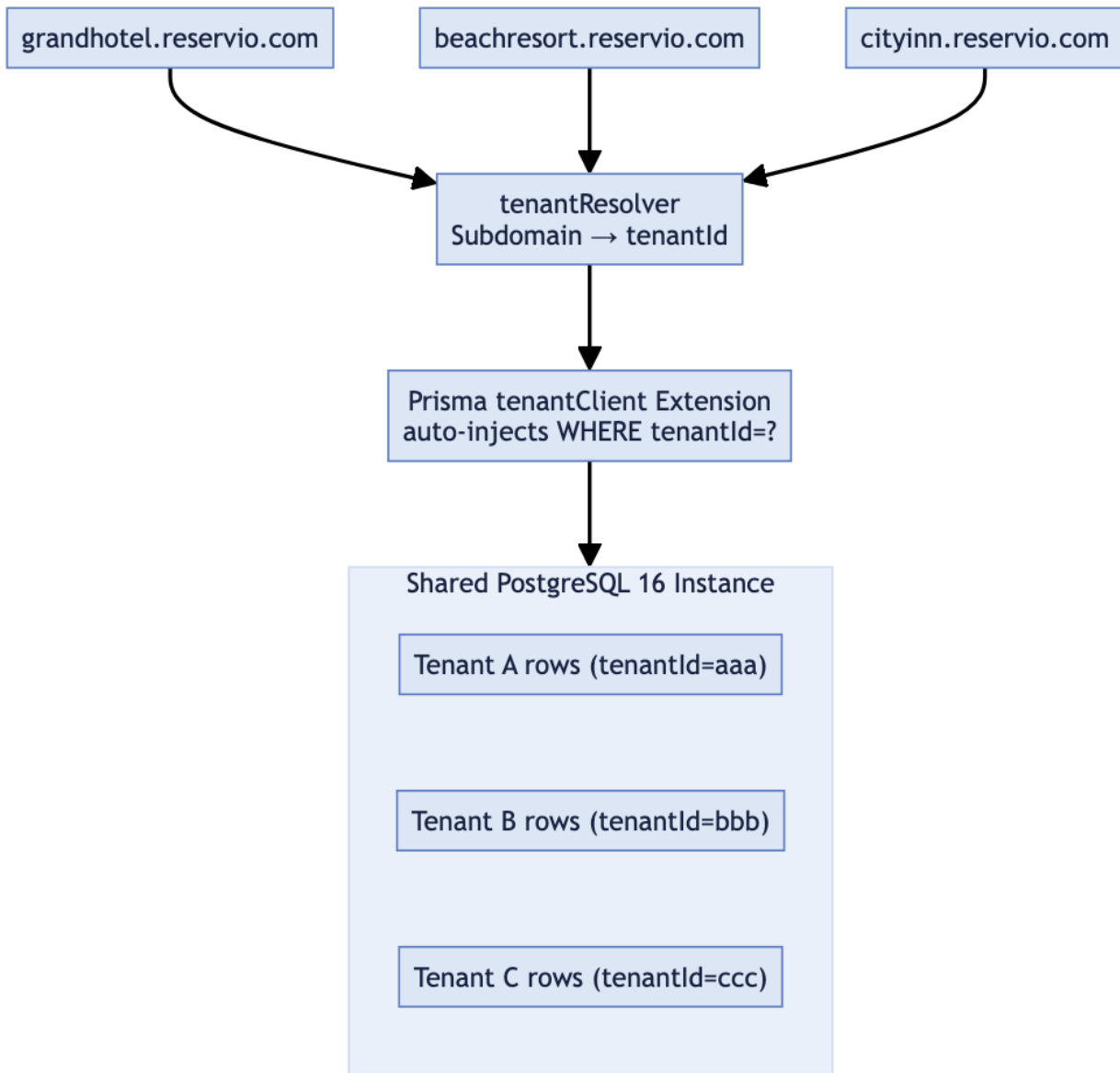


Figure 5.6 Multi-Tenant Data Isolation Architecture

5.4.4 Tenant Subdomain Routing

When a request arrives at grandhotel.reservio.com, the tenantResolver middleware extracts the subdomain, looks it up in the Tenant table, and attaches the tenant object to the request context.

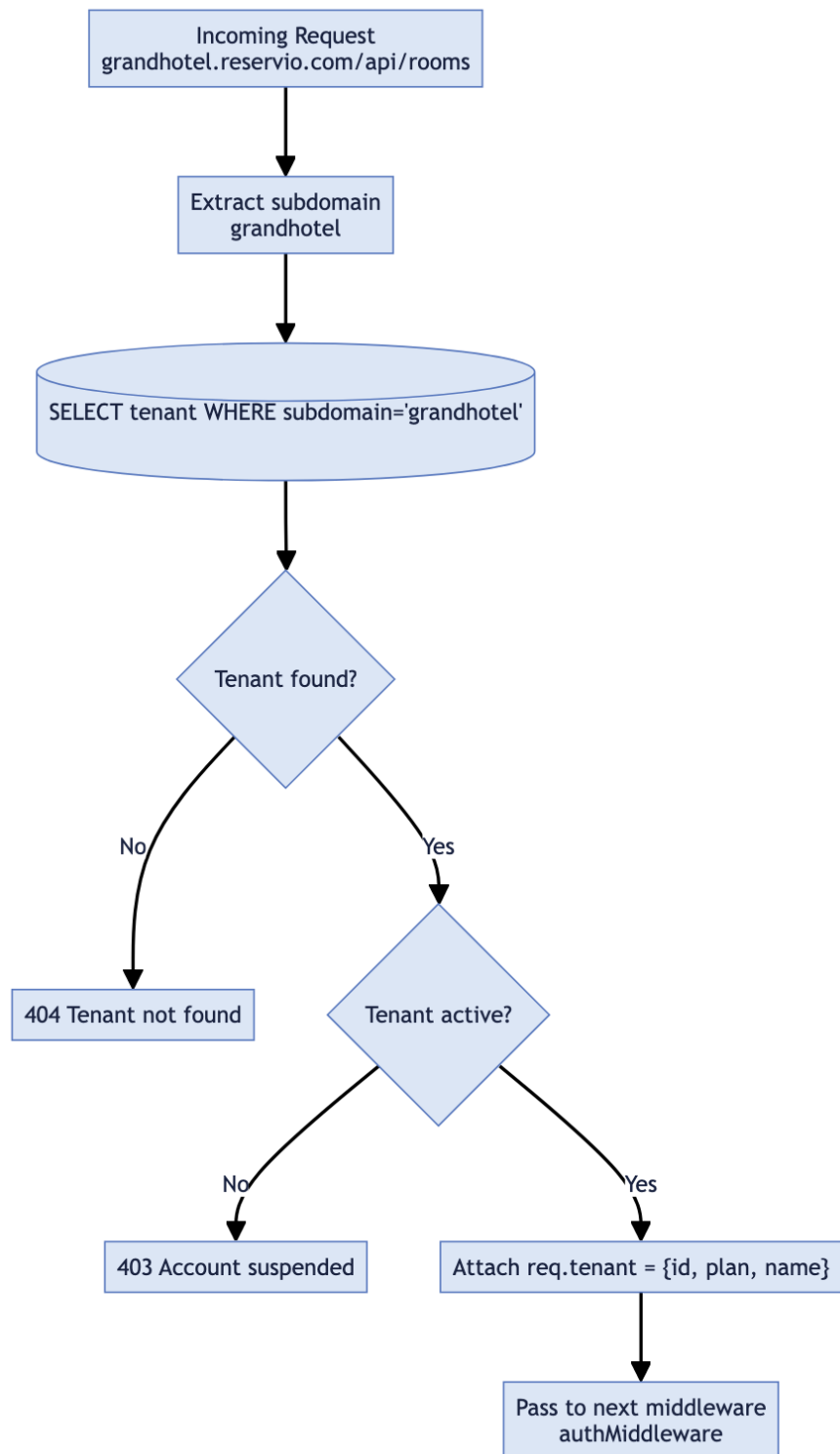


Figure 5.7 Tenant Subdomain Routing Flow

5.4.5 JWT Authentication Flow

On login, the API issues a signed JWT containing `userId`, `tenantId`, and `role`. Every subsequent protected request includes the JWT as a Bearer token, which is verified by `authMiddleware`.

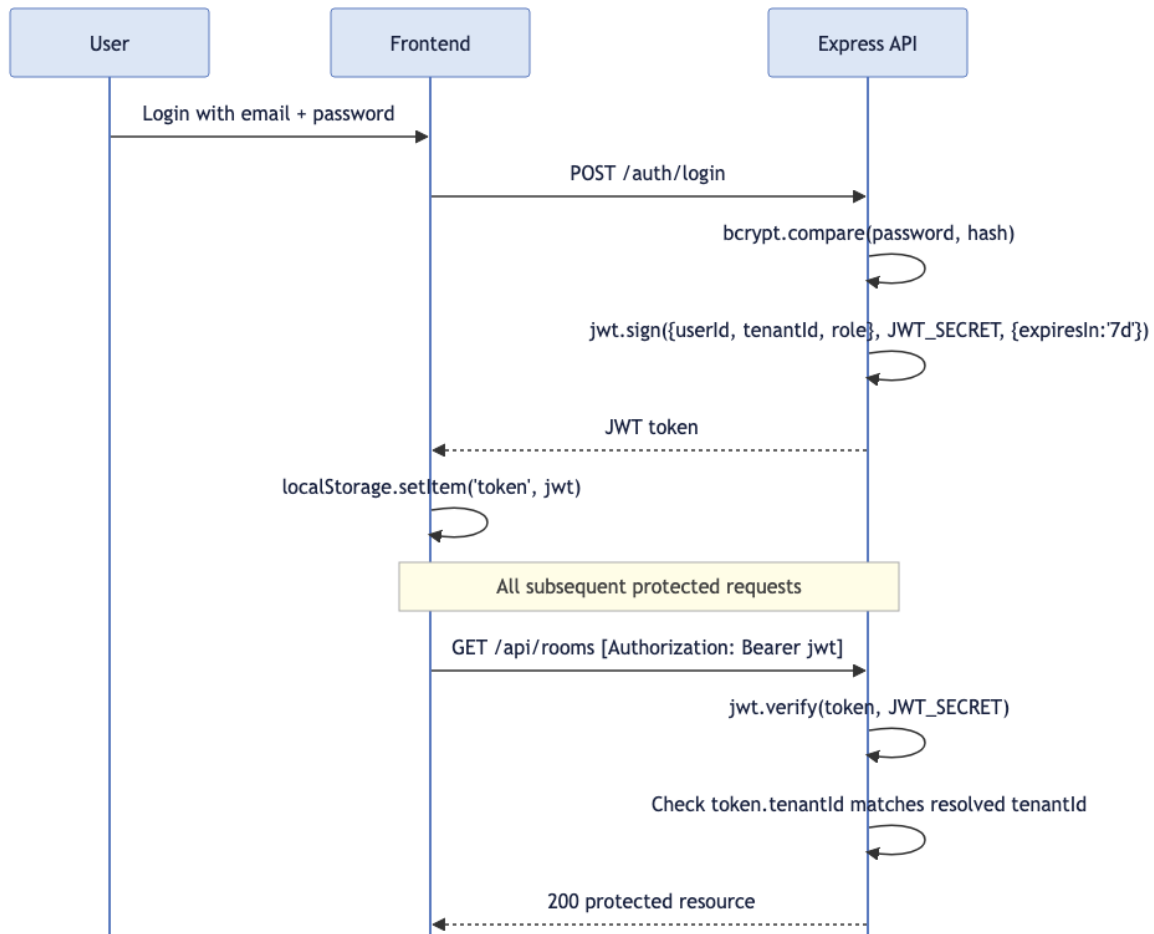


Figure 5.8 JWT Authentication Flow

5.4.6 WebSocket Event Flow

Socket.io rooms are named `tenant:{tenantId}` to prevent cross-tenant event leakage. This diagram shows how a housekeeping completion event propagates from the API to all relevant connected clients.

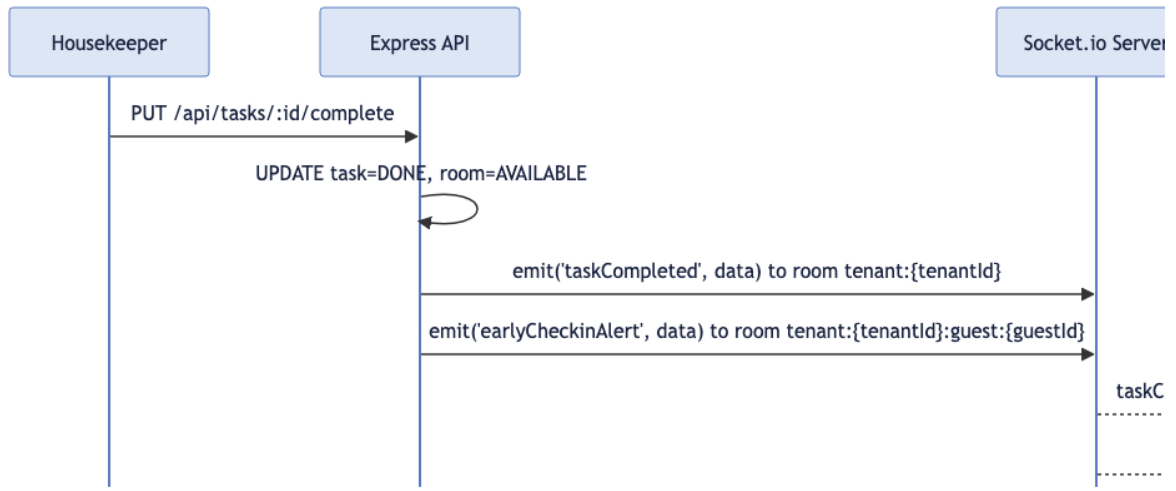


Figure 5.9 WebSocket Event Flow: Housekeeping Notification

5.4.7 5-Minute Concurrency Lock Mechanism

The lock mechanism prevents double-booking of parking spots and sunbeds. A DB-level `lockedUntil` field combined with a node-cron job ensures no spot can be held indefinitely.

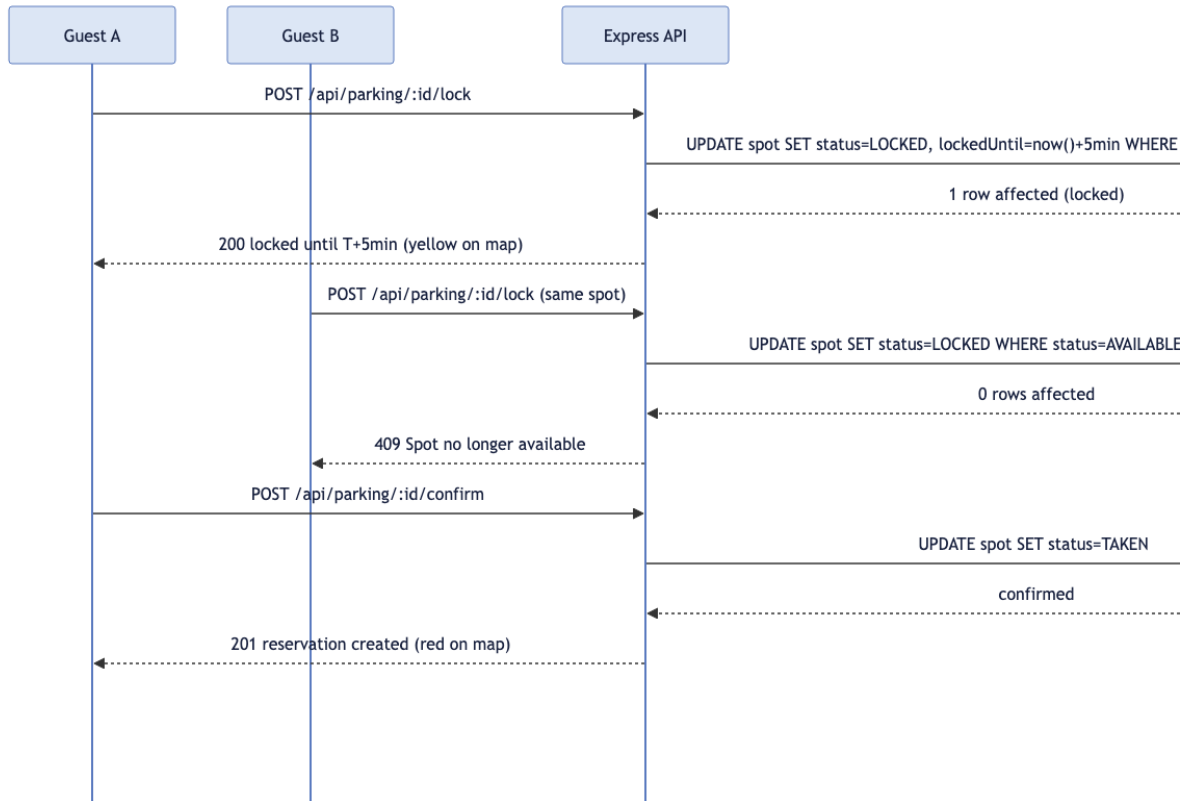


Figure 5.10 5-Minute Lock Sequence: Parking/Sunbed Concurrency Control

5.4.8 Dynamic Pricing Engine

The pricing engine is a node-cron job that runs nightly at 2:00 AM. It iterates all active Premium and Custom tenants, evaluates occupancy against configured rules, and adjusts room prices accordingly.

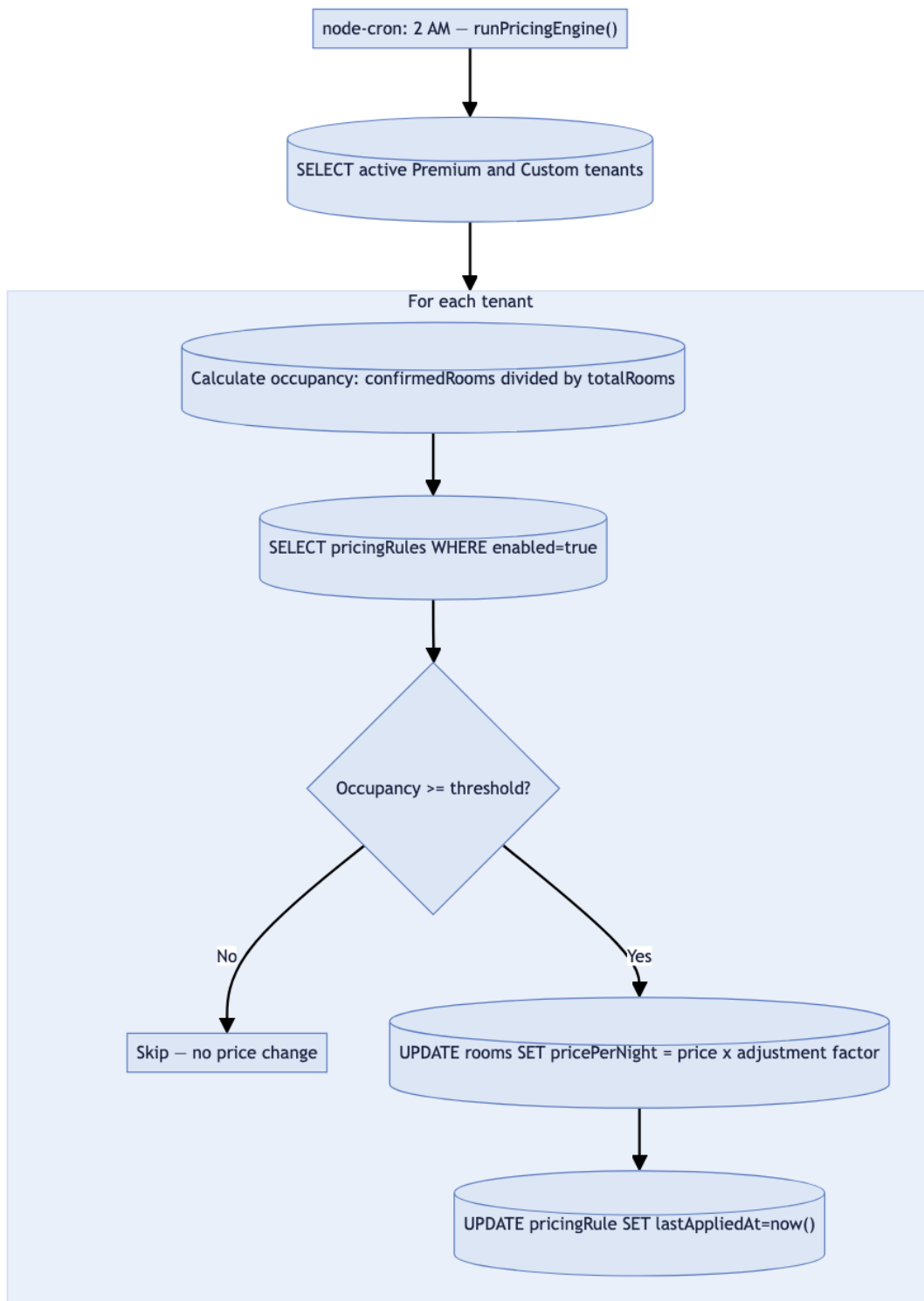


Figure 5.11 Dynamic Pricing Engine Flow: Nightly Cron Evaluation

5.4.10 Role Hierarchy

Reservio has seven distinct user roles organised into three levels: platform level (Super Admin), hotel level (Admin, Manager, Receptionist, Housekeeper, Technician), and self-service level (Guest).

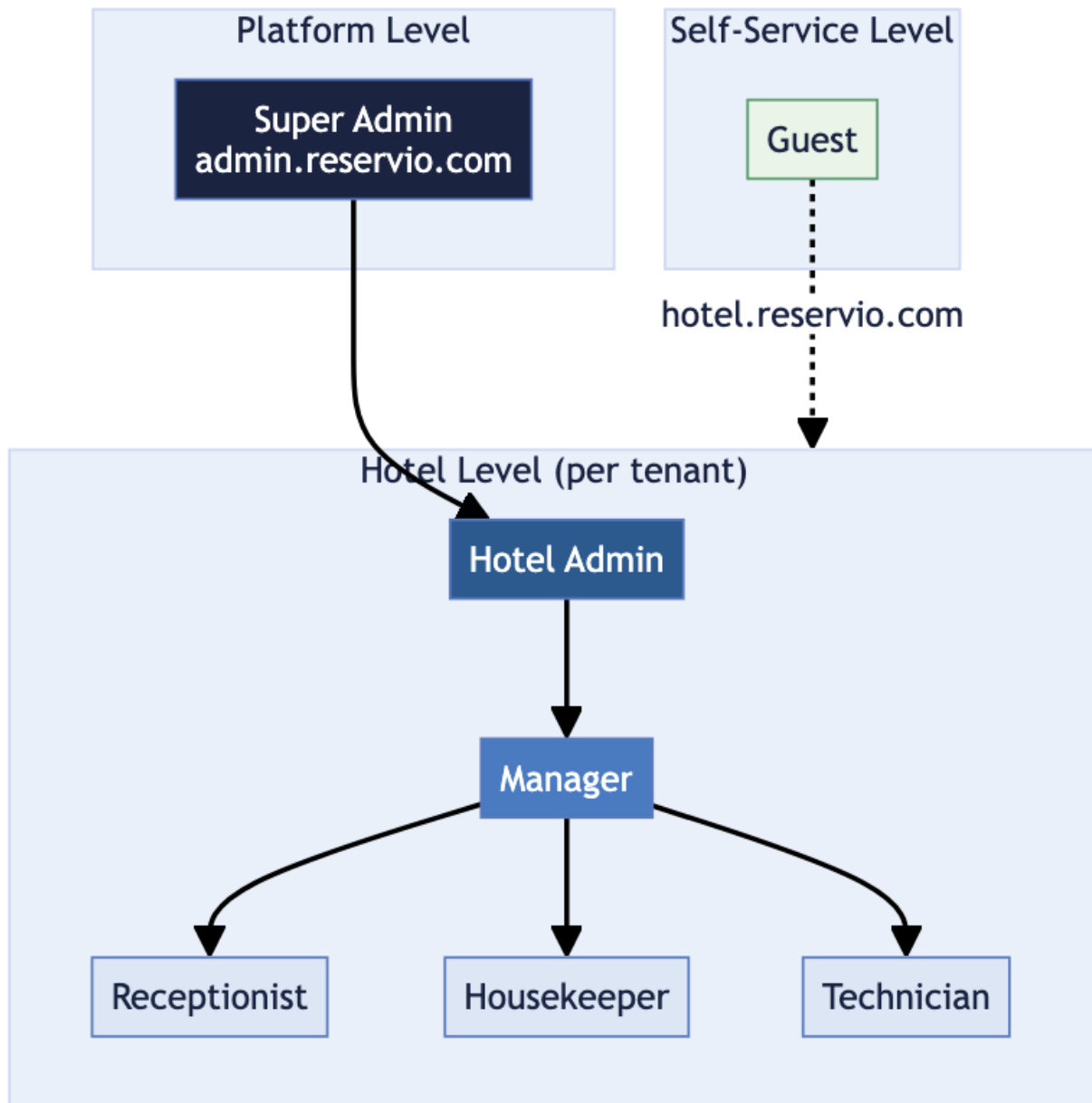


Figure 5.13 Role Hierarchy and Portal Access

5.4.11 Security Model

The security model operates in layers: transport (TLS), authentication (JWT + bcrypt), authorisation (RBAC + feature gates), and data isolation (tenant-scoped Prisma queries).

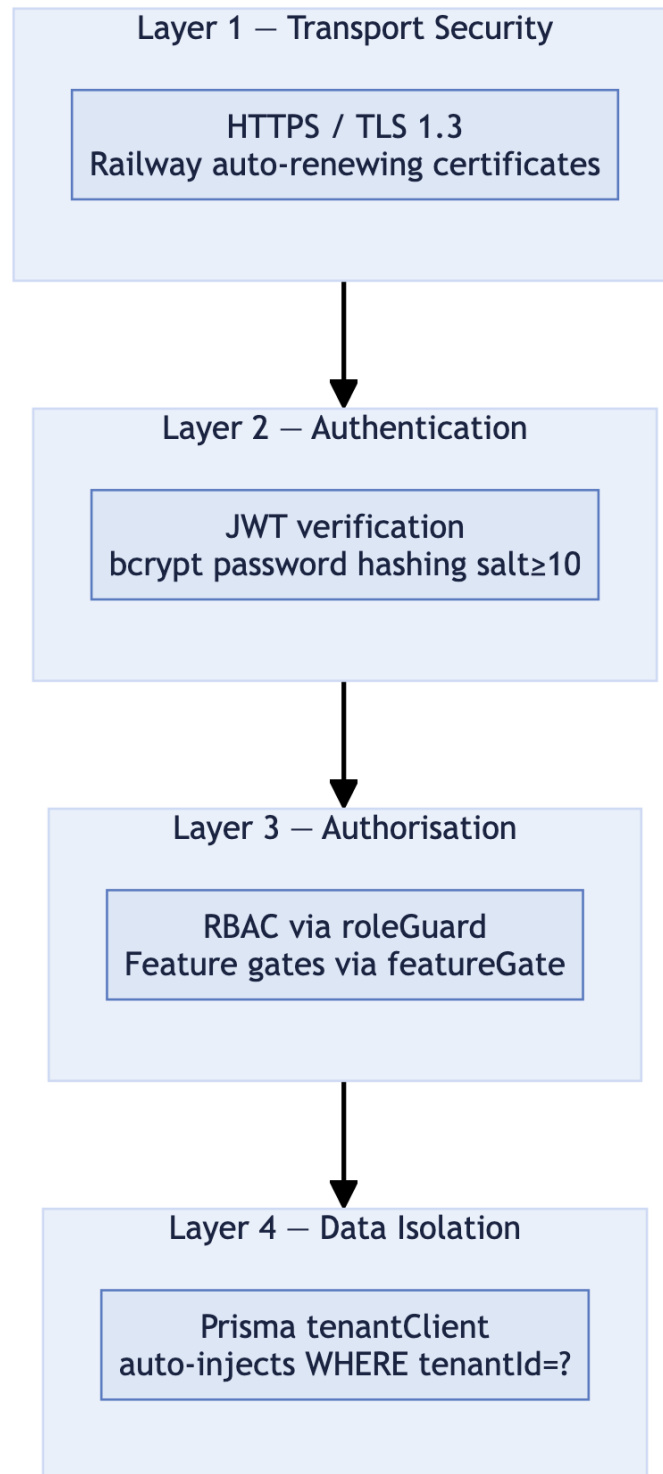


Figure 5.14 Security Layer Model

5.4.12 Subscription Tier Feature Map

This diagram maps the three subscription tiers (Basic, Premium, Custom) to their respective feature sets, illustrating which capabilities are unlocked at each level.



Figure 5.15 Subscription Tier Feature Map

5.5 Non-Functional Requirements

ID	Category	Requirement Criterion Implementation
NF	Performance	Page load time Initial load < 2s Vite code splitting,
R-0		lazy-loaded routes,
1		image optimisation
NF	Performance	API response time 95% of Prisma query
R-0		responses < optimisation, indexed
2		300ms tenantId columns, connection pooling
NF	Real-time	WebSocket delivery Notifications < Socket.io with
R-0		500ms after tenant-scoped rooms
3		trigger
NF	Security	Authentication All protected Express middleware
R-0		routes require validates JWT;
4		valid JWT mismatched tenantId rejected
NF	Security	Password storage Passwords never bcrypt with salt rounds
R-0		stored in plain ≥ 10
5		text
NF	Security	Role-based access Each role Role field in JWT
R-0		accesses only its checked by roleGuard
6		permitted middleware endpoints
NF	Data	Tenant separation No query returns Prisma client extension
R-0	Isolation	data from auto-injects tenantId on
7		another tenant all queries
NF	Data	Feature gating Premium featureGate middleware
R-0	Isolation	features checks tenant.plan

ID	Category	Requirement Criterion Implementation
8		inaccessible on before processing Basic plan
NF	Reliability	System uptime 99% uptime Railway auto-restart on
R-0		during demo crash; health check
9		period endpoint
NF	Reliability	Concurrency safety No 5-minute DB-level lock
R-1		double-booking with tenantId on
0		race conditions parking and sunbed records
NF	Scalability	Concurrent tenants System handles Shared infrastructure;
R-1		20+ active tenantId-indexed
1		tenants queries keep simultaneously performance consistent
NF	Scalability	Concurrent users 50 concurrent Stateless JWT,
R-1		users per tenant connection pooling
2		without degradation
NF	Usability	Mobile All guest-facing Tailwind responsive
R-1		responsiveness screens usable at utilities, mobile-first
3		375px+ design
NF	Usability	Onboarding time New hotel Guided setup flow;
R-1		operational in < tenant provisioned
4		10 minutes automatically on signup
NF	Maintainabili	Code quality No critical ESLint + Prettier
R-1	ty	linting errors in enforced via GitHub
5		main branch Actions CI

6. Class Diagrams & Data Dictionary

This chapter presents UML class diagrams for all Prisma entities in the Reservio schema, followed by a detailed data dictionary. Every entity (except Tenant itself) includes a tenantId field, enforcing row-level multi-tenancy through the Prisma client extension.

6.1 Class Diagrams

6.1.1 Tenant Entity

Root entity for multi-tenancy. All other entities reference Tenant.id via tenantId. The plan field determines feature access through featureGate middleware.

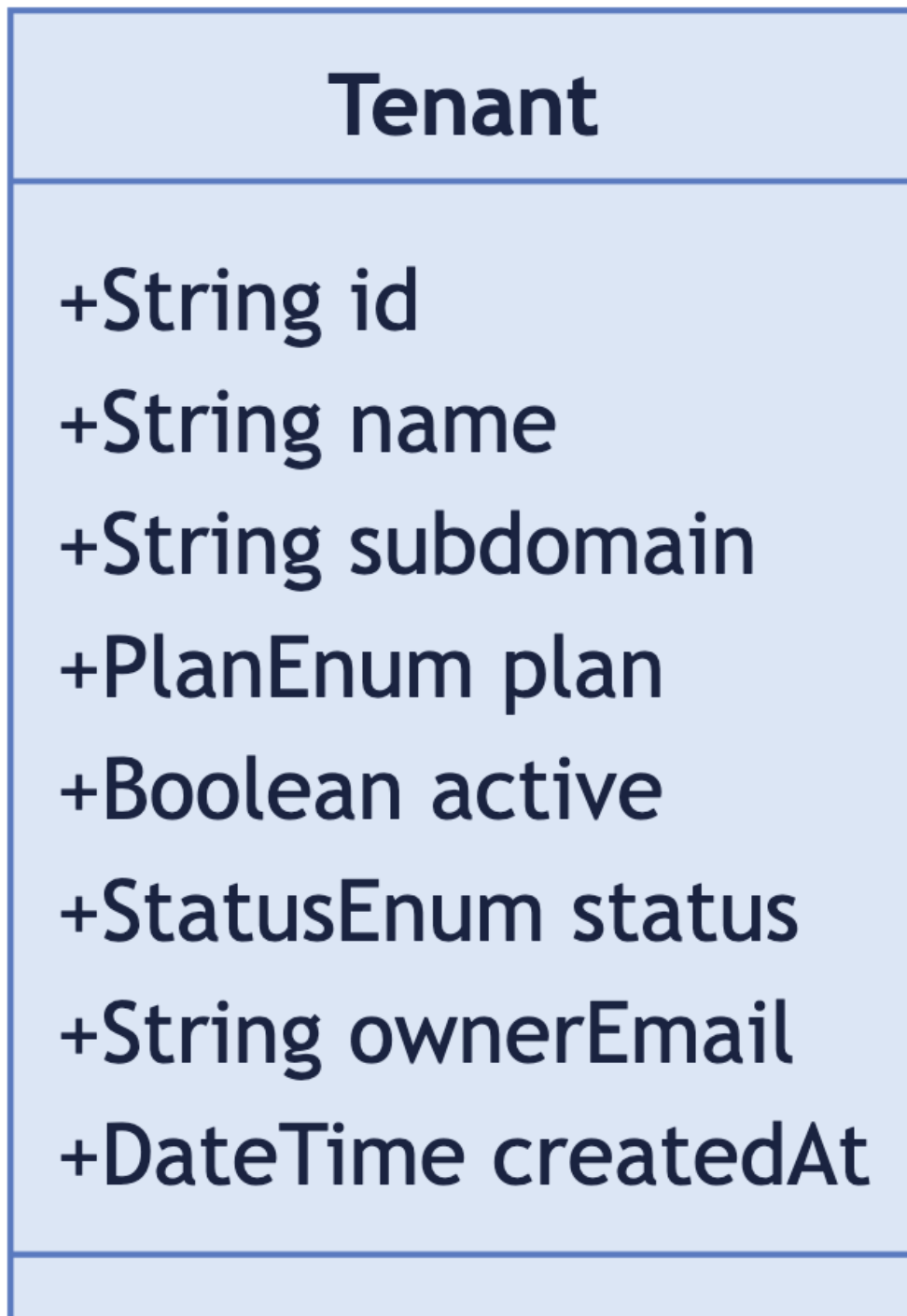


Figure 6.1 Class Diagram: Tenant Entity

6.1.2 User Entity

Represents all human actors. The role enum determines which portal and endpoints a user can access. passwordHash is never returned in API responses.

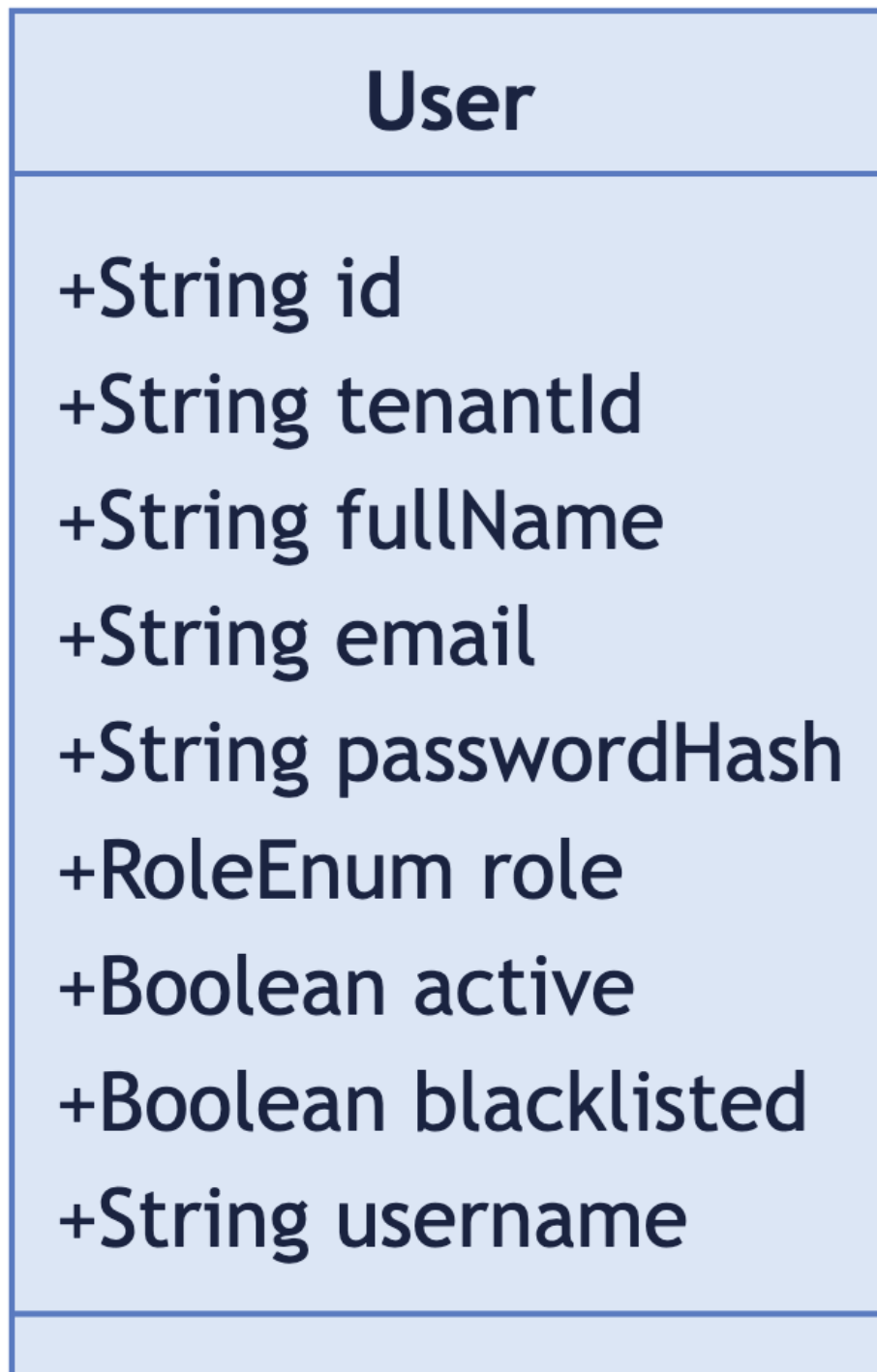


Figure 6.2 Class Diagram: User Entity

6.1.3 Room Entity

Represents a bookable hotel unit. pricePerNight may be adjusted nightly by the dynamic pricing engine. Photos are stored as an array of CDN URLs.

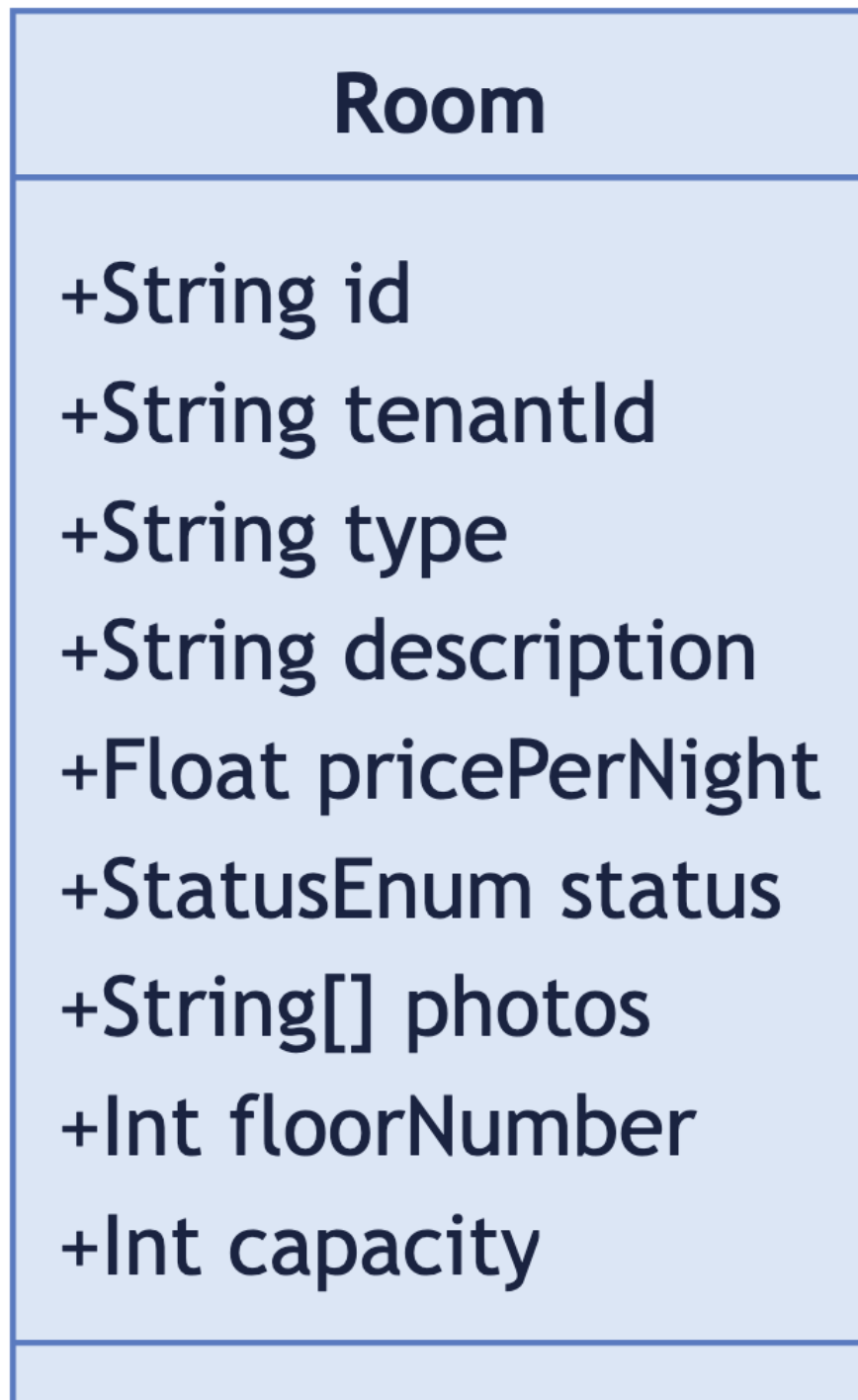


Figure 6.3 Class Diagram: Room Entity

6.1.4 Reservation Entity

Core booking record linking a Guest User to a Room for a date range. The status enum transitions are strictly controlled by the API middleware.

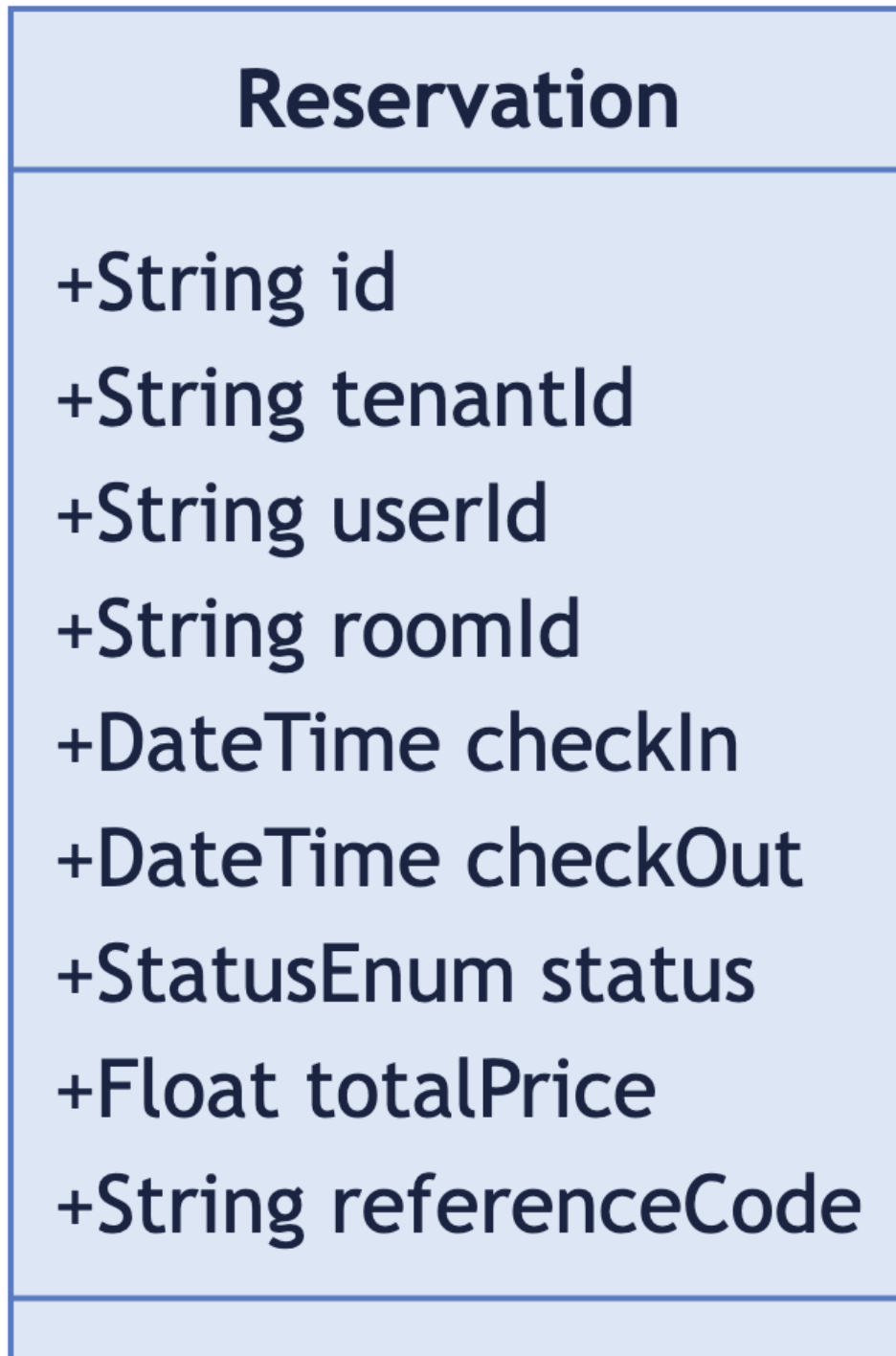


Figure 6.4 Class Diagram: Reservation Entity

6.1.5 ParkingSpot Entity

Physical parking space. The lockedUntil field implements the 5-minute concurrency lock. A node-cron job releases expired locks every minute.

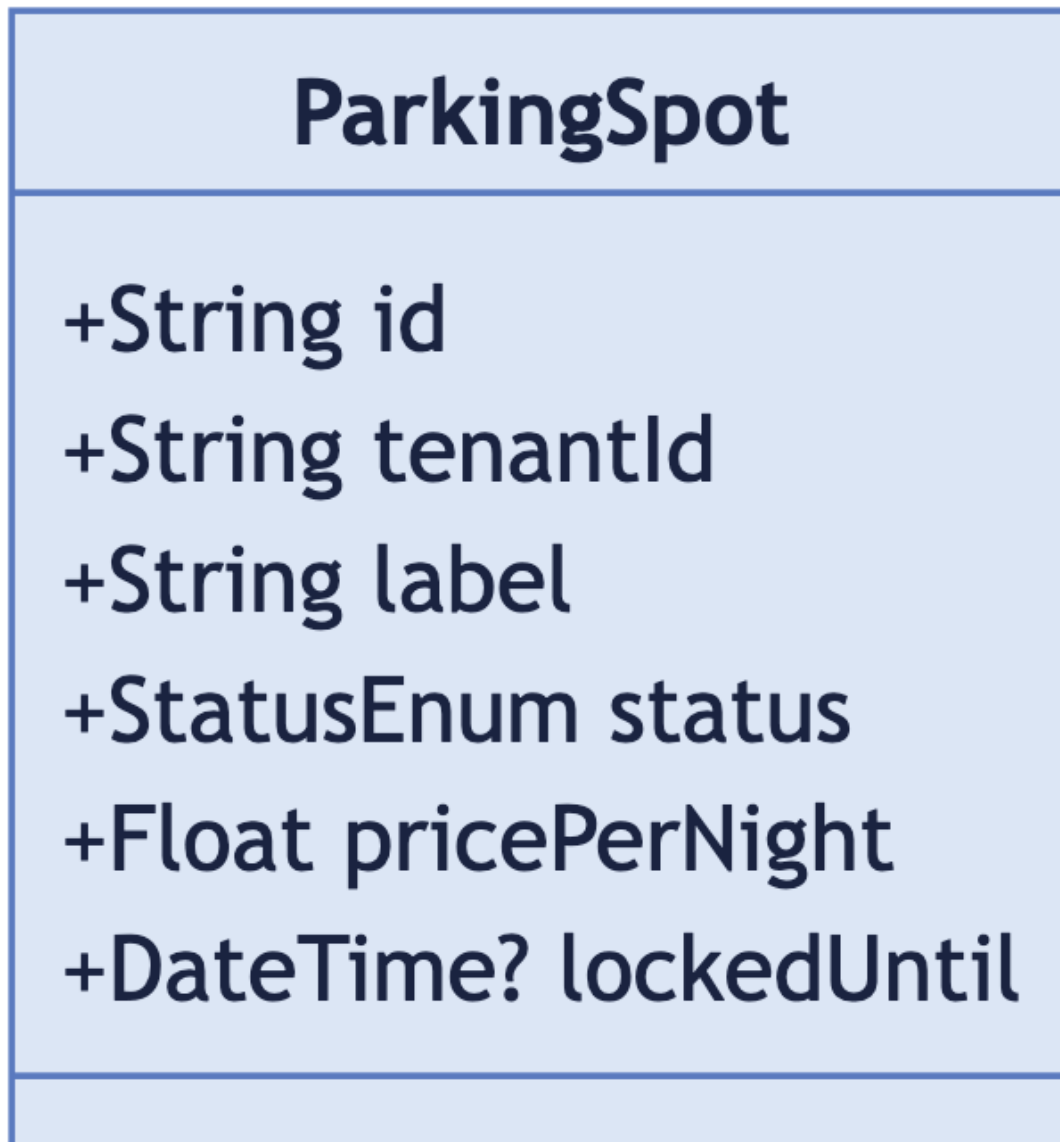


Figure 6.5 Class Diagram: ParkingSpot Entity

6.1.6 Sunbed Entity

Poolside sunbed with time-slot-based booking. Uses the same 5-minute lock mechanic as ParkingSpot. Zone field groups sunbeds by pool area.

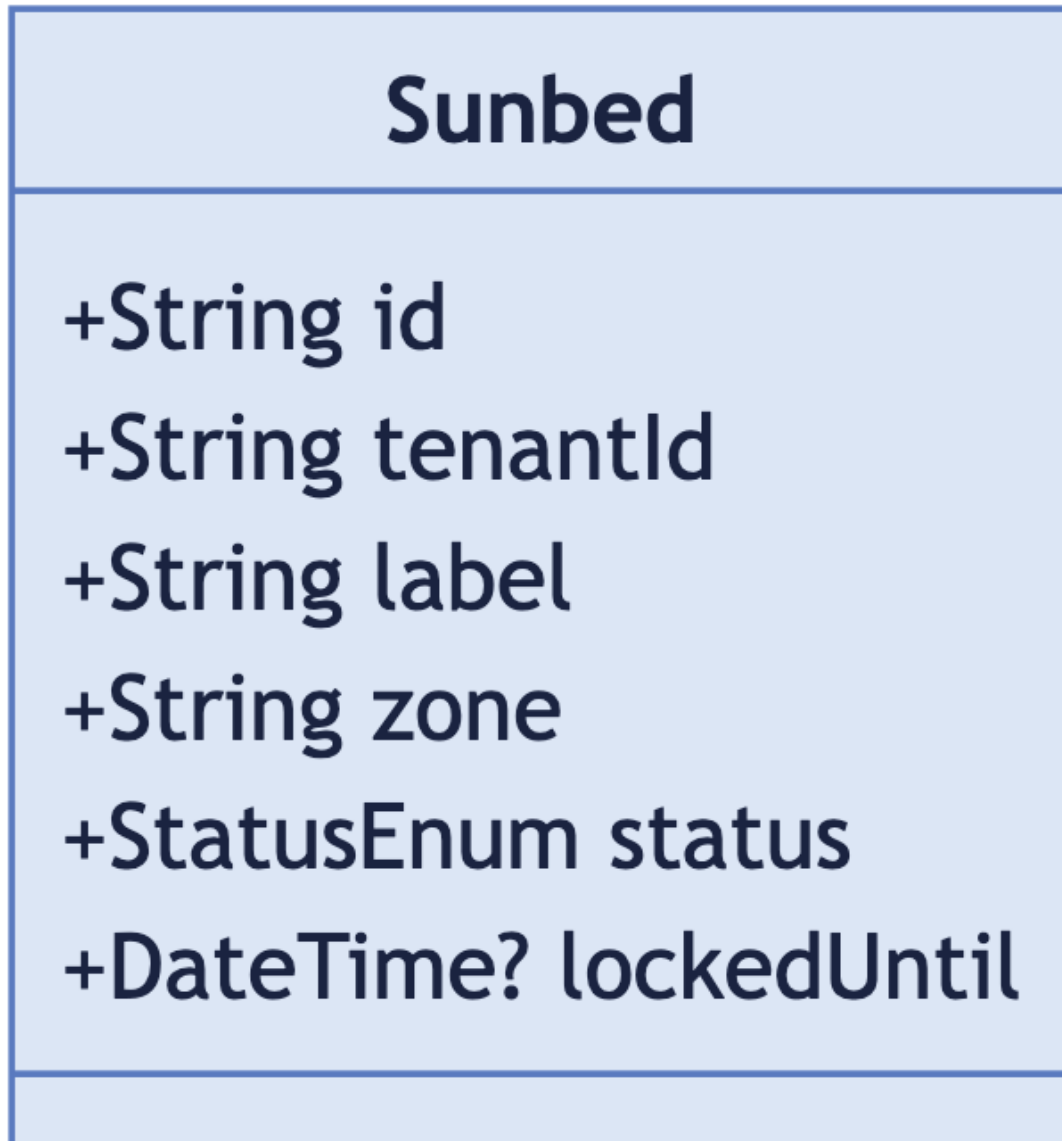


Figure 6.6 Class Diagram: Sunbed Entity

6.1.7 HousekeepingTask Entity

Auto-created on guest check-out. The urgency boolean allows staff override (US-14). Priority is calculated from check-out time proximity.

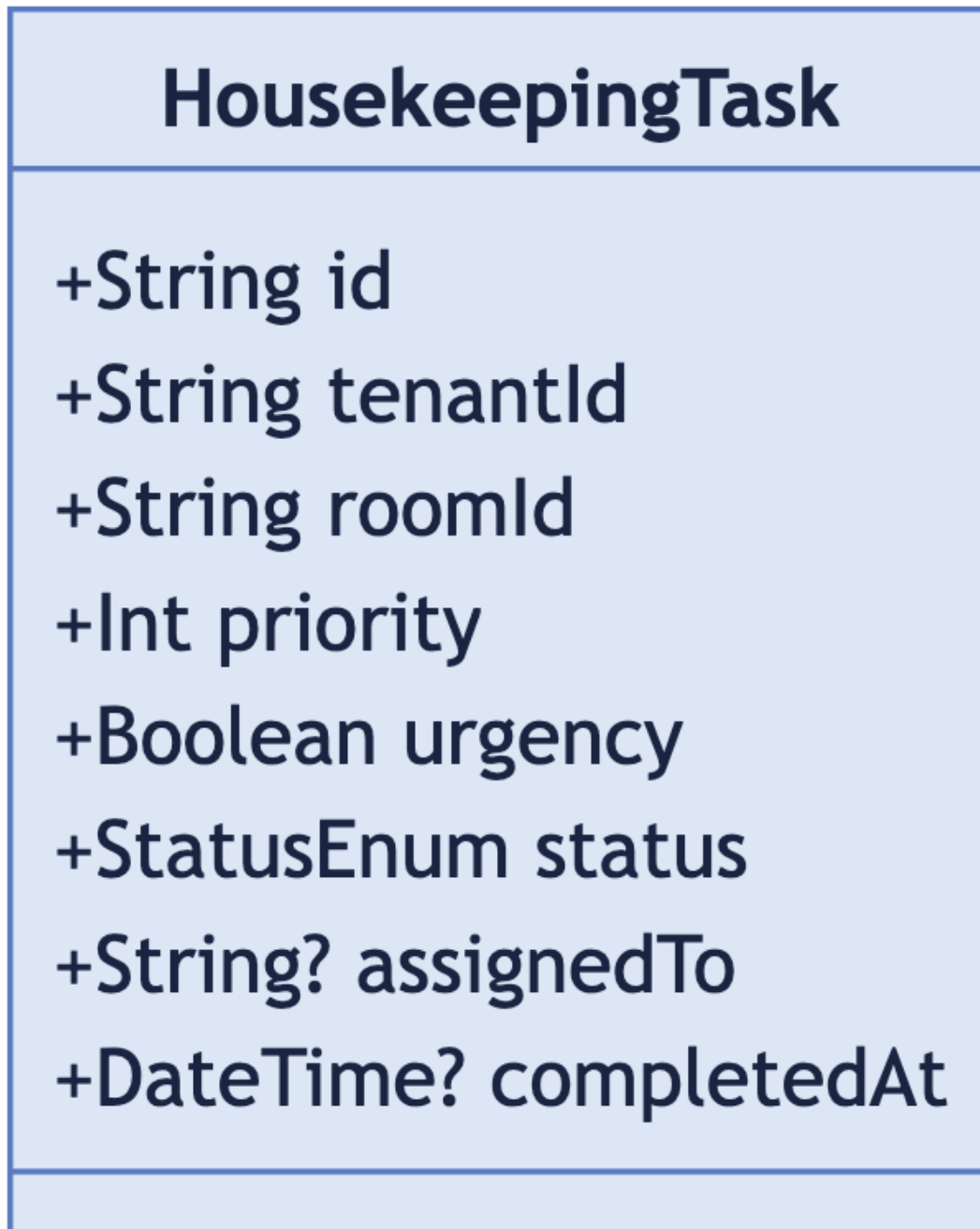


Figure 6.7 Class Diagram: HousekeepingTask Entity

6.1.8 Service Entity

Marketplace item configured per tenant by Hotel Admin. Category enum supports Spa, Room Service, Transport, and Other. Can be toggled without deletion.

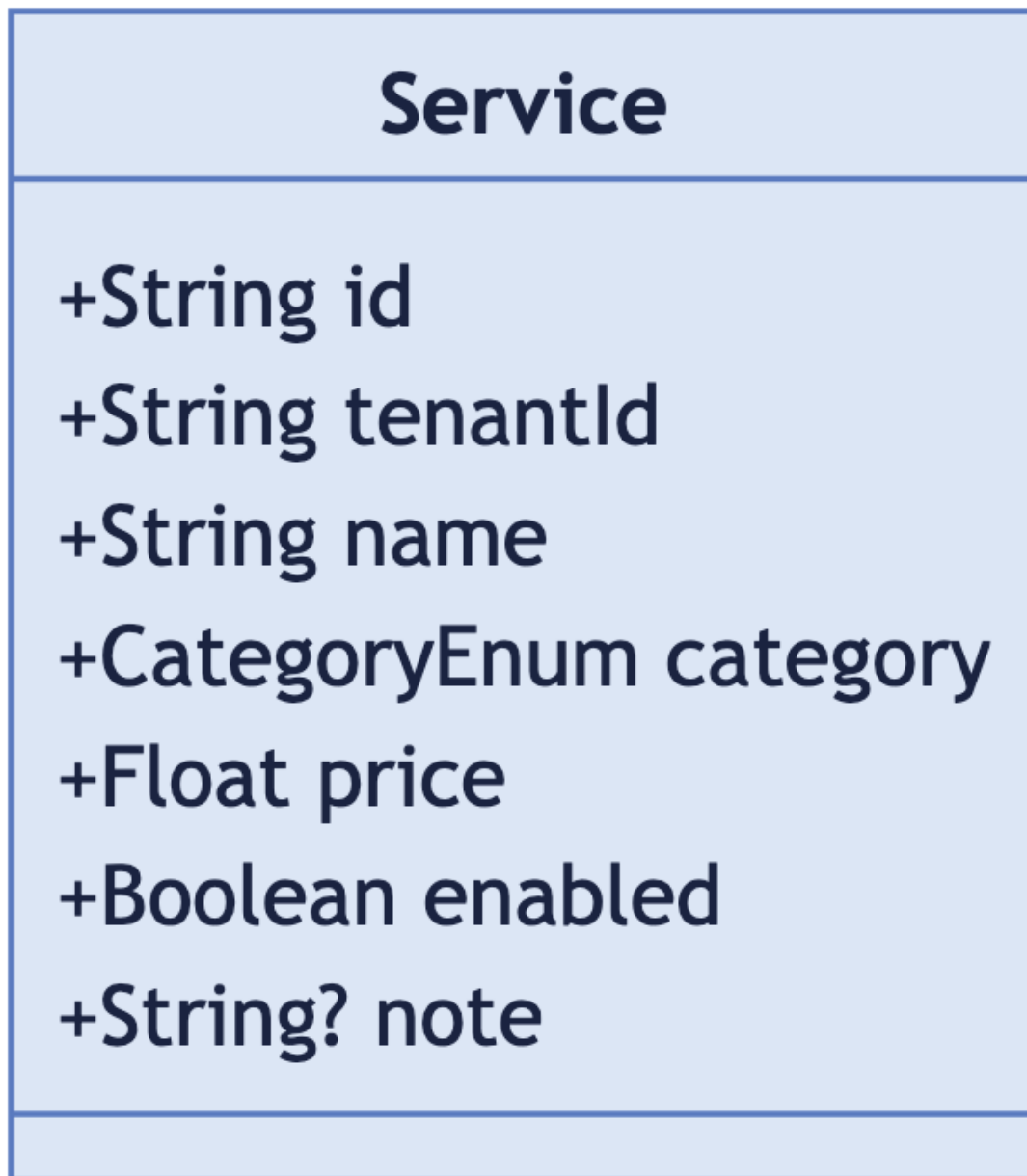


Figure 6.8 Class Diagram: Service Entity

6.1.9 Order Entity

Created when a guest orders a marketplace service. Linked to both a Service and a Reservation for room billing. Cost calculated as $\text{price} \times \text{quantity}$.

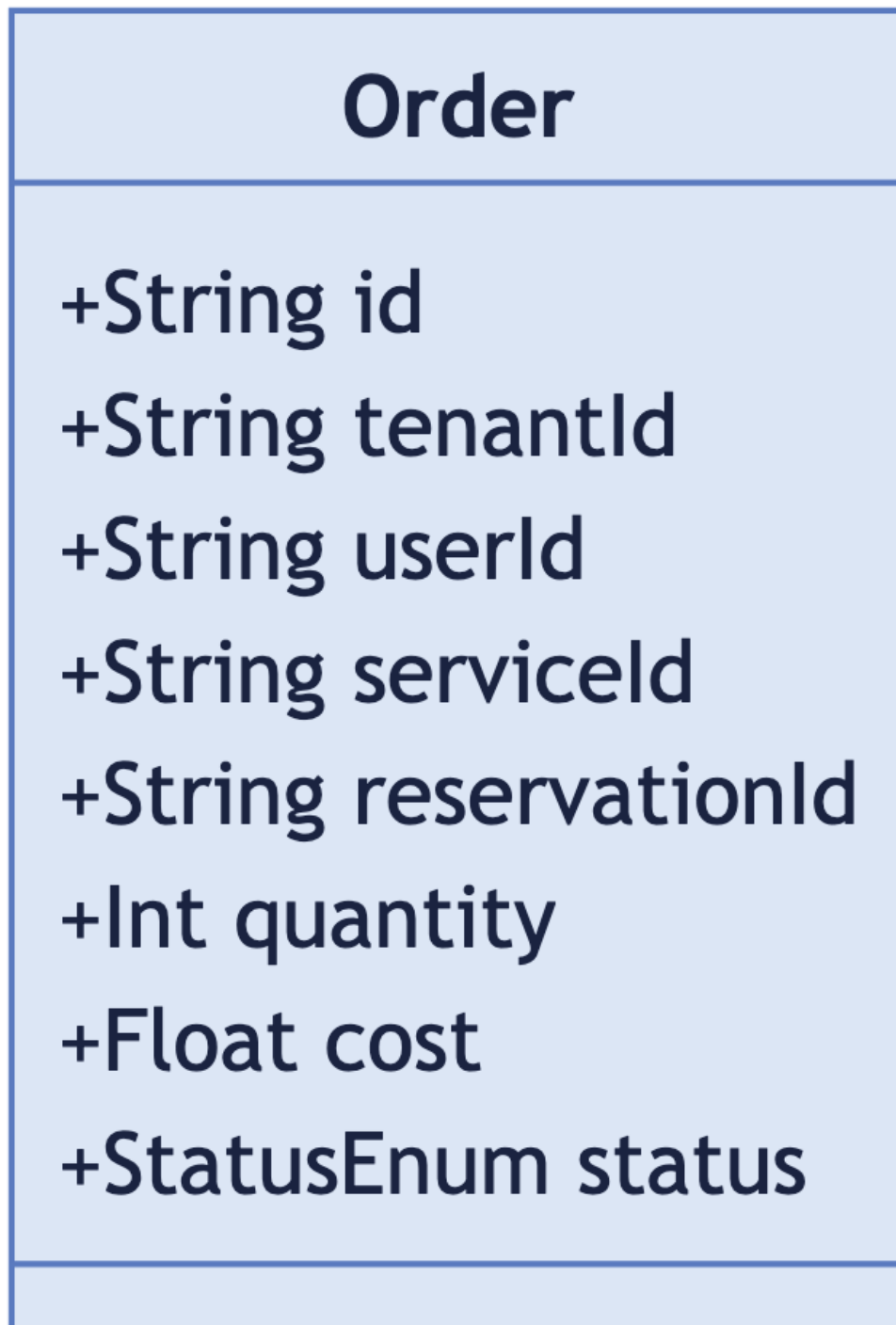


Figure 6.9 Class Diagram: Order Entity

6.1.10 PricingRule Entity

Defines dynamic pricing thresholds evaluated nightly by the node-cron job. adjustmentPercent can be positive (price increase) or negative (discount).

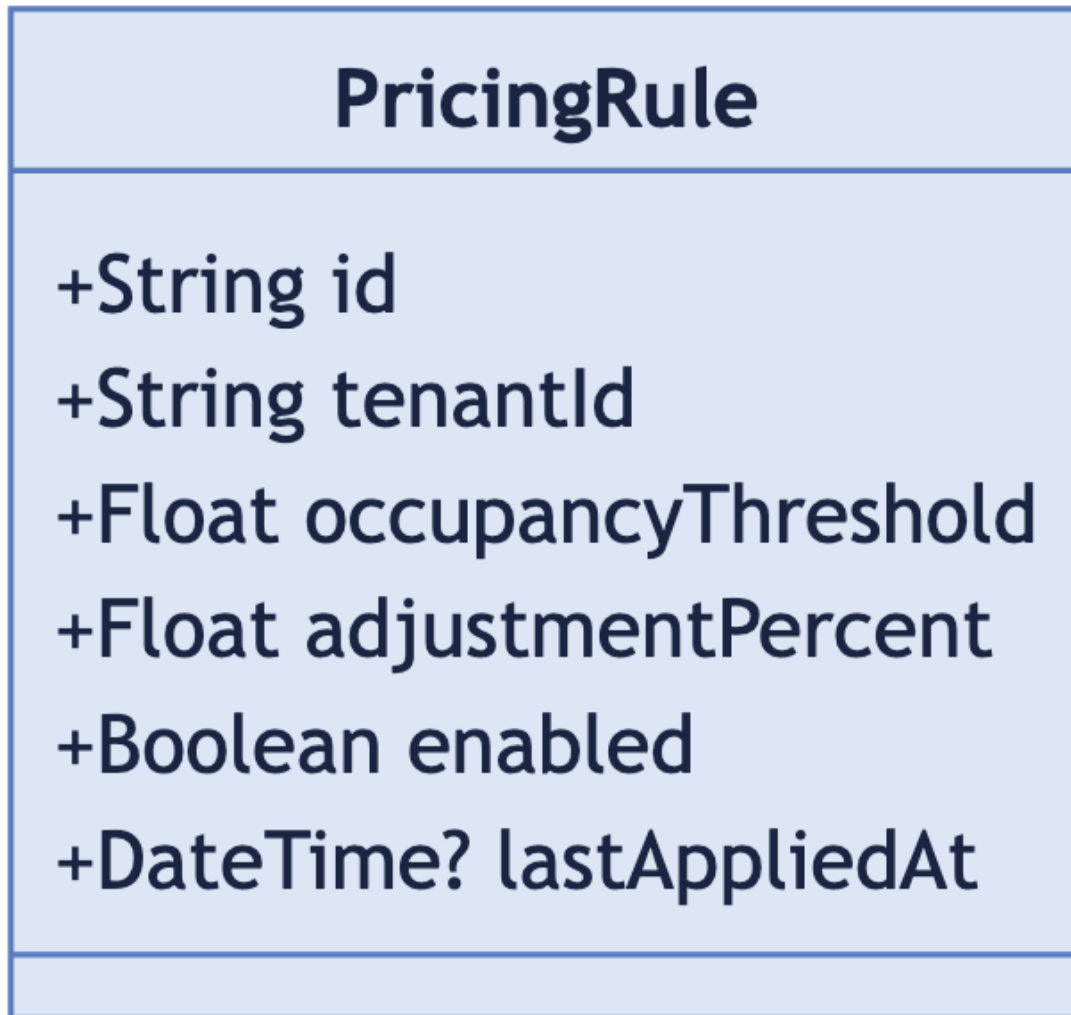


Figure 6.10 Class Diagram: PricingRule Entity

6.1.11 ActivityLog Entity

Audit trail for all significant system actions. Not exposed to guests or staff. The metadata JSON field stores before/after values for change events.

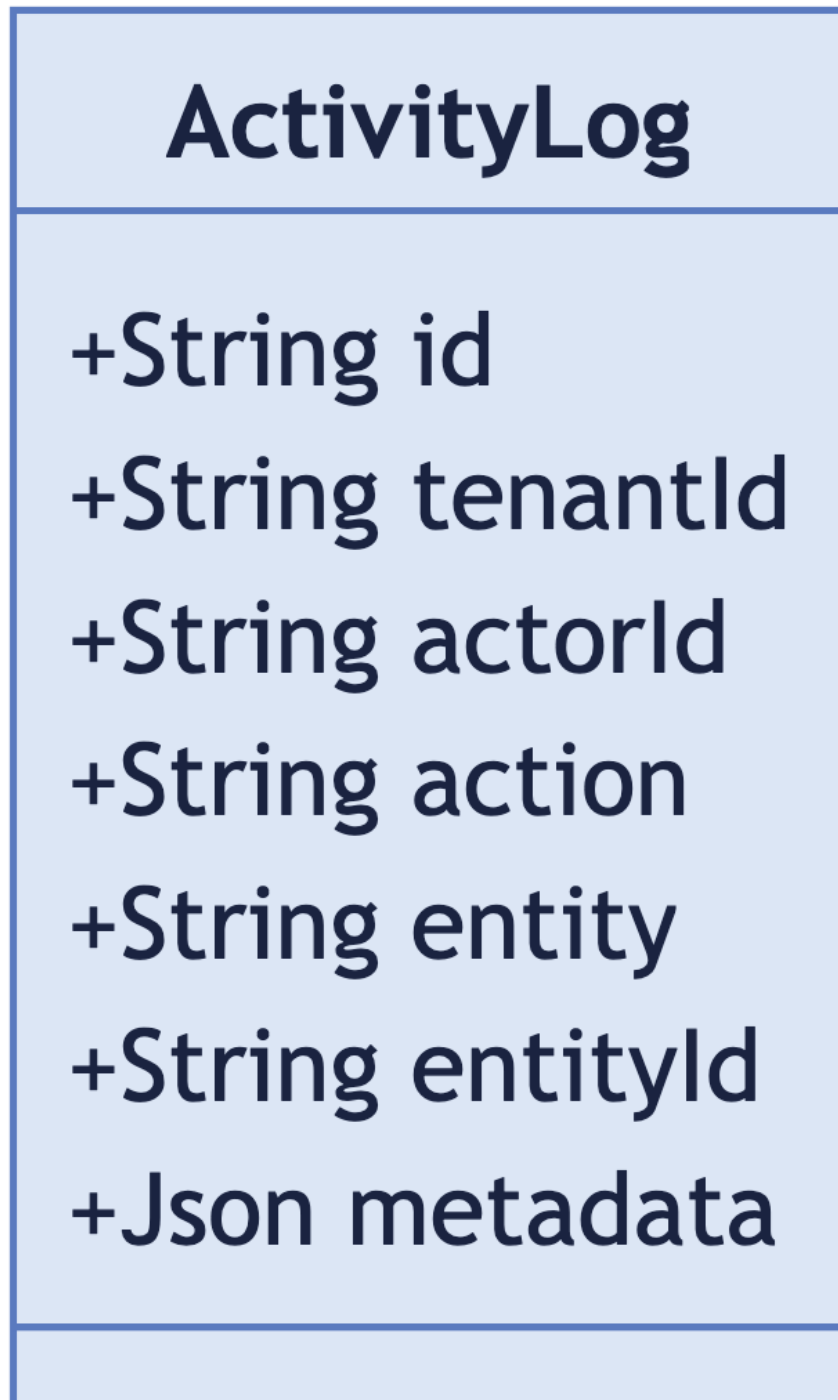


Figure 6.11 Class Diagram: ActivityLog Entity

6.1.12 Operational Class Methods & Controller Specifications

While the entity diagrams above define the structural properties and multi-tenant fields of the system schema, the operational behaviors of these objects are driven by backend application controllers. To prevent cross-tenant data exposure on shared infrastructure, all system services (excluding the cross-tenant Super Admin module) strictly mandate a passing tenantId parameter.

The following table acts as the definitive behavioral operation map for the Reservio system classes:

Class / Visibility	Function / Method	Return Description / User
Controller	Signature	Type Story Mapping
Name		
Auth Controller +(Public)	registerGuest(t JWT	
oller	enantId: String, data: O)	Registers a guest account scoped to the matching subdomain (US-01).

```
+(Public) loginUser(tenantId: String, Validates credentials via
bcrypt; returns token
credentials:
containing userId,
LoginDTO)
tenantId, and role
(US-02).
```

-(Private) generateUsername	String
e(fullName: String)	g Helper utility to auto-generate unique staff login IDs (US-16).
Room Controller +(Public)	List<
oller	Rooms(tenantId: String, RoomFilterDTO)
	Room> Lists available units, filtering out overlapping reservation windows (US-03).

+(Public) checkAvailability	Boolean	
tenantId:	an	Verifies real-time room availability before
String, roomId:		finalizing a checkout or
String,		booking (US-04).
checkIn: Date,		
checkOut: Date)		

+(Public) manageRoomInventory(tenantId: Allows Hotel Admins to manage physical rooms
String, action: (US-15).
CRUDAction, data: RoomDTO)

+(Public) createRoomBlock	RoomBlock	
(tenantId:	lock	Locks units out of rotation for offline
String, roomId:		building/maintenance
String, dates:		needs (US-15).
DateRange)		

Reservatio + (Public)	createReservation	Reservation
onController	on(tenantId: String, guestId: String, details: ReservationDTO)	vatio Generates a room booking defaulted to a n PENDING state (US-05).

```

+(Public) confirmReservation Void
ion(tenantId: String, reservationId: String,
status to CONFIRMED
and broadcasting via
reservationId: String)
Socket.io (US-10).
String)

```

+(Public) registerCheckIn	Void
(tenantId: String, reservationId: String)	Changes status to CHECKED_IN and automatically updates room status to OCCUPIED (US-11).

+(Public) processCheckOut	Void
(tenantId: String, reservationId: String)	Marks reservation CHECKED_OUT, frees up the room, and auto-dispatches a cleanup task (US-12).

Housekeep + (Public)	getPrioritizedTasks	List<Task>
ingController	askQueue(tenantId: String, housekeeperId: String)	Task> Fetches list ordered strictly by: urgency == true first, then priority, then creation time (US-20).

+(Public) markTaskAsClean Void		
	ed(tenantId:	Sets task to DONE, room
	String, taskId:	to AVAILABLE, and
	String)	fires a WebSocket early-arrival alert to matching guests (US-21).
+(Public) overrideTaskPri Void		
	ority(tenantId:	Front-desk action to flip the urgency variable,
	String, taskId:	triggering a real-time list
	String, urgent:	re-sort (US-14).
	Boolean)	
AmenityRequirement +(Public)	getParkingLotMapping	List<
	servati on p(tenantId:	Parki Fetches interactive lot status mapping (US-06,
	Contr olle String)	ngSpo US-19).
	r	t>
+(Public) getSunbedPoolMapping		
	p(tenantId:	Sunbe Fetches grid layout allocations by daytime
	String, date:	d> slots (US-22).
	Date)	
+(Public) acquireTemporaryLock		
	yLock(tenantId:	an Updates asset state to LOCKEDwith a 5-minute
	String,	lockedUntilexpiry
	assetId:	on window (US-06,
	String,	US-22).
	guestId:	
	String)	

+ (Public) confirmAssetRes	Asset	
ervation(tenant	Reser	Converts valid holds into TAKEN and links
Id: String,	vatio	the cost directly to the
lockId : String,	n	active room reservation
paymentDetails:		folio (US-06, US-22).
DTO)		
Marketplace + (Public)	browseServiceMa	List<
ceControl	rketplace(tenan	Item> Lists available menu items or extra services
ler	tId: String)	filterable by local category (US-07).

PDF

+(Public) placeServiceOrder	
er(tenantId: String,	Processes guest orders and appends expenses straight onto their final
reservationId: String,	folio checkout bill (US-07).
cartItems: List)	
HotelAdmin + (Public)	manageStaffAccount User
nController	unts(tenantId: Enables user addition and immediate status
er	String, action: deactivation (US-16). CRUDAction, staffId: String)

+(Public) configureDynamic	Price	
cPricingRules(tenantId: String, rule: PricingRule DTO)	ngRules	Saves occupancy-based price adjustment limits
	e	(Premium/Custom tiers only) (US-17).
+(Public) getAnalyticsDashboard	Summary	
hboard(tenantId: String, dateRange: DateRange)	ry	Compiles performance metrics like occupancy rates and daily financial revenue logs (US-18).
+(Public) exportAnalytics	FileS	
CSV(tenantId: String,	stream	Streams tabular booking records directly into download

+(Public) exportAnalytics	FileS	
dateRange:		pipelines (US-18).
DateRange)		
SuperAdmin+(Public)	onboardNewHotel	Tenant
nControl	(data:	t Creates a clean tenant record, configures their
er	TenantOnboardDT	initial dedicated wildcard subdomain, and issues an invitation (US-26).
+(Public) manageTenantAccountVoid		
ountStatus(target		Allows explicit platform approval or
etTenantId:		complete operational
String, status:		suspension (US-23).
TenantStatus)		
+(Public) updateSubscriptionVoid		
ionPlan(targetT		Alters plan configurations (Basic,
enantId:		Premium, Custom),
String,		triggering live changes
planTier:		via feature gates
SubscriptionTier		(US-24).
r)		
+(Public) getPlatformWideAnalytics()	Platform	
		ormMe Aggregates cross-platform metrics performance health metrics without accessing individual guest data (US-25).

+ (Public) getPlatformWide	Platform	Aggregates
Analytics()	ormMe	cross-platform performance health metrics without accessing individual guest data (US-25).
Notification + (Public)	joinTenantRoom(Void	
ionEngine()	socket: Socket,	Places real-time websocket connections
WS)	tenantId:	into distinct isolated channels based on their tenantId.
+(Public) broadcastRoomStatusUpdate(tenantId: String, roomId: String, status: String)	Void	
atusUpdate(tena		Broadcasts updates when front desk
nId: String,		changes a room status
roomId: String,		(US-13).
status: String)		
+(Public) sendEarlyCheckInAlert(tenantId: String, guestId: String, message: String)	Void	
nAlert(tenantId		Pushes targeted alerts straight to a specific
: String,		user's web panel
guestId:		(US-08).
String,		
message:		
String)		
Background + (Public)	runNightlyPricingEngine()	Void
dJobWorker		Scans through active PricingRule items

runNightlyPricingEngine() Scans through active PricingRule items across all active hotels and adjusts pricing relative to current room capacity thresholds (US-17).

+(Public) reclaimExpiredLocks() Scans room, parking,

and sunbed lock arrays,
automatically flushing
any items exceeding
their 5-minute hold
limits (US-06, US-22).

6.2 Data Dictionary

Detailed field-by-field description of all Prisma models in the Reservio schema.

6.2.1 Tenant Model

Field	Type	Description / Constraints
id	String	PK, UUID auto-generated
name	String	Hotel display name shown on guest portal
subdomain	String	Unique subdomain. Used by tenantResolver for routing
plan	Enum	BASIC PREMIUM CUSTOM - controls featureGate access
active	Boolean	false = suspended; tenantResolver rejects all requests
status	Enum	PENDING APPROVED - self-signup starts PENDING
ownerEmail	String	Email of primary Hotel Admin
createdAt	DateTime	Tenant provisioning timestamp

6.2.2 User Model

Field	Type	Description / Constraints
id	String	PK, UUID
tenantId	String	FK to Tenant.id. Auto-scoped by Prisma extension
fullName	String	Display name for portals and dashboards
email	String	Unique within tenant scope. Login identifier
passwordHash	String	bcrypt hash, salt rounds >= 10. Never returned in responses
role	Enum	GUEST RECEPTIONIST HOUSEKEEPER TECHNICIAN MANAGER HOTEL_ADMIN SUPER_ADMIN
active	Boolean	false = deactivated; login rejected by authMiddleware

Field	Type	Description / Constraints
username	String	Auto-generated slug. Used for internal references

6.2.3 Room Model

Field	Type	Description / Constraints
id	String	PK, UUID
tenantId	String	FK to Tenant.id
type	String	Room type label (Standard, Deluxe, Suite, etc.)
description	String	Full text description shown on guest portal
pricePerNight	Float	Base price. Modified nightly by PricingRule engine
status	Enum	AVAILABLE OCCUPIED MAINTENANCE
photos	String[]	Array of photo URLs (CDN or picsum in development)
floorNumber	Int	Optional floor for housekeeping priority sorting
capacity	Int	Maximum guest count for the room

6.2.4 Reservation Model

Field	Type	Description / Constraints
id	String	PK, UUID
tenantId	String	FK to Tenant.id
userId	String	FK to User.id (the booking guest)
roomId	String	FK to Room.id
checkIn	DateTime	Start date of stay (midnight local time)
checkOut	DateTime	End date of stay

userId	String	FK to User.id (the booking guest)
status	Enum	PENDING CONFIRMED CHECKED_IN CHECKED_OUT CANCELLED
totalPrice	Float	Calculated: pricePerNight x nights (post-rule adjustment)
referenceCode	String	Human-readable booking reference (RSV-20260421-ABCD)

6.2.5 HousekeepingTask Model

Field	Type	Description / Constraints
id	String	PK, UUID
tenantId	String	FK to Tenant.id
roomId	String	FK to Room.id
priority	Int	Lower = higher priority. Calculated from check-out time
urgency	Boolean	Set true by staff override (US-14). Always sorts to top
status	Enum	PENDING IN_PROGRESS DONE
assignedTo	String?	FK to User.id (Housekeeper). Optional
completedAt	DateTime?	Timestamp when marked DONE

6.2.6 PricingRule Model

Field	Type	Description / Constraints
id	String	PK, UUID
tenantId	String	FK to Tenant.id (Premium/Custom tenants only)
occupancyThreshold	Float	Percentage (0-100) at or above which adjustment activates

occupancyThreshold	Float	Percentage (0-100) at or above which adjustment activates
adjustmentPercent	Float	Positive = price increase, negative = discount (e.g. 20 = +20%)
enabled	Boolean	Admin can disable rules without deleting
lastAppliedAt	DateTime	Timestamp of last nightly evaluation that triggered this rule

7. Data Flow Diagrams

Data flow diagrams show how information moves through the Reservio system for each of the five user portals. Each DFD traces the path from user action through middleware layers, to the database, and back including any WebSocket events emitted.

7.1 Guest Portal Data Flow

The Guest portal handles room browsing (public, no auth), reservation booking, parking/sunbed maps, marketplace orders, and real-time check-in notifications.

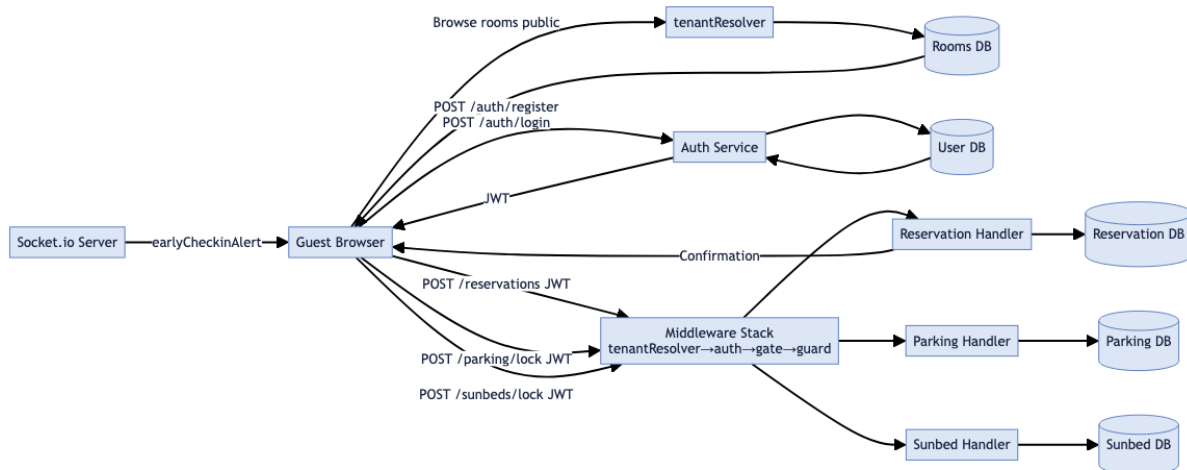


Figure 7.1 Data Flow Diagram: Guest Portal

7.2 Staff Dashboard Data Flow

The Staff dashboard handles reservation confirmation, check-in/out processing, room status updates, and housekeeping priority overrides. All updates broadcast via tenant-scoped WebSocket events.

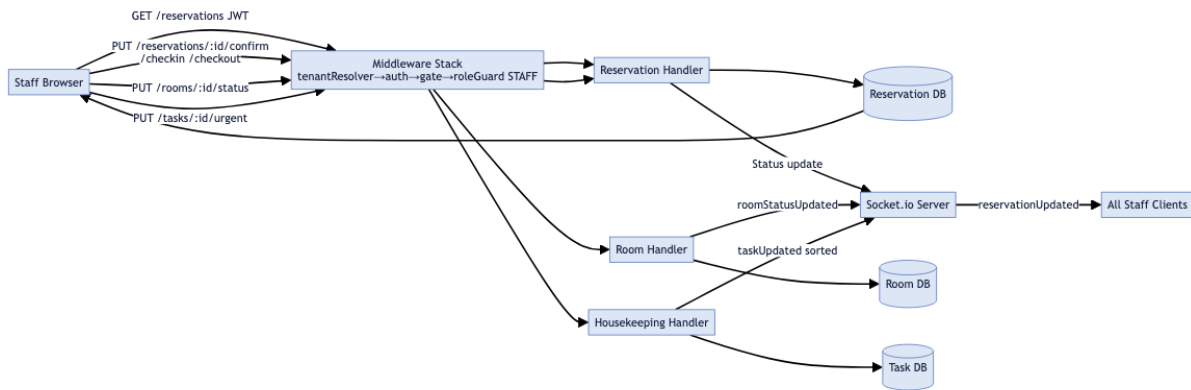


Figure 7.2 Data Flow Diagram: Staff Dashboard

7.3 Housekeeper Task Board Data Flow

The Housekeeper board retrieves the auto-sorted task list and handles mark-as-cleaned events. Completing a task triggers downstream WebSocket notifications to front desk and eligible guests.

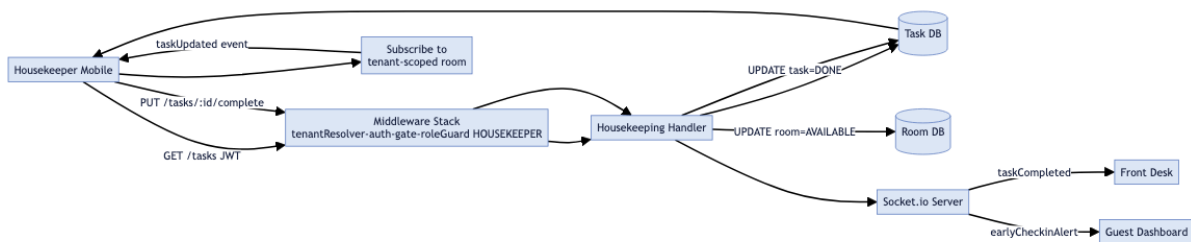


Figure 7.3 Data Flow Diagram: Housekeeper Task Board

7.4 Hotel Admin Panel Data Flow

The Admin panel handles room and staff management (all tiers) plus dynamic pricing and analytics (Premium/Custom only). The node-cron job feeds back into the pricing rules at 2:00 AM nightly.

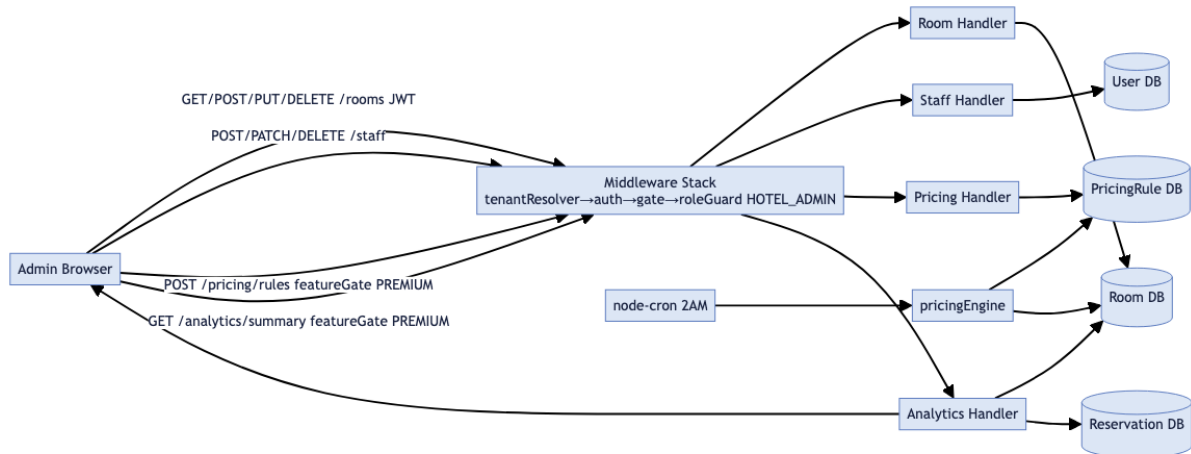


Figure 7.4 Data Flow Diagram: Hotel Admin Panel

8. Detailed User Story Specifications

This chapter provides full structured specifications for all 26 user stories including: preconditions, step-by-step main flow, alternative/exception flows, postconditions, subscription tier requirement, and primary API endpoint.

Guest Detailed Specifications

US-01 Guest Account Registration

Field	Value
User Story ID	US-01
Actor / Role	Guest
Subscription Gate	All tiers
Primary Endpoint	POST /api/auth/register
Preconditions	Hotel subdomain is accessible; tenant is active and approved

Main Success Scenario:

1. Guest navigates to hotel subdomain registration page

2. Guest enters full name, email address, and password

3. System validates all fields (required, email format, min 8-char password)

**4. System checks email uniqueness within the tenant (Prisma query scoped by
tenantId)**

5. System hashes password with bcrypt (salt rounds ≥ 10)

6. System creates User record with role=GUEST, tenantId attached

7. System returns JWT containing userId, tenantId, role

8. Guest is redirected to guest dashboard

Alternative / Exception Flows:

- Email already registered - display This email is already registered error
- Invalid email format - inline validation error
- Password too short - inline validation error
- Tenant suspended - display Service unavailable message Postconditions: Guest account is created and scoped to the hotel tenant. Guest can log in immediately.

US-02 Guest Login

Field	Value
User Story ID	US-02
Actor / Role	Guest
Subscription Gate	All tiers
Primary Endpoint	POST /api/auth/login

Field	Value
Preconditions	Guest account exists within tenant; account is active

Main Success Scenario:

- 1. Guest navigates to hotel login page**
- 2. Guest enters registered email and password**
- 3. System resolves subdomain to tenantId via tenantResolver middleware**
- 4. System queries User WHERE email=input AND tenantId=resolved**
- 5. System runs bcrypt.compare(inputPw, storedHash)**
- 6. On match: system generates signed JWT (7-day expiry) with userId, tenantId,
role=GUEST**
- 7. JWT is returned to client and stored client-side**
- 8. Guest redirected to guest dashboard showing upcoming reservations**

Alternative / Exception Flows:

- Wrong password - Invalid email or password (no hint which is wrong)
- Account deactivated - Your account has been deactivated message
- Account belongs to different tenant - treated as not found (security)

Postconditions:

Guest is authenticated. JWT is stored client-side. All subsequent API calls include Bearer token.

US-03 Browse Available Rooms

Field	Value
User Story ID	US-03
Actor / Role	Guest
Subscription Gate	All tiers
Primary Endpoint	GET /api/rooms
Preconditions	Tenant is active; at least one room exists in inventory

Main Success Scenario:

- 1. Guest navigates to rooms page (no login required)**
- 2. Frontend sends GET /api/rooms with X-Tenant-Subdomain header**
 - tenantResolver resolves subdomain to tenantId
- 4. Prisma query: SELECT from rooms WHERE tenantId=X AND status=AVAILABLE**
- 5. API returns room list with type, price, description, and photo URLs**
- 6. Frontend renders room cards in a responsive grid**
- 7. Guest can view each room detail by clicking the card**

Alternative / Exception Flows:

- No rooms available - display No rooms currently available placeholder

- Tenant not found (bad subdomain) - 404 Not Tenant Found
- Network error - display error toast with retry button

Postconditions:

Guest can see all available rooms. No reservation is created at this stage.

US-04 Check Room Availability

Field	Value
User Story ID	US-04
Actor / Role	Guest
Subscription Gate	All tiers
Primary Endpoint	GET /api/rooms?checkIn=&checkOut=
Preconditions	Guest has entered check-in and check-out dates; check-in must be before check-out
Main Success Scenario:	
1. Guest enters check	-in and check-out dates in the date picker
2. Frontend sends GET	/api/rooms?checkIn=X&checkOut=Y
3. API queries reserv	ations overlapping the date range (PENDING, CONFIRMED,
CHECKED_IN statuses)	
4. API also queries R	oomBlock records overlapping the date range
5. Rooms with any ove	rlap are excluded from results
6. API returns availa	ble room IDs; frontend filters and re-renders the room grid
7. Unavailable rooms	are hidden or greyed out

Alternative / Exception Flows:

- Check-in date in the past - client-side validation prevents submission
- Check-in equals check-out - validation error Minimum 1 night stay
- All rooms booked - display No rooms available for these dates

Postconditions:

Guest sees only rooms available for the requested dates. No reservation is created.

US-05 Make a Reservation

Field	Value
User Story ID	US-05

Field	Value
Actor / Role	Guest
Subscription Gate	All tiers
Primary Endpoint	POST /api/reservations
Preconditions	Guest is logged in; room is available for selected dates; guest is not on hotel blacklist

Main Success Scenario:

1. Guest selects a room and clicks Book Now

2. System re-validates availability in real time (race-condition guard)

3. authMiddleware verifies GUEST JWT

4. System checks guestBlacklist table for guest userId + tenantId

5. System creates Reservation record: status=PENDING, tenantId, userId, roomId,

checkIn, checkOut

6. System calculates total price (price x nights, adjusted by any active pricing rule)

7. Confirmation response includes reservationId and reference code

8. Frontend displays booking confirmation screen

Alternative / Exception Flows:

- Room taken between browse and confirm - This room was just booked, please choose another

- Guest blacklisted - Sorry, we are unable to process your booking
- JWT expired - redirect to login
- Dates overlapping a new reservation - re-run availability check and return error

Postconditions:

Reservation exists in PENDING status. Staff must confirm before status moves to CONFIRMED.

US-06 Reserve a Parking Spot

Field	Value
User Story ID	US-06
Actor / Role	Guest
Subscription Gate	Premium, Custom
Primary Endpoint	POST /api/parking/:id/lock, POST /api/parking/:id/confirm
Preconditions	Guest is logged in; tenant has parking spots configured (Premium/Custom plan); spot is AVAILABLE

Main Success Scenario:

- 1. Guest opens parking map; API returns all spots with current status**
- 2. Guest clicks an AVAILABLE spot (shown green)**
- 3. POST /api/parking/:id/lock - status=LOCKED, lockedUntil=now+5min**
- 4. Spot turns yellow with a 5-minute countdown timer on the map**
- 5. Guest reviews spot details and clicks Confirm**

6. POST /api/parking/:id/confirm - status=TAKEN, creates ParkingReservation record

7. Spot turns red on the map for all connected clients via WebSocket

Alternative / Exception Flows:

- Guest selects a spot just locked by another guest - This spot is no longer available
- 5-minute timer expires before confirmation - spot returns to AVAILABLE
- Guest cancels - POST /api/parking/:id/release - status=AVAILABLE

Postconditions:

Parking spot is TAKEN and linked to guest. Spot shown red on map for all guests.

US-07 Order In-App Services

Field	Value
User Story ID	US-07
Actor / Role	Guest
Subscription Gate	Premium, Custom
Primary Endpoint	GET /api/services, POST /api/orders
Preconditions	Guest is logged in; guest has an active reservation; tenant has services configured

Main Success Scenario:

1. Guest navigates to marketplace; GET /api/services returns tenant service catalogue

2. Services are displayed as cards with category tabs (Spa, Room Service, Transport,

Other)

3. Guest selects one or more services and adds to cart

4. Guest clicks Bill to Room

5. POST /api/orders with {serviceId, reservationId, quantity}

6. API validates reservationId belongs to authenticated guest and is active

7. Order record created with status=PENDING

8. Cost is logged against the reservation for billing reconciliation

Alternative / Exception Flows:

- No active reservation - You must have an active reservation to order services
- Service not available - marked unavailable in catalogue
- Tenant plan is Basic - 403 Feature not available on your current plan

Postconditions:

Order created. Service cost logged against reservation. Hotel staff can view order in admin panel.

US-08 Receive Early Check-In Notification

Field	Value
User Story ID	US-08
Actor / Role	Guest
Subscription Gate	Premium, Custom (WebSocket)
Primary Endpoint	PUT /api/tasks/:id/complete (triggers notification)
Preconditions	Guest has a confirmed reservation for today; housekeeper has marked the room as cleaned

Main Success Scenario:

1. Housekeeper marks room as cleaned (US-21)

2. API handler queries reservations WHERE roomId=X AND checkIn=today AND

status=CONFIRMED

3. For each matching reservation: fetch guest userId

4. Emit WebSocket event earlyCheckinAlert to socket room

tenant:{tenantId};guest:{userId}

5. Guest dashboard receives the event and displays notification badge

6. Notification shows: room number, status=Ready, timestamp

Alternative / Exception Flows:

- No confirmed reservation for today on that room - no notification sent
- Guest is offline - notification displayed on next dashboard load via polling fallback
- Multiple guests with same-day check-in - each receives individual notification

Postconditions:

Guest is notified in real time. Dashboard badge count increments.

US-22 Reserve a Sunbed

Field	Value
User Story ID	US-22
Actor / Role	Guest
Subscription Gate	Premium, Custom
Primary Endpoint	POST /api/sunbeds/:id/lock, POST /api/sunbeds/:id/confirm
Preconditions	Guest is logged in; tenant on Premium or Custom plan; sunbeds exist and at least one is AVAILABLE

Main Success Scenario:

- 1. Guest opens sunbed reservation map**
- 2. GET /api/sunbeds - tenant sunbeds with status and colour coding**
- 3. Guest selects a date and time slot (Morning 8-12, Afternoon 12-18, Full Day)**
- 4. Guest clicks an AVAILABLE sunbed (green)**
- 5. POST /api/sunbeds/:id/lock - status=LOCKED, lockedUntil=now+5min**
- 6. Sunbed turns yellow; countdown begins**
- 7. Guest confirms - POST /api/sunbeds/:id/confirm {date,slot}**
- 8. SunbedReservation INSERT; sunbed status=TAKEN**
- 9. Cost billed to active reservation (consistent with US-07 billing logic)**

Alternative / Exception Flows:

- Sunbed locked by another guest just before click - This sunbed is no longer available
- Lock expires - sunbed returns to AVAILABLE; guest shown timeout message
- Guest has no active reservation - standalone booking created

Postconditions:

Sunbed TAKEN and linked to guest for selected date and slot.

Staff Detailed Specifications

US-09 View Reservation Dashboard

Field	Value
User Story ID	US-09
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	GET /api/reservations
Preconditions	Staff is logged in with role=RECEPTIONIST or MANAGER; tenant is active

Main Success Scenario:

- 1. Staff navigates to reservation dashboard**
- 2. GET /api/reservations with optional query params (?status=&dateFrom=&dateTo=)**
 3. authMiddleware validates JWT; roleGuard confirms STAFF role
- 4. Prisma query: SELECT reservations JOIN users JOIN rooms WHERE tenantId=X**
- 5. Results ordered by checkIn date descending by default**
- 6. Dashboard displays table: guest name, room number, dates, status badge, actions**
- 7. Staff can apply filters by status or date range**
- 8. Search box filters by guest name in real time**

Alternative / Exception Flows:

- No reservations exist - No reservations found empty state

- Session expired - redirect to staff login

Postconditions:

Staff sees full reservation list for their hotel. Data is scoped exclusively to their tenantId.

US-10 Confirm Pending Reservations

Field	Value
User Story ID	US-10
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	PUT /api/reservations/:id/confirm
Preconditions	A reservation with status=PENDING exists; staff is logged in

Main Success Scenario:

- 1. Staff views reservation dashboard and locates a PENDING reservation**
- 2. Staff clicks the Confirm button**
- 3. PUT /api/reservations/:id/confirm**
 4. authMiddleware + roleGuard(STAFF) validates request
- 5. Reservation status updated to CONFIRMED in DB**
- 6. WebSocket event reservationUpdated emitted to tenant room**
- 7. All connected staff clients update status badge to Confirmed (green)**
- 8. Activity log entry created: confirmed by staff userId**

Alternative / Exception Flows:

- Reservation already confirmed by another staff member - This reservation was already confirmed
- Reservation cancelled before staff confirms - Cannot confirm a cancelled reservation

Postconditions:

Reservation status is CONFIRMED. Guest can now check in on the arrival date.

US-11 Register Guest Check-In

Field	Value
User Story ID	US-11
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	PUT /api/reservations/:id/checkin
Preconditions	Reservation is CONFIRMED; checkIn date is today or past; guest is present at reception

Main Success Scenario:

- 1. Staff finds reservation in dashboard filtered to today arrivals**
- 2. Staff clicks Register Check-In**
- 3. PUT /api/reservations/:id/checkin**
- 4. Reservation status updated to CHECKED_IN**
- 5. Room status updated to OCCUPIED**
- 6. WebSocket broadcast: roomStatusUpdated to all staff**

7. Activity log: checked in by staff userId at timestamp

Alternative / Exception Flows:

- Reservation not yet confirmed - staff must confirm first (US-10)
- Room already OCCUPIED - data inconsistency error logged; staff alerted
- Check-in date is in the future - warning shown but staff can override

Postconditions:

Guest is officially checked in. Room shows OCCUPIED. Reservation is CHECKED_IN.

US-12 Process Guest Check-Out

Field	Value
User Story ID	US-12
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	PUT /api/reservations/:id/checkout
Preconditions	Reservation is in CHECKED_IN status

Main Success Scenario:

1. Staff finds active reservation in dashboard
2. Staff clicks Process Check-Out
3. PUT /api/reservations/:id/checkout
4. Reservation status changes to CHECKED_OUT
5. Room status changes to AVAILABLE
6. INSERT HousekeepingTask: priority=1, roomId, tenantId, status=PENDING

7. WebSocket: roomStatusUpdated + taskCreated events emitted

8. Activity log entry created

Alternative / Exception Flows:

- Outstanding unpaid service orders - warning displayed but checkout can proceed
- Room already in MAINTENANCE - room stays MAINTENANCE, task still created

Postconditions:

Guest is checked out. Room available for new guests. Housekeeping task auto-queued.

US-13 Update Room Status

Field	Value
User Story ID	US-13
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	PUT /api/rooms/:id/status
Preconditions	Staff is logged in; room exists in tenant inventory

Main Success Scenario:

1. Staff navigates to the room status grid

2. Staff selects a new status from the room card dropdown (Available / Occupied / Maintenance)

3. PUT /api/rooms/:id/status { status: newStatus }

4. DB update: room.status = newStatus

5. WebSocket: emit roomStatusUpdated to tenant room

6. All connected staff and admin clients re-render the room card with new status

colour

Alternative / Exception Flows:

- Attempt to set AVAILABLE on an OCCUPIED room without check-out - warning shown
- WebSocket connection dropped - status still persisted; client refreshes on reconnect

Postconditions:

Room status persisted. All connected devices reflect the change in real time.

US-14 Override Housekeeping Priority

Field	Value
User Story ID	US-14
Actor / Role	Staff
Subscription Gate	All tiers
Primary Endpoint	PUT /api/tasks/:id/urgent
Preconditions	A housekeeping task with status=PENDING exists for the target room; staff is logged in

Main Success Scenario:

1. Staff views housekeeping priority panel
2. Staff clicks Mark Urgent on a task
3. PUT /api/tasks/:id/urgent { urgency: true }
4. DB update: task.urgency = true

5. API re-queries sorted task list: urgency DESC, priority ASC

6. WebSocket: emit taskUpdated with full sorted task array

7. Housekeeper task board re-renders with urgent task at top, displayed with red

badge

Alternative / Exception Flows:

- Task already marked urgent - toggle removes urgency flag
- Task already completed - Cannot modify a completed task

Postconditions:

Task urgency flag set. Housekeeper sees the task at top of queue immediately.

Housekeeper Detailed Specifications

US-20 Receive Auto-Prioritised Task Queue

Field	Value
User Story ID	US-20
Actor / Role	Housekeeper
Subscription Gate	All tiers
Primary Endpoint	GET /api/tasks
Preconditions	Housekeeper is logged in; housekeeping tasks exist for the tenant

Main Success Scenario:

1. Housekeeper opens task board on mobile device

2. GET /api/tasks - roleGuard(HOUSEKEEPER) - tasks for tenantId

3. Tasks sorted: urgency=true first, then priority ASC, then createdAt ASC

4. Each task card shows: room number, floor, check-out time, priority badge, urgency

flag

5. Housekeeper subscribes to Socket.io room tenant:{tenantId}

6. When staff marks a task urgent (US-14) - taskUpdated event received - list

re-renders automatically

Alternative / Exception Flows:

- No tasks - No tasks assigned empty state
- WebSocket disconnects - task board shows stale data badge; reconnect on focus

Postconditions:

Housekeeper sees real-time, correctly prioritised task queue. Urgent tasks always appear first.

US-21 Mark Room as Cleaned

Field	Value
User Story ID	US-21
Actor / Role	Housekeeper
Subscription Gate	All tiers
Primary Endpoint	PUT /api/tasks/:id/complete
Preconditions	Housekeeper is logged in; task with status=PENDING exists for room

Main Success Scenario:

1. Housekeeper finds task card and taps Mark as Cleaned

2. PUT /api/tasks/:id/complete

3. Task status changes to DONE

4. Room status changes to AVAILABLE (PUT /api/rooms/:id/status)

5. Emit taskCompleted via Socket.io - front desk view updates

6. Query: SELECT reservations WHERE roomId=X AND checkIn=today AND

status=CONFIRMED

7. For each matching reservation: emit earlyCheckinAlert to guest (US-08)

8. Activity log entry created

Alternative / Exception Flows:

- Task already completed - This task has already been marked as done
- Room has a CONFIRMED reservation for today - early check-in alert sent (US-08)

Postconditions:

Room is AVAILABLE. Task is DONE. Front desk is notified. Guest may receive early check-in alert.

Hotel Admin Detailed Specifications

US-15 Manage Hotel Rooms

Field	Value
User Story ID	US-15
Actor / Role	Hotel Admin

Field	Value
Subscription Gate	All tiers (Basic max 20 rooms)
Primary Endpoint	POST/PUT/DELETE /api/rooms, POST /api/room-blocks
Preconditions	User is logged in as Hotel Admin for the tenant

Main Success Scenario:

- 1. Admin navigates to Room Management**
- 2. GET /api/rooms - all rooms including non-available (admin view)**
- 3. Admin clicks Add Room - fills form: type, description, pricePerNight, photos**
- 4. POST /api/rooms - INSERT room with tenantId**
- 5. To edit: PUT /api/rooms/:id - update fields**
- 6. To delete: DELETE /api/rooms/:id (blocked if active reservations exist)**
- 7. To block dates: POST /api/room-blocks { roomId, from, to, reason }**

Alternative / Exception Flows:

- Delete with active reservation - Cannot delete a room with active bookings
- Photo upload fails - error toast; room can be saved without photo

Postconditions:

Room inventory updated. Changes visible to guests immediately (available rooms).

US-16 Manage Staff Accounts

Field	Value
User Story ID	US-16
Actor / Role	Hotel Admin
Subscription Gate	All tiers (Basic max 5 staff accounts)
Primary Endpoint	POST/PATCH/DELETE /api/staff
Preconditions	User is logged in as Hotel Admin

Main Success Scenario:**1. Admin navigates to Staff Management****2. GET /api/staff - all staff accounts for tenant****3. Admin clicks Add Staff - fills: name, email, role**

(Receptionist/Housekeeper/Technician/Manager)

4. POST /api/staff - bcrypt.hash - INSERT user with role, tenantId, active=true**5. Username auto-generated from first.last pattern + random suffix if collision****6. To deactivate: PATCH /api/staff/:id/deactivate - active=false****7. To delete: DELETE /api/staff/:id - soft delete (preserves activity logs)*****Alternative / Exception Flows:***

- Email already used in tenant - Email already registered for this hotel
- Deactivating own account - blocked: Cannot deactivate your own account

Postconditions:

Staff can log in with their credentials. Deactivated accounts cannot authenticate.

US-17 Configure Dynamic Pricing Rules

Field	Value
User Story ID	US-17
Actor / Role	Hotel Admin
Subscription Gate	Premium, Custom only
Primary Endpoint	POST /api/pricing/rules, cron: runPricingEngine()
Preconditions	Tenant is on Premium or Custom plan; Hotel Admin is logged in

Main Success Scenario:

- 1. Admin navigates to Dynamic Pricing section**
- 2. GET /api/pricing/rules - existing rules for tenant**
- 3. Admin sets: occupancyThreshold (%) and adjustmentPercent (+/- %)**
- 4. POST /api/pricing/rules - INSERT PricingRule with tenantId**
- 5. Admin can enable/disable rules with toggle**
- 6. Nightly cron job (node-cron 0 2 * * *) runs runPricingEngine()**
- 7. Engine calculates current occupancy per tenant (confirmed/total rooms x 100)**

8. If occupancy $\geq$ threshold - UPDATE rooms.pricePerNight x (1 + adjustment/100)

9. Admin sees real-time occupancy % on the pricing dashboard

Alternative / Exception Flows:

- Tenant on Basic plan - featureGate returns 403
- Multiple rules: rules applied in order of occupancyThreshold descending
- Zero rooms - cron skips tenant

Postconditions:

Pricing rules saved. Next nightly run will apply adjustments if thresholds are met.

US-18 View Analytics Dashboard

Field	Value
User Story ID	US-18
Actor / Role	Hotel Admin
Subscription Gate	Premium, Custom only
Primary Endpoint	GET /api/analytics/summary, GET /api/analytics/export
Preconditions	Tenant is on Premium or Custom plan; Hotel Admin is logged in; reservations data exists

Main Success Scenario:

1. Admin navigates to Analytics

2. GET /api/analytics/summary?from=&to=

3. featureGate validates Premium/Custom plan

4. API aggregates: COUNT(reservations), SUM(totalPrice), occupied/total rooms x

5. Returns KPI data: totalBookings, dailyRevenue (array), occupancyPct, avgStayLength

6. Frontend renders KPI cards and line chart

7. Admin can change date range; frontend re-fetches

8. Export: GET /api/analytics/export - streams CSV of reservation rows

Alternative / Exception Flows:

- No reservations in range - KPIs show zero; chart shows flat line
- Export on Basic plan - 403 Feature not included in your plan

Postconditions:

Admin has visibility into hotel performance. No cross-tenant data is included.

US-19 Manage Parking Lot

Field	Value
User Story ID	US-19
Actor / Role	Hotel Admin
Subscription Gate	All tiers
Primary Endpoint	POST/PATCH/DELETE /api/parking
Preconditions	Hotel Admin is logged in; tenant may be any plan (parking management is All tiers)

Main Success Scenario:

1. Admin navigates to Parking Management

2. GET /api/parking - all spots for tenant

3. Admin clicks Add Spot - label, pricePerNight

4. POST /api/parking - INSERT ParkingSpot with tenantId, status=AVAILABLE

5. To set price: PATCH /api/parking/:id/price

6. To remove: DELETE /api/parking/:id (only if not TAKEN)

7. Occupancy summary shows: total spots, taken, available, locked

Alternative / Exception Flows:

- Delete a TAKEN spot - Cannot remove a spot that is currently reserved
- Label already used - Spot label must be unique within your parking lot

Postconditions:

Parking inventory updated. Changes visible to guests on the parking map immediately.

Super Admin Detailed Specifications

US-23 Manage Tenant Accounts

Field	Value
User Story ID	US-23
Actor / Role	Super Admin
Subscription Gate	Super Admin only
Primary Endpoint	GET/PUT/DELETE /api/admin/tenants
Preconditions	User is logged in as Super Admin (cross-tenant JWT with role=SUPER_ADMIN)

Main Success Scenario:

1. Super Admin navigates to Tenants list

2. GET /api/admin/tenants - all tenants with stats (_count reservations, staff, rooms)

3. Super Admin views pending tenants and clicks Approve

4. PUT /api/admin/tenants/:id/approve - active=true, status=APPROVED

5. To suspend: PUT /api/admin/tenants/:id/suspend - active=false

6. Suspended tenant - tenantResolver marks all requests 403 until reactivated

7. To permanently delete: DELETE /api/admin/tenants/:id (cascade deletes all data)

8. Activity log records all Super Admin actions

Alternative / Exception Flows:

- Tenant not found - 404
- Deleting active tenant - confirmation modal: This will permanently delete all hotel data

Postconditions:

Tenant account status updated. Changes take effect on next API call from that tenant.

US-24 Manage Subscription Plans

Field	Value
User Story ID	US-24
Actor / Role	Super Admin
Subscription Gate	Super Admin only
Primary Endpoint	PUT /api/admin/tenants/:id/plan

Field	Value
Preconditions	Super Admin is logged in; target tenant exists

Main Success Scenario:

- 1. Super Admin navigates to Tenants list**
- 2. Super Admin locates the target hotel**
- 3. Clicks Change Plan - dropdown (Basic / Premium / Custom)**
- 4. PUT /api/admin/tenants/:id/plan { plan: newPlan }**
- 5. DB update: tenant.plan = newPlan**
- 6. On next API call from tenant: tenantResolver loads updated tenant record**
 7. featureGate reads the new plan - grants or denies Premium/Custom endpoints
- 8. Downgraded tenant immediately loses access to restricted features**

Alternative / Exception Flows:

- Same plan selected - no-op with Plan unchanged toast
- Tenant suspended - plan change still persists; takes effect when reactivated

Postconditions:

Plan updated in DB. Feature access gates updated on next request from tenant.

US-25 View Platform-Wide Analytics

Field	Value
User Story ID	US-25
Actor / Role	Super Admin

Field	Value
Subscription Gate	Super Admin only
Primary Endpoint	GET /api/admin/analytics
Preconditions	Super Admin is logged in

Main Success Scenario:

- 1. Super Admin navigates to Platform Analytics**
- 2. GET /api/admin/analytics?from=&to=&plan=**
- 3. API aggregates across ALL tenants (no tenantId filter - this is cross-tenant)**
- 4. Returns: totalActiveTenants, totalReservations, breakdownByPlan, totalStaffUsers**
- 5. Admin can filter by date range and plan tier**
- 6. Chart shows tenant growth over time (weekly/monthly)**
- 7. No individual guest data (names, emails) is included in any aggregate**

Alternative / Exception Flows:

- No tenants yet - all metrics show 0
- Date range with no reservations - zero revenue, valid response

Postconditions:

Super Admin has platform-level visibility. No guest PII is exposed.

US-26 Onboard a New Hotel

Field	Value
User Story ID	US-26
Actor / Role	Super Admin
Subscription Gate	Super Admin only
Primary Endpoint	POST /api/admin/tenants
Preconditions	Super Admin is logged in; target subdomain is not already taken

Main Success Scenario:

- 1. Super Admin navigates to Onboard New Hotel**
- 2. Super Admin fills form: hotel name, subdomain, plan, owner email**
- 3. POST /api/admin/tenants**
- 4. System validates subdomain uniqueness across all tenants**
- 5. INSERT Tenant record: name, subdomain, plan, active=true, status=APPROVED**
- 6. Subdomain immediately resolvable by tenantResolver on next request**
- 7. System sends invitation email to owner (console.log in academic version)**
- 8. Super Admin sees new tenant in list with ACTIVE badge**

Alternative / Exception Flows:

- Subdomain already taken - This subdomain is already in use
- Invalid subdomain (spaces/special chars) - validation error
- Owner email already has a Super Admin account - warning shown

Postconditions:

Hotel tenant provisioned and active. Hotel Admin can set up the workspace.

9. API Endpoint Catalogue

All REST API endpoints in the Reservio Express.js backend. Every endpoint passes through: tenantResolver → authMiddleware → featureGate → roleGuard.

Auth Level	Description
Public	No JWT required. tenantResolver still resolves tenant from subdomain.
Guest	JWT with role=GUEST.
Staff+	JWT with role RECEPTIONIST, MANAGER, HOTEL_ADMIN, or SUPER_ADMIN.
Hotel Admin	JWT with role=HOTEL_ADMIN. featureGate checks plan tier for Premium endpoints.
Housekeeper	JWT with role=HOUSEKEEPER.
Super Admin	JWT with role=SUPER_ADMIN. No tenantId restriction in middleware.

9.1 Authentication

Met Endpoint	Auth	Description
POST /api/auth/register	Public	Register new guest account scoped to tenant. Returns JWT.
POST /api/auth/login	Public	Authenticate guest or staff. Returns signed JWT with tenantId + role.

Met Endpoint	Auth	Description
POST /api/auth/refresh	JWT	Refresh an expiring JWT. Returns new
T		JWT with extended expiry.

9.2 Rooms

Met	Endpoint	Auth Description
GET		
GET	/api/rooms	Public List available rooms for tenant.
T		?checkIn=&checkOut= for availability filter.
POST /api/rooms		Hotel Admin Create a new room. Body: {type,
T		description, pricePerNight, photos}.
PUT /api/rooms/:id		Hotel Admin Update room details.
T		
DELETE /api/rooms/:id		Hotel Admin Delete room. Rejected if active reservations
LET		exist.
E		
PUT /api/rooms/:id/status		Staff Update room status. Broadcasts via
T		WebSocket.
POST /api/room-blocks		Hotel Admin Block date range for maintenance.
T		

9.3 Reservations

Met	Endpoint	Auth Description
GET		
GET	/api/reservations	Staff+ List all tenant reservations. Filterable by
T		status, date, guest name.

Met	Endpoint	Auth Description
GE	/api/reservations/my	Guest List authenticated guest reservations.
T		
POS /a	pi/reservations	Guest Create reservation. Validates availability in
T		real time.
PU	/api/reservations/:id/	Staff Confirm PENDING reservation. Emits
T	confirm	WebSocket event.
PU	/api/reservations/:id/	Staff Register check-in. Room set to
T	checkin	OCCUPIED.
PU	/api/reservations/:id/	Staff Process check-out. Creates housekeeping
T	checkout	task. Room set to AVAILABLE.
PU	/api/reservations/:id/	Staff+Guest Cancel a PENDING or CONFIRMED
T	cancel	reservation.

9.4 Parking

Met Endpoint	Auth	Description
hod		
GE /api/parking	Public	List all parking spots with current status for
T		the visual map.
POS /api/parking	Hotel Admin	Add a new parking spot. Body: {label,
T		pricePerNight}.
PAT /api/parking/:id/price	Hotel Admin	Update per-spot pricing.
CH		

Met Endpoint	Auth	Description
DELETE /api/parking/:id	Hotel Admin	Remove spot. Rejected if status=TAKEN.
LET		
E		
POST /api/parking/:id/lock	Guest	Lock spot for 5 minutes. Sets
T		lockedUntil=now+5min.
POST /api/parking/:id/confirm	Guest	Confirm locked spot. Creates
Term		ParkingReservation.

9.5 Sunbeds

Met Endpoint	Auth	Description
GET /api/sunbeds	Public	List sunbeds with status and zone for pool
T		map.
POST /api/sunbeds/:id/lock	Guest	Lock sunbed for 5 minutes. Body: {date,
T		slot}.

Met E ndpoi nt	Auth	Description
POS / api/su nbeds/ :id/co nf	Guest	Confirm locked sunbed. Creates
T irm		SunbedReservation.
POS / api/su nbeds/ :id/rel ea	Guest	Release a locked sunbed.
T se		

9.6 Marketplace & Orders

Met	Endpoint	Auth Description
hod		
GE	/api/services	Guest List all available services for tenant by
T		category.
POS /ap	i/services	Hotel Admin Add marketplace service. Body: {name,
T		category, price}.
POS /ap	i/orders	Guest Place service order. Body: {serviceId,
T		reservationId, quantity}.
GE	/api/orders	Hotel Admin List all orders for tenant.
T		

9.7 Housekeeping

Met	Endpoint	Auth Description
hod		
GE	/api/tasks	Housekeeper List tasks sorted by urgency DESC, priority
T		+Staff ASC. Tenant-scoped.

Met	Endpoint	Auth Description
PU	/api/tasks/:id/urgent	Staff Set urgency flag. Emits taskUpdated with
T		re-sorted list.
PU	/api/tasks/:id/complet House	keeper Mark DONE. Updates room. Emits
T	e	earlyCheckinAlert if applicable.

9.8 Pricing & Analytics

Met	Endpoint	Auth Description
hod		
GE	/api/pricing/rules	Hotel Admin List pricing rules for tenant.
T		(Premium)
POS /a	pi/pricing/rules	Hotel Admin Create pricing rule. featureGate:
T		(Premium) PREMIUM CUSTOM only.
PU	/api/pricing/rules/:id	Hotel Admin Update rule threshold or adjustment.
T		(Premium)
DE /ap	i/pricing/rules/:id	Hotel Admin Delete a pricing rule.
LET		(Premium)
E		
GE	/api/analytics/summa	Hotel Admin Aggregated KPIs with date range filter.
T	ry	(Premium)
GE	/api/analytics/export	Hotel Admin Stream CSV export of reservations.
T		(Premium)

9.9 Staff Management

Met	Endpoint	Auth Description
hod		
GE	/api/staff	Hotel Admin List all staff accounts for tenant.
T		
POS /a	pi/staff	Hotel Admin Create staff account. Auto-generates

Met	Endpoint	Auth Description
T		username. bcrypt hashes password.
PAT	/api/staff/:id/deactiva	Hotel Admin Deactivate staff account.
CH	te	
DE /ap	i/staff/:id	Hotel Admin Soft-delete staff (preserves audit logs).
LET		
E		

9.10 Super Admin

Met	Endpoint	Auth Description
hod		
GE	/api/admin/tenants	Super Admin List all tenants with stats.
T		
POS /ap	i/admin/tenants	Super Admin Create new tenant. Provisions subdomain
T		immediately.
PU	/api/admin/tenants/:i	Super Admin Approve pending tenant.
T	d/approve	
PU	/api/admin/tenants/:i	Super Admin Suspend tenant. All API calls from that
T	d/suspend	tenant get 403.
PU	/api/admin/tenants/:i	Super Admin Change subscription plan. Feature gates
T	d/plan	update immediately.
DE /api	/admin/tenants/:i	Super Admin Permanently delete tenant and all data
LET d		(cascade).
E		
GE	/api/admin/analytics	Super Admin Platform-wide aggregated metrics. No
T		individual guest data.

10. Risk Register

This risk register identifies and evaluates project risks across infrastructure, security, multi-tenancy, development process, and proposed AI features. Each risk is assessed for likelihood, impact, and mitigation strategy.

ID	Risk Description	Likelihood	Impact	Mitigation Category
R- Multi-tenant	01 middleware complexity delays	Medium	High	Tackle tenantResolver Multi-tenancy and Prisma extension as the very first task in Sprint 1
R- Cross-tenant data	02 leak introduced by a developer query	Low	Critical	Prisma client extension Multi-tenancy al auto-injects tenantId.
R- WebSocket events	03 leaking across tenants	Low	High	All Socket.io rooms use Real-time tenant:{tenantId} pattern.
				QA includes cross-tenant event isolation test

ID	Risk Description	Likelihood	Impact Mitigation Category
R- Feature gate logic	Medium	Medium	Centralised featureGate Security
04 inconsistently applied across endpoints	Medium	Medium	um middleware. Developers never gate in controllers directly
R- Railway wildcard	Medium	Medium	DevOps spikes wildcard Infrastructure
05 sub domain config issues in production	Medium	Medium	um DNS in Week 1. Fallback: path-based routing
R- Railway free-tier	Medium	Medium	Monitor memory usage. Infrastructure
06 memory limit causes service restarts	Medium	Medium	um Health check endpoint. Railway auto-restarts on OOM

ID Risk Description	Likelihood	Impact Mitigation Category
R- Race condition in	Low	High 5-minute DB-level lock Concurrency
07 parking/s unbed		with tenantId. node-cron
booking under		releases expired locks
concurrent load		every minute
R- Dynamic pricing	Low	Medium Wrap pricing engine in Reliability
08 cron job fails silently		retry/catch with
R- Scope creep from	Medium	High Super Admin stories are Scope
09 SaaS complexity	um	Sprint 4. Core hotel
absorbs all sprint		features always
capacity		prioritised first

R- Scope creep from	Medi	High Super Admin stories are Scope
R- Team memb er does	Medi	Medi Klea creates all branches. Git / Process
10 not resolv e git merge	um	um git checkout --theirs
confli cts cor rectly		when teammate version is authoritative
R- bcrypt perfor mance	Low	Low Salt rounds set to 10. Performance
11 under high		Connection pooling
concur rent login		mitigates DB bottleneck
reques ts		
R- JWT secret expos ure	Low	Critic .env is in .gitignore from Security
12 via .env c ommit ted		al project initialisation.
to rep ositor y		Railway injects secrets as env vars

R- Scope creep from	Medi	High Super Admin stories are Scope
R- Pri sma m igratio n	Low	Medi Migrations run as part of Database
13 fails on Rai lway		um CI/CD deploy step.
Postgr eSQL instan ce		Failures block deployment
R- Op enAI API key	Medi	Medi Use GPT-4o-mini AI Feature
14 costs e xceed budget	um	um (lowest cost model).
if AI f eature s		Per-tenant request rate
appro ved		limiting

11. Glossary

This glossary defines all technical terms, acronyms, and domain-specific concepts used throughout the Reservio SRS documentation.

Term	Definition
Activity Log	Audit trail entity recording every significant system action with actor, target, timestamp, and metadata.
bcrypt	Password hashing algorithm with configurable salt factor used in Reservio (salt rounds ≥ 10). Passwords are never stored or returned in plain text.

Term	Definition
Cron Job	Scheduled background task. Reservio uses node-cron within the Node.js process for nightly dynamic pricing evaluation and lock expiry cleanup.
Express.js	Node.js web framework used to build the Reservio REST API. All route handlers are preceded by the full middleware chain.
Feature Gate	The featureGate middleware checks the tenant plan (BASIC PREMIUM CUSTOM) before allowing access to plan-restricted endpoints. Returns HTTP 403 if insufficient.
GitHub Actions	CI/CD pipeline platform. Runs lint + test on every push to main and triggers Railway deployment on success.
Housekeeper	Hotel staff role responsible for cleaning and maintaining rooms. Interacts exclusively with the auto-prioritised task board (US-20, US-21).
Isolation Forest	Machine learning algorithm (scikit-learn) proposed for optional anomaly detection. Identifies unusual booking patterns by isolating outliers in random decision trees.
JWT	JSON Web Token. Signed token issued on login containing userId, tenantId, and role. Every protected route validates the JWT signature and tenant match.
Lock (5-minute)	Concurrency mechanism for parking spots and sunbeds. Guest selection sets status=LOCKED and lockedUntil=now+5min at the DB level. Cron releases expired locks.
Multi-tenancy	Architecture where a single application serves multiple customers. Reservio uses row-level multi-tenancy: all hotels share one PostgreSQL instance, isolated by tenantId on every table.
node-cron	Lightweight Node.js scheduler used for background jobs. No separate process or message queue required. Runs within the Express.js service.
Prisma	ORM used for database access. A Prisma client extension (tenantClient.js) automatically injects WHERE tenantId=? into all queries, preventing cross-tenant data leakage.
Railway	Cloud PaaS platform hosting Reservio. Provides Node.js service, managed PostgreSQL 16, wildcard subdomain support (*.reservio.com), and automatic TLS.
RBAC	Role-Based Access Control. Implemented via roleGuard middleware. Roles: GUEST, RECEPTIONIST, HOUSEKEEPER, TECHNICIAN, MANAGER, HOTEL_ADMIN, SUPER_ADMIN.
React	Frontend JavaScript library. Reservio uses React 18 with Vite. All five portals are served as a single SPA gated by JWT role.
Row-Level	Multi-tenancy model where all tenants share DB tables; isolation

node-cron	Lightweight Node.js scheduler used for background jobs. No separate process or message queue required. Runs within the Express.js service.
Multi-tenancy	enforced by a tenantId column and Prisma middleware scoping every query.
Socket.io	WebSocket library used for real-time features. Tenant isolation enforced by named rooms: tenant:{tenantId}. Events never cross tenants.
SPA	Single Page Application. React/Vite SPA loads once and updates dynamically. All routing is client-side via React Router v6.
Subdomain	Each hotel tenant gets a unique subdomain (e.g. grandhotel.reservio.com). tenantResolver reads it to resolve tenantId.
Super Admin	Reservio platform operator. Has cross-tenant visibility and no tenantId restriction. Manages hotel accounts, subscription plans, and platform-wide analytics.
Tailwind CSS	Utility-first CSS framework used for Reservio responsive UI. Supports darkMode class toggle and custom rv- prefixed design tokens.
tenantId	UUID uniquely identifying a hotel tenant. Present as a foreign key on every database table. Auto-injected into all Prisma queries by the client extension.
tenantResolver	Express.js middleware that runs on every incoming request. Extracts subdomain, queries Tenant table, attaches tenant object to req.tenant.
Vite	Frontend build tool used with React. Provides fast hot module replacement in development and optimised code-splitting for production.
WebSocket	Full-duplex communication protocol. Reservio uses Socket.io for real-time notifications: room status updates, task re-sorting, early check-in alerts, reservation changes.
5-Minute Lock	See Lock (5-minute).

Chapter 12 Future Work

This chapter outlines planned extensions to the Reservio platform beyond the current requirements specification. The features described below are not part of the initial release scope but represent high-value directions identified during design and development.

12.1 Planned Deployment

The platform is architected for deployment on Railway Cloud PaaS, which hosts the Express API, the managed PostgreSQL 16 database, and the Vite-built React SPA as a static site. The following deployment milestones are planned:

- **Production environment:** Full deployment to Railway with wildcard DNS (*.reservio.com) configured via Cloudflare, enabling subdomain-per-tenant routing without infrastructure changes.
- **CI/CD pipeline:** GitHub Actions workflow running ESLint, Jest unit tests, and Prisma migration checks on every push to main, followed by an automatic Railway deploy hook on green build.
- **Environment hardening:** Secrets managed via Railway environment variables; JWT_SECRET, DATABASE_URL, and SOCKET_IO_ORIGIN never committed to source control.
- **Database migrations:** Prisma Migrate used for schema versioning; migration history tracked in the repository to ensure reproducible deployments across environments.
- **Monitoring and observability:** Integration of Railway metrics dashboards with structured JSON logging from the Express API, enabling request tracing and error alerting in production.

12.2 AI-Powered Features

As the platform matures, several AI-driven capabilities are planned to improve the guest experience, reduce staff workload, and protect platform integrity. These features leverage large language models and machine learning techniques integrated directly into the existing multi-tenant architecture.

12.2.1 Integrated AI Chatbot

An AI-powered conversational assistant will be embedded in both the guest-facing portal and the hotel staff dashboard. The chatbot will be powered by a large language model accessed via the

Anthropic Claude API and will operate within the tenant context, ensuring that all responses are scoped to the specific hotel's data, policies, and service catalogue.

Guest portal chatbot

- Answers frequently asked questions about check-in and check-out times, amenities, room types, and hotel policies using hotel-specific knowledge loaded at runtime.
- Guides guests through the reservation process, parking spot selection, sunbed booking, and service marketplace orders via natural-language conversation.
- Handles post-booking queries such as reservation modification requests, invoice questions, and early check-in availability without involving staff.
- Escalates to a human staff member via a real-time Socket.io notification when the query falls outside the chatbot's confidence threshold.

Hotel staff and admin chatbot

- Provides instant answers to staff on platform features, API usage, and housekeeping procedures without requiring access to external documentation.
- Assists hotel admins in configuring dynamic pricing rules, subscription features, and staff account management through guided conversation.
- Generates natural-language summaries of reservation dashboards, occupancy trends, and activity logs on request, reducing time spent on manual reporting.
- Operates with role-awareness: responses differ based on whether the user holds the STAFF, MANAGER, or HOTEL_ADMIN role, enforced via the existing roleGuard.

12.2.2 Fake Booking Detection

A machine-learning fraud detection module will be integrated into the reservation creation pipeline to identify and flag suspicious bookings before they are confirmed. The detector runs asynchronously after a reservation is inserted, allowing the normal booking flow to proceed without latency impact while flagged reservations are held for staff review.

The model will be trained on behavioural signals associated with fraudulent or no-show bookings and will produce a risk score attached to each reservation record. Key detection signals include:

- **Velocity anomalies:** Multiple reservations created from the same account or IP address within a short window, particularly across different tenant subdomains.

- **Identity inconsistencies:** Mismatches between the registered email domain, the guest name pattern, and the billing address associated with the booking.
- **Check-in pattern analysis:** Same-day bookings for high-value rooms with no prior account activity, or bookings made for dates significantly in the past or far in the future beyond typical advance-booking windows.
- **Cancellation history:** Accounts with a high ratio of confirmed-then-cancelled reservations or repeated no-shows across tenants on the platform.
- **Anomalous service orders:** Large service marketplace orders billed to a room reservation that was created moments earlier by a newly registered account.

Reservations flagged as high-risk will receive a riskScore field in the database and will trigger a real-time Socket.io alert to the staff dashboard. Staff can review the flagged reservation and either confirm or cancel it. All detection decisions will be logged in the ActivityLog table for audit purposes. The Super Admin portal will expose aggregate fraud statistics across all tenants to support platform-wide monitoring and model retraining.